

CS Nea Tamodoro:

An android pomodoro Focus
App

Centre Number: 28275

Centre name: Nottingham University Academy of Science and Technology

By Jennifer Hoang

Candidate number: 1469

Contents

1 Analysis	6
1.1 Problem introduction	6
1.2 Initial Concept	7
1.3 Existing solutions	7
1.3.1 Pomofocus	7
1.3.1.1 Main page	7
1.3.1.2 Summary tab	9
1.3.1.3 Rankings tab	10
1.3.1.4 Overview	10
1.3.2 Forest	11
1.3.2.1 Main page	11
1.3.2.2 Timer settings page	12
1.3.2.3 Statistics Overview page	13
1.3.2.4 Overview	13
1.3.3 Focus To-Do	14
1.3.3.1 Main page	14
1.3.3.2 Report Page	15
1.3.3.3 To-do List Page	18
1.3.3.4 Overview	19
1.4 Overall analysis & conclusion of existing solutions	20
1.5 User Surveys	22
1.5.1 Section A: General study habits	22
1.5.2 Section B: Timer app preferences	25
1.5.3 User survey analysis	26
1.6 Objectives	29
1.6.1 Timer functions	29
1.6.2 Pet Functions	30
1.6.3 Task management	30
1.6.4 Optional Features	31
1.7 User Limitations	31
1.8 Potential Solutions	32
1.9 Chosen Solution	33
2 Design	34
2.1 Introduction / Foundational Info	34
2.2 General Structure	35
2.3 Timer Page	37
2.3.1 Description	37
2.3.2 Data Dictionary & APIs	38
2.3.3 Start Timer	39
2.3.4 Stop Timer	40

2.3.5 Pause Timer	41
2.3.6 Skip Timer	42
2.3.7 Theme Settings	45
2.4 Task Management	46
2.4.1 Description	46
2.4.2 Database Implementation	47
2.4.3 Edit Task	47
2.4.4 Delete, Complete & Clear	49
2.5 Statistics	49
2.5.1 Description	49
2.5.2 Data Dictionary	50
2.5.3 Database implementation	52
2.5.4 Streaks	52
2.5.4.1 Data Dictionary	53
2.5.4.2 updateStreaks():	53
2.5.5 Reports	54
2.6 Pet	56
2.6.1 Description	56
2.6.2 Data Dictionary	57
2.6.3 Update Variables	58
2.6.4 Resetting CFQ	59
2.6.5 Calculating Happiness	59
2.7 UI	60
2.7.1 Timer Page	61
2.7.2 Tasks Page	63
2.7.3 Statistics Page	65
2.7.4 Settings/Themes Page	66
2.7.5 Pet Page	67
2.8 Overall Relations	68
3 Technical Solution	69
3.1 Completeness	69
3.2 Summary of Techniques Used	70
3.3 Code	74
3.3.1 MainActivity.kt	74
3.3.1.1 activity_main.xml	76
3.3.1.2 bottom_navbar.xml	77
3.3.2 Timer - TimerFragment.kt	78
3.3.2.1 Variable declarations & imports	78
3.3.2.2 onCreateView()	80
3.3.2.3 startTimer()	85
3.3.2.4 logSession()	86
3.3.2.5 stopPauseTimer()	87

3.3.2.6 skipTimer()	88
3.3.2.7 Notification Management	89
3.3.2.8 updateButtonAppearance	91
3.3.2.9 fragment_timer.xml	92
3.3.2.10 settings_main.xml	96
3.3.3 To-Do List	97
3.3.3.1 TaskItem.kt	98
3.3.3.2 TaskItemDao.kt	99
3.3.3.3 TaskItemsDatabase.kt	100
3.3.3.4 TaskItemRepository	101
3.3.3.5 TaskItemViewHolder.kt	102
3.3.3.6 OnTaskCompletedListener.kt	103
3.3.3.7 TaskViewModel.kt	105
3.3.3.8 TodoApplication.kt	106
3.3.3.9 TasksFragment.kt	107
3.3.3.10 TaskItemAdapter	109
3.3.3.11 NewTaskSheet.kt	110
3.3.3.12 fragment_tasks.xml	114
3.3.3.13 singular_task_item.xml	116
3.3.3.14 fragment_new_task_sheet.xml	117
3.3.4 Sessions & Statistics	120
3.3.4.1 Session.kt	120
3.3.4.2 SessionDao.kt	120
3.3.4.3 SessionsDatabase.kt	121
3.3.4.4 SessionManager.kt	121
3.3.4.5 Statistics.kt	124
3.3.4.6 StatsFragment.kt	127
3.3.4.7 fragment_stats.xml	130
3.3.5 Pet Functions	133
3.3.5.1 Pet.kt	133
3.3.5.2 PetFragment.kt	136
3.3.5.3 fragment_pet.xml	140
4 Testing	145
4.1 MainActivity	145
4.2 Timer	148
4.2.1 Formatting Time	148
4.2.2 Starting Timer	149
4.2.3 Adding Sessions	150
4.2.4 Notifications	151
4.2.5 Settings	152
4.2.6 Themes	152
4.2.7 deleteStats()	153

4.2.8 Updating button appearance & Custom Duration	153
4.2.9 Skip Timer	155
4.2.10 Focus/Break button	157
4.3 To-Do List	157
4.3.1 Add Tasks	158
4.3.2 Updating Tasks	162
4.3.3 Delete Tasks	163
4.4 Statistics	163
4.4.1 Update Streak	163
4.4.2 Update Highest Streak	164
4.4.3 All time Stats	164
4.5 Pet	165
4.5.1 Goal Hour picker	165
4.5.2 Sessions Counter	165
4.5.3 Happiness Meter	165
5 Evaluation	167
5.1 Objectives Met	167
5.2 Possible future developments	168
5.2.1 Feedback	169
5.3 Final Conclusion	174

1 Analysis

1.1 Problem introduction

In today's world, the focus on productivity and education has never been greater, yet distractions have become increasingly pervasive. This introduces the proliferating issue of short attention span among students including myself, who form the primary target demographic for this project.

From my personal experience, concentrating for prolonged periods of time especially during exams has been a great hurdle. There are many times I have struggled to meet deadlines and prepare for tests due to procrastination and a distracting environment. However, later on I've been able to considerably reduce this issue with the Pomodoro method; a studying technique in which students time themselves and only need to focus for that short duration. The short bursts of concentration makes blocks of studying feel more manageable. As I got more used to focusing for 25 minute intervals, I was able to slowly increase the duration I could study at a time, going up to 45 minutes, to even an hour. Through this, I set my mind on making an app revolved around Pomodoro style studying.

Furthermore, in this day and age where remote learning and self-study is becoming more and more common, I believe the use of productivity tools for effective organization is the key to efficiency and success. Even if students are not self studying, organizational tools are still vital. This has become apparent to me since I noticed how often others around me appear to struggle with keeping up with deadlines due to forgetting, or not planning schedules well enough. As a result, I wish to create an app that can also support users with task organization.

Inspired by these difficulties myself and others face, I aim to support students alike in their academic journeys with a Pomodoro timer of my own. The difference between mine and many others will be its gamified experience. Initially, motivation was one of the biggest obstacles in my ability to focus, as well as many others'. The way I aim to combat this is by creating a focus timer that makes the process of studying more enjoyable and rewarding.

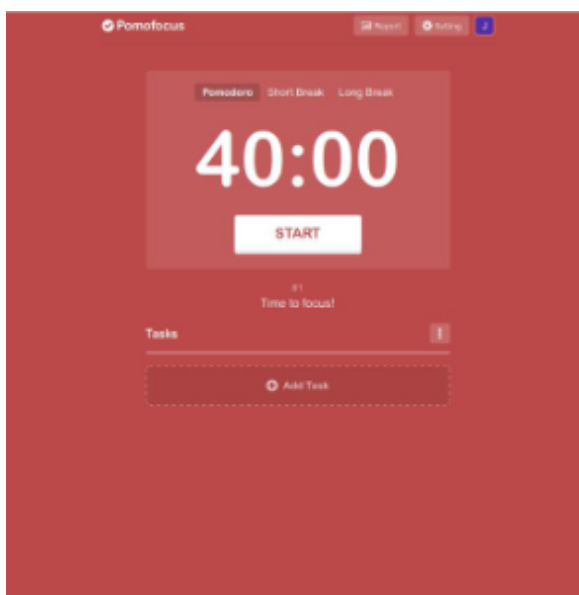
1.2 Initial Concept

My application will be a productivity timer specifically incorporating the Pomodoro technique. Furthermore, I would also like it to have a task management system where students can create and check off tasks to keep organized and on top of their studies. Currently, there are many existing Pomodoro-style timers specialized for studying. However, little to none provide students with motivation through game like mechanics. This is what will be separating my app from many others. The way I aim to do this is by integrating a pet system where users are incentivized to focus, to maintain and care for them.

1.3 Existing solutions

1.3.1 Pomofocus

1.3.1.1 Main page



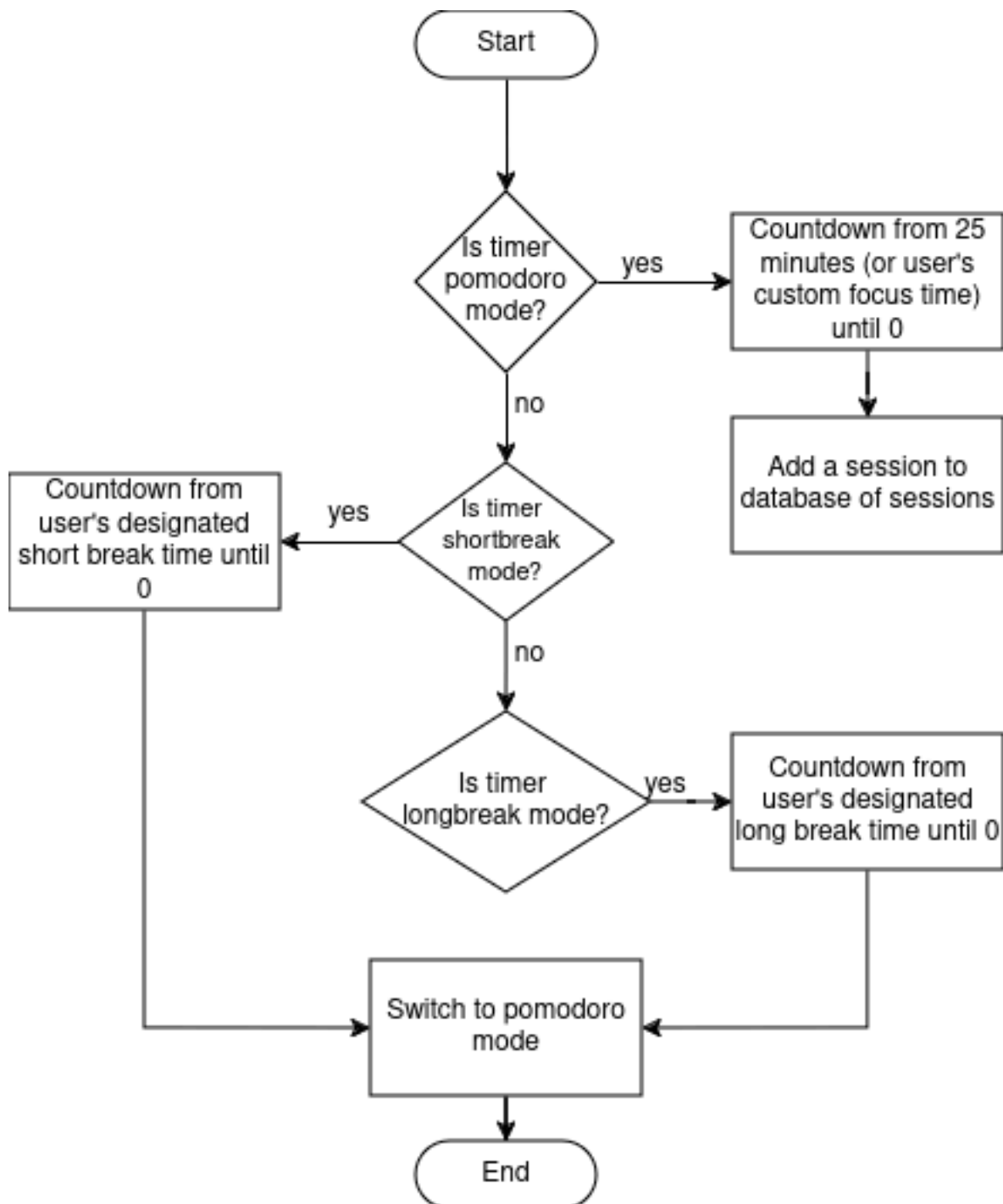
Main colour palette: Muted reds, blues and yellows. Can be changed in themes settings.

Pomofocus offers 3 different timer types- pomodoro, short break, and long break, helping users manage their time during breaks.

Furthermore, there is a feature to add tasks, handy to keep users on track and reminding them of the things they need to prioritize.

As seen, The timer design features a muted colour palette and minimalistic shapes encasing different parts of the page. This blocky style of UI draws users to key features of the timer, while maintaining an undistracting environment.

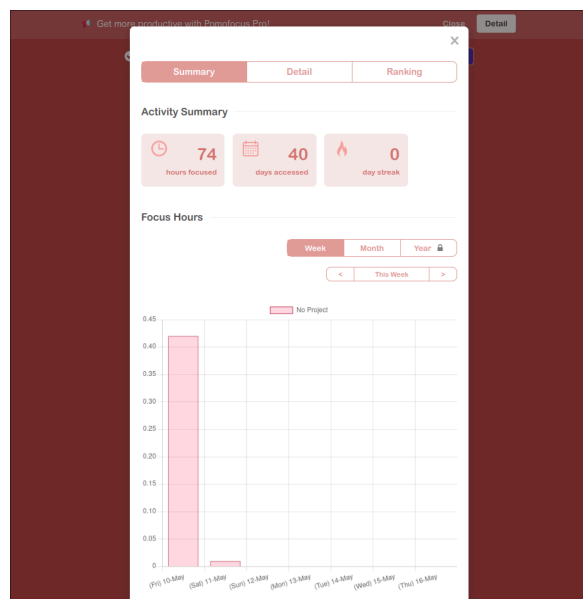
Below is my break down of the timer.



How I think it could have been implemented is the program checks for the timer mode the user is in (break or focus), and counts down from the correct duration. After this, a session is added containing the details such as session duration, given that the timer was on pomodoro mode.

At the end of each break, the timer auto-switches to pomodoro mode as the user should be ready to focus after a break, and vice versa at the end of each focus session.

1.3.1.2 Summary tab



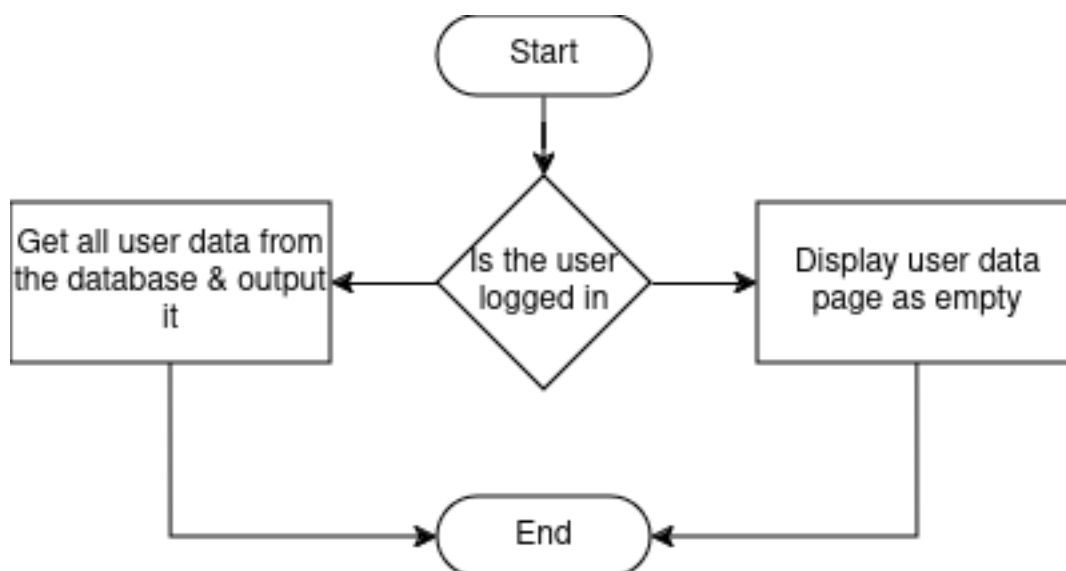
Stats tracked: Hours focused, days accessed, current streak & calendar view showing the frequency a user studies per week. Users can optionally filter statistics based on what subject tag was used on a session.

This summary tab displays the users' stats, something that is a must have for students who want to keep track of how much they study each week.

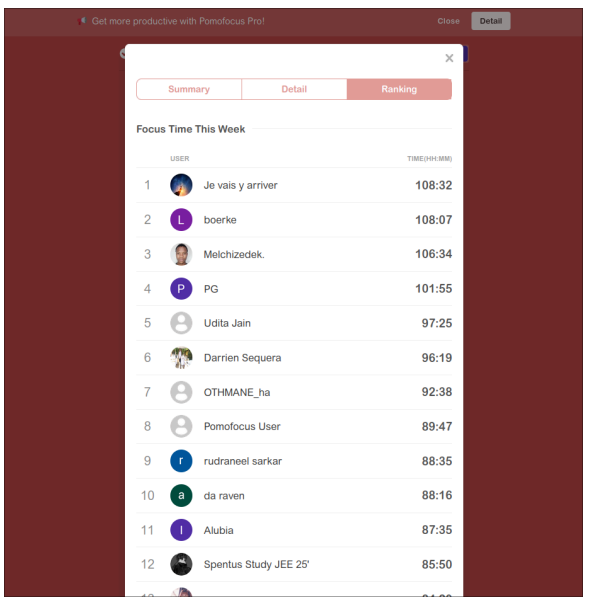
As seen in the image, it is laid out in both a simplistic and intuitive manner, presenting only key stats of the user while giving an unobtrusive option to view user stats in full detail (this just allows users to see the date, time and subject of each session they've logged). I feel that by abstracting most of the user's activity log, it prevents them from feeling overwhelmed or distracted by this page.

Additionally, there is a streak system where the number of consecutive days focused for is logged, increasing the source of motivation.

Below is my modular breakdown of the functions of the summary tab.



1.3.1.3 Rankings tab

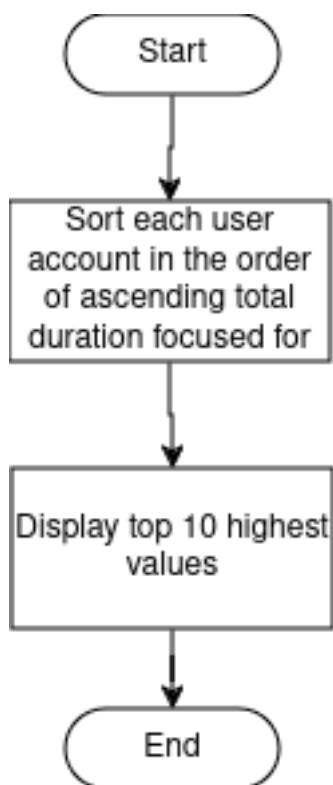


The screenshot shows the 'Ranking' tab in the Pomofocus app. It displays a leaderboard titled 'Focus Time This Week' with a list of users and their focus times. The users are ranked from 1 to 12, with the top user having a focus time of 108:32. The interface includes tabs for 'Summary', 'Detail', and 'Ranking', and buttons for 'Close' and 'Detail'.

USER	TIME(HH:MM)
1 Je vais y arriver	108:32
2 boerke	108:07
3 Melchizedek	106:34
4 PG	101:55
5 Udit Jain	97:25
6 Darrien Sequera	96:19
7 OTHMANE_ha	92:38
8 Pomofocus User	89:47
9 rudraneel sarkar	88:35
10 da raven	88:16
11 Alubia	87:35
12 Spentus Study JEE 25'	85:50

Pomofocus also has a ranking system, featuring a leaderboard where users can compare and compete for the highest study times per week. This leaderboard is displayed in a list from the top to the bottom of the page. Additionally, the times of each user is displayed to ensure the authenticity of a user's rank. As users are ranked on a weekly basis, the leaderboard is constantly changing, encouraging competition.

A modular breakdown of the leaderboard may be:



1.3.1.4 Overview

Pomofocus is a simple focus timer that not only allows users to log their tasks and goals but motivates them through an online leaderboard and aesthetically pleasing user interface. I believe that the constantly changing leaderboards is a key factor in keeping users engaged with this app, with the

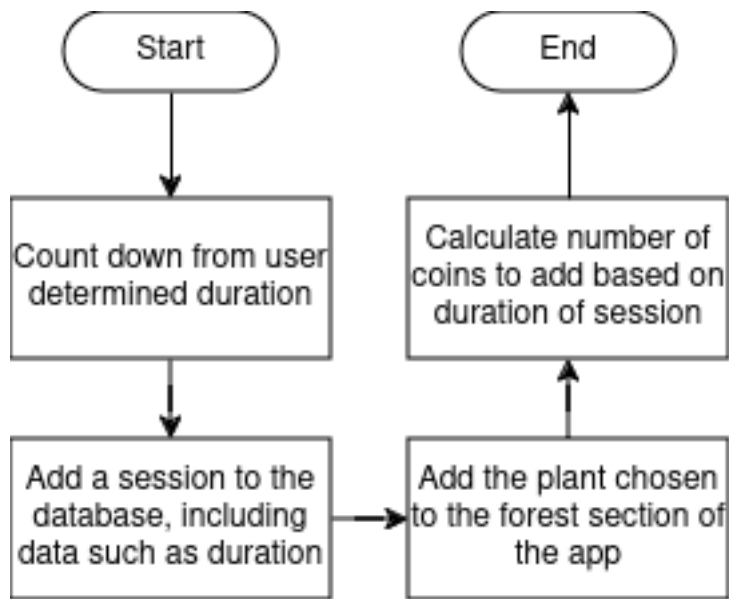
way it promotes competition. While that certainly is one of its most popular features, the app doesn't lack in features that keep users productive despite its addictive nature. This app also crucially provides a custom task list creator, allowing users to keep track of what they need to do. Overall, the research of this app inspires me to add a leaderboard to my own.

1.3.2 Forest

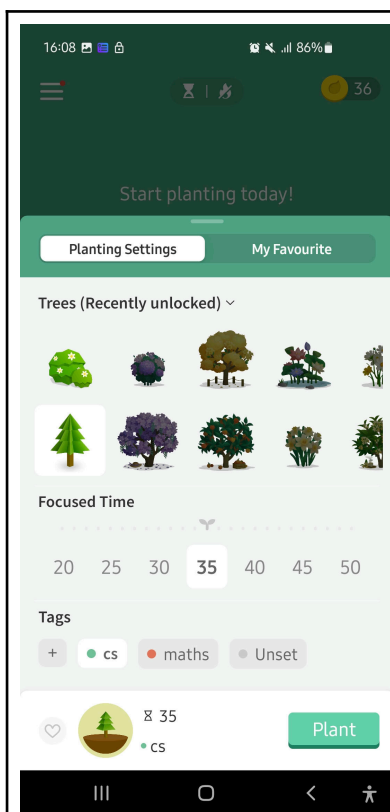
1.3.2.1 Main page

	The main page of the forest app features an incredibly distraction-free, minimalist design. It is apparent that the simplicity of the timer page is done purposefully, as the 3 bars at the top lead you to more features of the app, that are not as closely related to productivity, separating work from play. The primary feature is a customizable timer that includes a break mode. The timer can either be a countdown or a stopwatch timer. This is a particularly useful feature to me, as sometimes things can get in the way of work and we may have to pause or stop a session. Having a stopwatch mode allows me to be more flexible with my studies. Each time a session is completed, users grow a plant and gain coins, which can be used to buy new variations of plants. The longer a user consecutively studies for, the larger the plant grows. If a user fails to complete a session, their plant dies. Another aspect of the forest timer is the ability to lock certain apps. It is highly reported that users get distracted by other apps on their phones, like social media. When users forcibly unlock these apps, their tree is killed.
--	---

Below is a flowchart and break down of how I think the timer page of forest works:



1.3.2.2 Timer settings page



Upon clicking on the plant icon in the center, users can view more details about this specific pomodoro session, for example:

- What category of task you're doing (study, sport, etc)

- What plant you're looking to grow

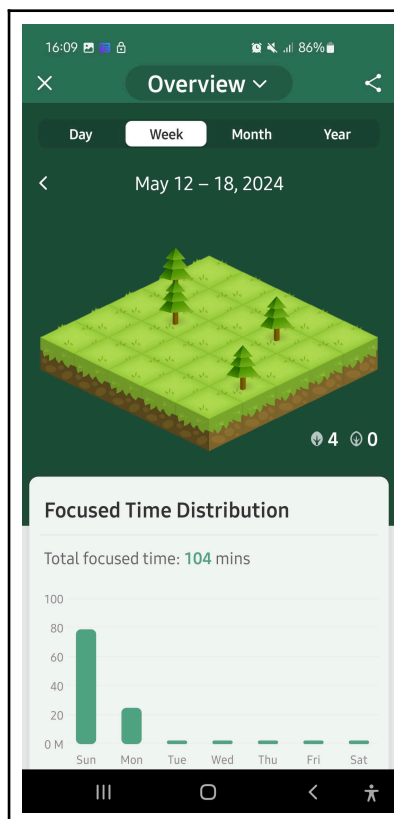
- The time you want to study for

Additionally, you can set the time you want to study for here.

As seen in the image, there are many plants that are both real life plants (great for botanical enthusiasts) and fantasy type plants. Each plant has a unique aesthetically pleasing design. These fun features keep users engaged in focusing for longer amounts of time to get these rewards and unlock new plants.

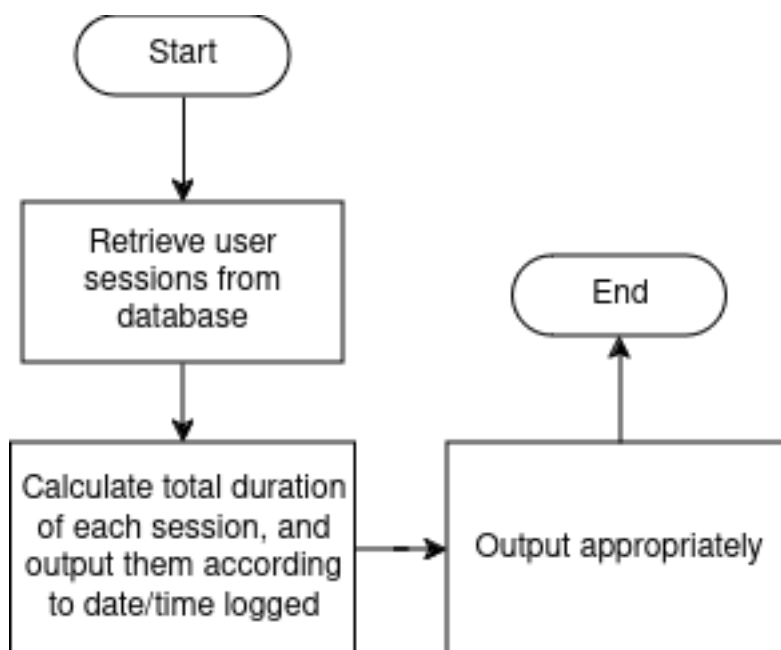
Functionally, this page allows users to input desired variables in order for the timer to execute its functions (such as logging session duration, and choosing which plant to log)

1.3.2.3 Statistics Overview page



Lastly, the app includes a statistics page for users to analyze and share their focus times. The usage data is presented as a simple graph. The bar chart is shown to represent the “Focused time distribution”, which appears to show users the days that they’re most active.

As you can see, there’s also a plot of land where you can view the number of trees planted that day/week/month/year. The way this is displayed is through a plot of land, physically containing each tree you’ve chosen to plant for specific sessions. Many users are left satisfied after seeing a fully flourishing forest covering their land after a week of focusing. This view can be toggled to display land respective to each week, month and even year.



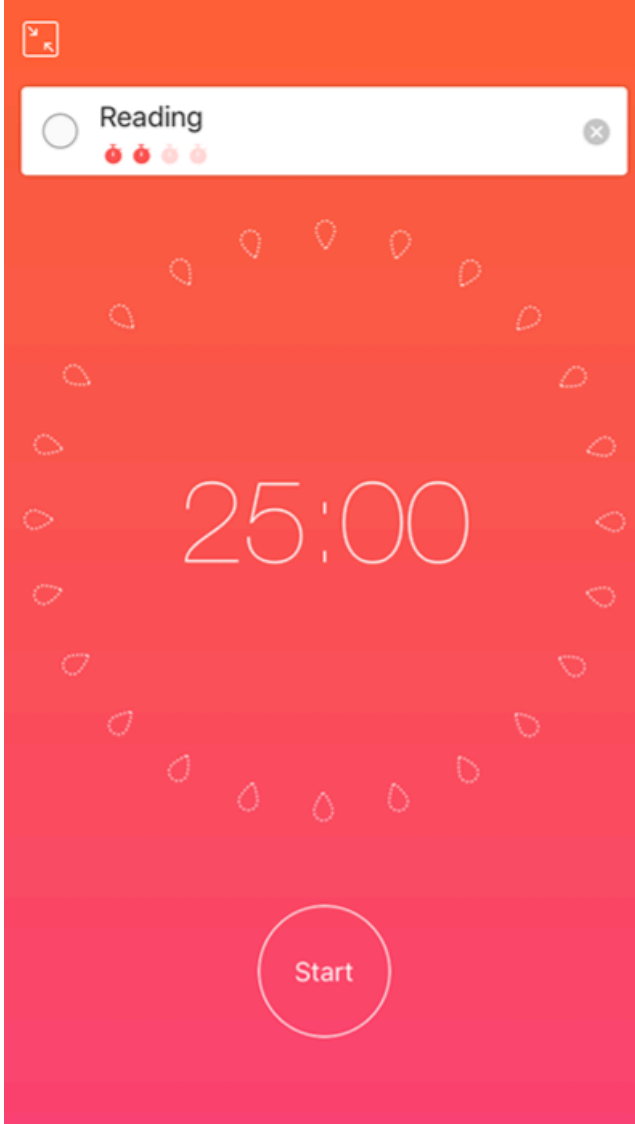
1.3.2.4 Overview

Forest is a Pomodoro app that not only times sessions for users to log their productivity, but also motivates them through the tree planting feature. This therapeutically engaging aspect is what I strive my app to mirror. Another one

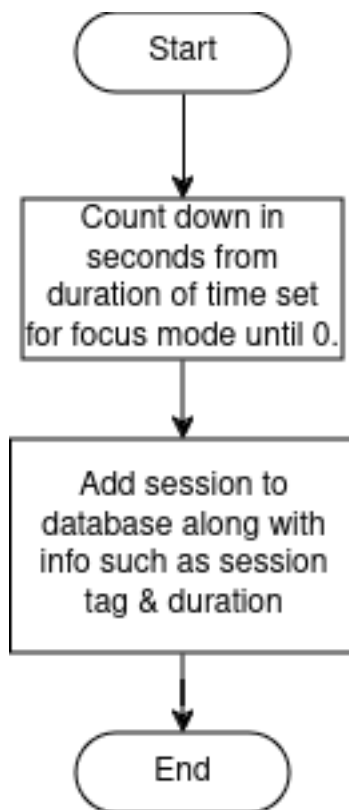
of its most prevalent features is its “Stay focused mode” where once a session starts, the app locks users out of distracting applications on their devices. Overall, I hope to achieve the same atmosphere in my app- encouraging productivity whilst still being aesthetically pleasing and providing an enjoyable user experience.

1.3.3 Focus To-Do

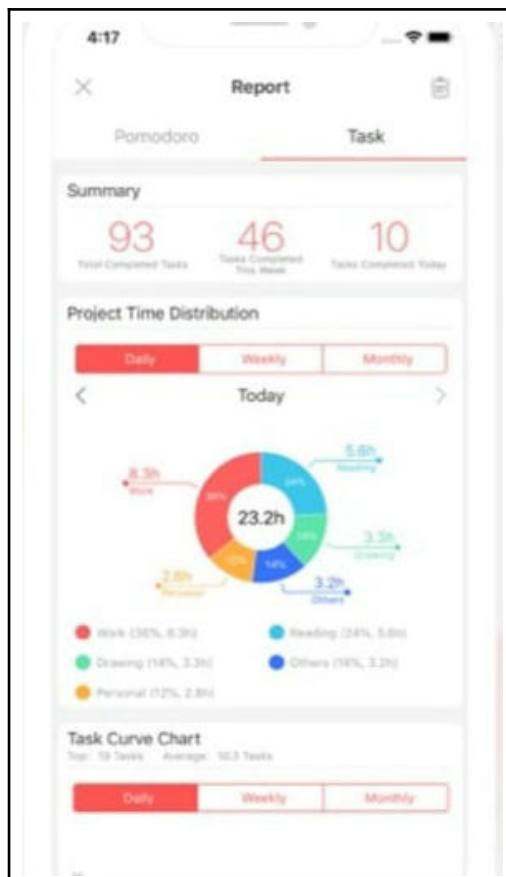
1.3.3.1 Main page

	In Focus To-Do, like other timers, users can set a custom amount of time they want to focus for. In addition to this, users can set tasks they need to complete before each session, a great feature to allow them to stay on track and focus on what’s important. As seen in the screenshot of the main page, Focus To-Do takes on a sleek modern design, providing a distraction free environment. However, a key difference separating Focus To-Do’s timer from others is its custom themes, engineered to create a relaxed environment for users to focus.
--	--

This is a modular break down of the functions I believe to be on this page:

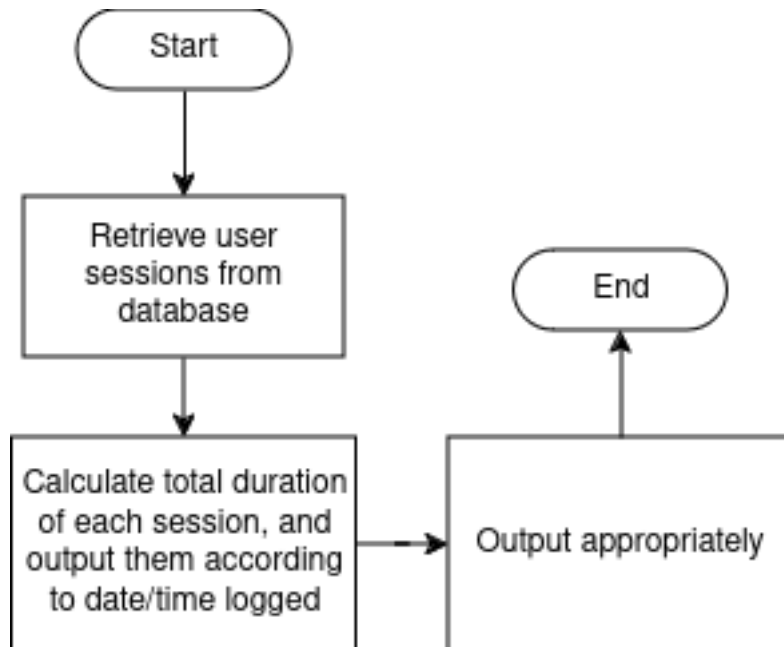


1.3.3.2 Report Page



Focus To-Do's detailed task section can be used for users to organize their plans for work and study. The categories for each of the tasks allow users to fully personalize their app to maximize efficiency. Moreover, not only is it a task manager, but a reminder app; users can set reminders for events such as birthdays.

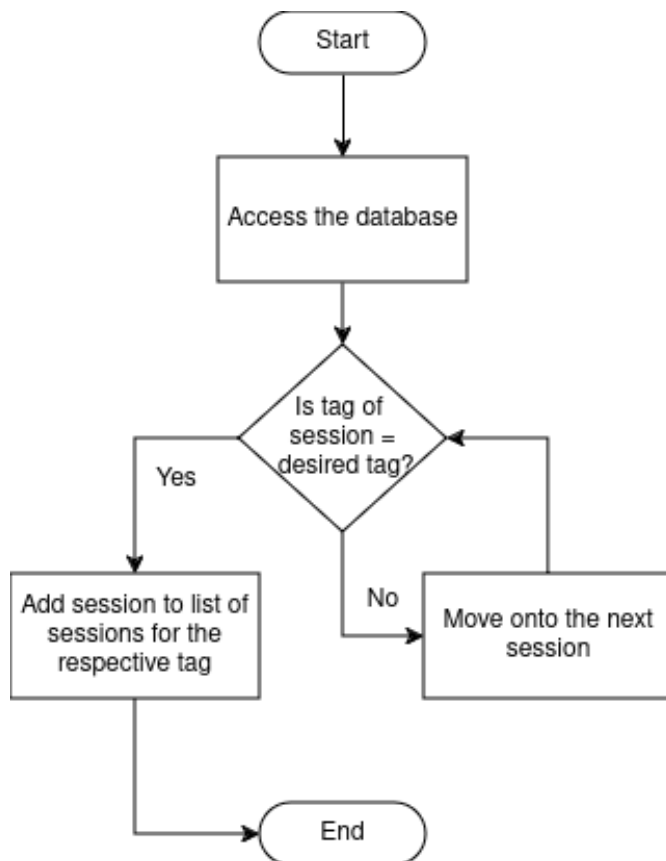
Upon breaking down the stats page for focus-to-do, I found it to contain the same functions as forest:



Furthermore, as seen in the reports page screenshot, user sessions are divided into the categories/tags assigned to each session. This is then displayed onto a graph, clearly showing the ratios of focus duration between the different category. On the next page is a break down of how I believe this has been implemented.

This is how I think the feature allowing users to compare focus durations is implemented:

1. A list for each unique tag will be returned. The total duration of each list will be stored. For each tag, I believe a function as seen below will be executed:



2. Next, these respective total durations are added into a dictionary. It could look something like this:

```
dict = {guitar: 20 , maths: 30}
```

To find the ratio between different subjects studied for, a mathematical algorithm can be applied. I believe this could be something like:

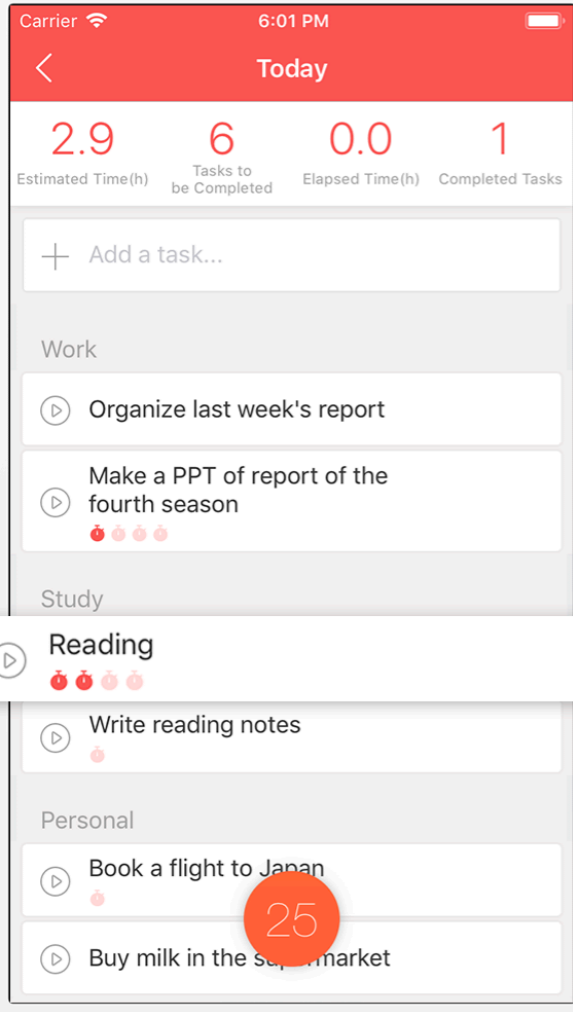
```
totalDuration = dict[guitar] + dict[maths]
ratioGuitar = dict[guitar]/totalDuration
ratioPiano = dict[piano]/totalDuration
```

Using the dict i created, this example should return

- ratioGuitar = 0.4
- ratioPiano = 0.6

and therefore it represents the user has focused on guitar 40% of the time, while they've played piano 60% of the time.

1.3.3.3 To-do List Page



Carrier 6:01 PM

< Today

2.9 6 0.0 1

Estimated Time(h) Tasks to be Completed Elapsed Time(h) Completed Tasks

+ Add a task...

Work

Organize last week's report

Make a PPT of report of the fourth season

Study

Reading

Write reading notes

Personal

Book a flight to Japan

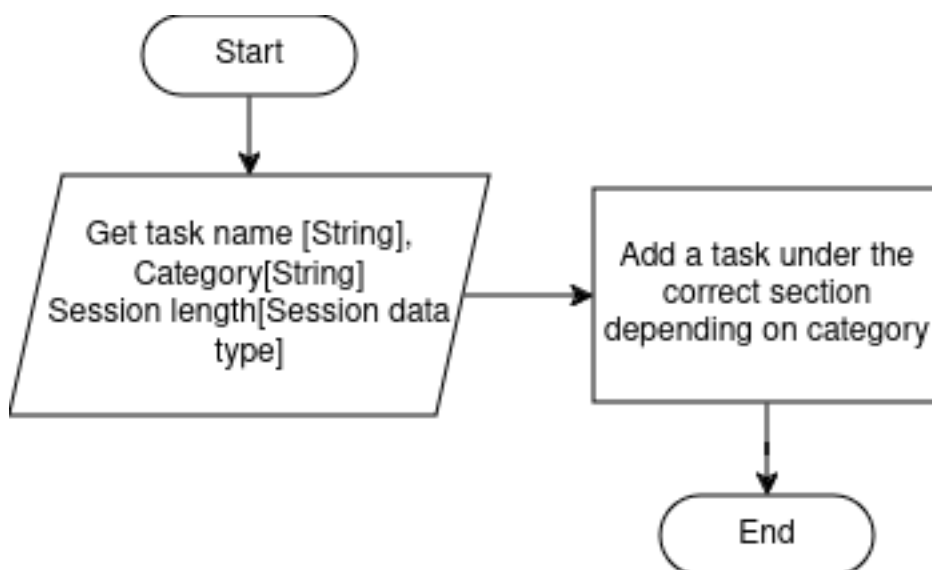
Buy milk in the supermarket

25

Focus To-Do's clean report page displays user activity regarding their focus times in a simple pie chart. It shows the users the ratio and percentage of times they spend on each activity, an incredibly insightful statistic for many. Users can also view the total number of tasks they've completed all time, that week and that day.

The design of the statistics/task page is incredibly condensed and full of information. Personally, I preferred Pomofocus' separation of the tasks section and the statistics section. However, I can see how users may be motivated to complete as many tasks as possible upon being greeted by their stats every time they check their to-do list.

Below is a flow chart breaking down how I believe tasks are added.



1.3.3.4 Overview

Focus To-Do is an easily customizable pomodoro/task manager app that provides users with detailed reports of their productivity. Not only can it be used for studying, but also for general daily life, as users can record things such as shopping lists and events. This way, it is not limited to just students that have trouble concentrating but also people who generally want to increase their organization. This is what makes Focus To-Do stand out from the rest.

1.4 Overall analysis & conclusion of existing solutions

Common features across all 3 apps are their minimalistic interfaces, evidently prioritizing simplicity. I've concluded that simplicity helps users enter a focused state more easily, as they aren't overwhelmed by cluttered visuals or unnecessary options. For example, it's common advice from schools and experts to have a clean desk and environment to study in. I believe this advice can be applied to a focus-encouragement app. However, I'll have to think about how to implement an appealing yet minimal design. Forest especially demonstrates how an app can be aesthetically pleasing yet functional, using its nature inspired visuals to reward productivity in a calming way without being intrusive. This balance is crucial; overly decorative apps can become distracting, while overly plain ones risk lacking user engagement.

All three apps also offer customizable timer settings, allowing users to adapt session lengths to fit their personal productivity needs. This flexibility respects the fact that not all users follow the traditional 25/5 Pomodoro structure; some may prefer longer or shorter work cycles depending on their tasks and mental stamina. I believe such support widens the appeal and audience of the app. Personally, on work-intensive days, I prefer to hour-long of deep work followed by 10 minutes of break, whereas usually I'd prefer a 25/5 structure.

Another widely implemented feature is task management or goal setting, seen most robustly in Focus To-Do. This addresses a key problem faced by many: not knowing what to work on. A built-in task manager ensures that users can plan, sort, and commit to specific tasks before starting a session. Without this structure, people often waste time deciding what to do or bounce between tasks, which undermines focus and effectiveness.

In terms of user motivation, the apps take unique approaches. Pomofocus and Forest includes a leaderboard and daily streak system, using gamification and social competition to keep users coming back. The leaderboard introduces external accountability and can drive motivation through a sense of progress or comparison. Streaks, on the other hand, implemented by Focus To-Do, appeal to users' desire for consistency and reward habit formation—missing a day feels like breaking a commitment. Forest is also unique in the way that trees can be grown each session, to create a forest. It's also important to note that Forest keeps this feature separate from the timer, avoiding distraction during work periods. This idea of compartmentalization (reward and interaction after focus) is something I plan to incorporate through a dedicated pet interaction page.

Statistics tracking is another major element found in all three apps. These tools let users visualize how much time they've spent studying, which subjects they've focused on, and how their productivity changes over time. This supports users in building self-awareness and accountability. For example, a student may realize they've been neglecting a subject. It has been recommended for a-level students to spend 1 hour a day on each subject, so a study tracker definitely is helpful for this use-case. This data can inform behavior change, making the app a tool not just for task execution, but self-improvement.

From this analysis, I've identified key features that will strongly influence my app's development:

- Leaderboard (Note: This feature is responsible for Pomofocus' success and popularity)
 - Introduces a social, competitive element to drive consistent use. Inspired by Pomofocus, this can be a powerful motivator for users who thrive with accountability or friendly competition.
- Task Manager
 - Provides structure to users' study sessions, reducing decision fatigue and increasing focus. Based on Focus To-Do's integration of tasks and timing.
- Statistics viewer
 - Helps users reflect on their habits and make informed improvements. A standard feature across all three apps, and a key tool for users trying to build a routine.
- Minimal UI
 - I noticed compared to other timers, these top 3 popular ones have a sleek simple UI, attracting users and encouraging them to focus without any distractions
- Daily streaks
 - Encourages habit formation through visible progress. Losing a streak creates a light psychological consequence that motivates daily use. (This is scientifically proven)
- Task categories/labels (also a common feature among apps)
 - Helps users analyze where their time is going, and plan with intention. Essential for students balancing multiple subjects or responsibilities.
- Pet activity is separated from the productivity side
 - Inspired by Forest's approach, this design choice keeps motivational elements (like the pet system) from interfering with focus. Users can enjoy rewards and interaction after sessions, preventing distractions during work.
- Custom Timer Settings
 - Supports diverse workflows and productivity needs. Not all users thrive under fixed Pomodoro intervals, so flexibility increases the app's relevance.

1.5 User Surveys

As the target demographic for my app is primarily students and people in education, I sent out a survey with 2 main sections- one to grasp general difficulties students face when studying, and another optional section tailored for individuals who do use Pomodoro timers, to gather insights into their likes and dislikes regarding the app of their choice.

1.5.1 Section A: General study habits

Do you time yourself studying?

Yes: 33%

No: 67%

Of those who times themselves, what app do you use?

Pomodoro: 56%

Forest: 30%

Clock: 10%

Goodtime: 4%

How often do you find yourself unable to concentrate while studying?

A lot: 33%

A little: 67%

Never: 0%

Comments:

I had noticed that a higher amount of people who timed themselves, responded with "A little" over "A lot", compared to students who didn't time themselves. This supports the idea that the pomodoro technique aids concentration.

What is the main reason you struggle with concentrating?

I noticed that there was a particular trend in reasons that hinder each student's focus, so I formed 3 categories:

Organization	Distraction	Motivation
"Lack of management and self-control"	"Thoughts side-tracking"	"Procrastination" x 3 people
"My physical health (being unwell), being tired, how specific the	"My pc being around me, I get distracted by	"Lack of motivation"

work is (if it is a set of questions, I know how long it will take, but if it is independent research for EPQ the length of time is unknown and can be a lot more overwhelming) and whether the environment is distracting/noisy"	anything and everything on it"	
"Literally too much stuff to do its kinda overwhelming sometimes and then i end up only doing a tiny bit of everything"	"Short attention span" x2	N/A
"I have zero time management skills"	"If I am doing work on my phone and I am tired of revising it is easy to stop revising and do something in my phone. So distractions and tiredness"	N/A
"Other plans"	"I get distracted super easily"	N/A
"If I take a break it lowkey ruins the rest of my work flow, like I wont want to get back to work"	"Too many distractions like on pc"	N/A
N/A	"My phone" x 2 people	N/A

Organization: 32%

Distraction: 43%

Motivation: 24%

Comment: It is apparent from these responses that students struggle with prolonged amounts of concentration due to technology. Perhaps this means I should include a feature that penalizes students for side tracking on their phones.

What do you find helps you most with concentrating on revision?

Once again, there was a general trend among answers to this question as well. I organized this into the categories: Timer, Motivation, Environment and Other.

Timer	Motivation	Environment	Other
"Timing myself helps me set a time that I want to study till and this helps me most with concentrating."	"Someone to revise with me"	"Being at NUASt cuz I just can not focus at home"	"Music" x5
"Having a timer or using pomodoro" x 2 people	"Revising In a group"	"Sitting on my desk with my phone down and just getting started"	"Brown noise"
N/A	"Sitting in pindrop silence with someone who will work with me in pindrop silence so we can motivate one another to work and actually work properly"	"Not using my phone as much"	"Ambience e.g nature sounds :)"
N/A	"Learning with a friend"	"Not being home (studying at school or the library or outside)"	"Preparing beforehand (having a shower, cleaning your desk, planning what you will do exactly and taking enjoyable breaks)"
N/A	"Realisation of my impeding future"	N/A	N/A

N/A	"Threatening myself that it needs to be done or my career will be over 😭"		
-----	---	--	--

Timer: 14%

Motivation: 29%

Environment: 19% N/A

Other: 38%

1.5.2 Section B: Timer app preferences

How does using a Pomodoro timer impact your productivity and focus?

"It gives me like an end goal to work towards so studying doesn't feel endless."

"I used to struggle with studying enough so timing it helps me stay on track I guess."

"Helps me keep track of how much work I'm doing" "Every time I put on a timer, I'm forced to 100% concentrate during that duration. It helps me get in the zone."

What features do you find most useful in a Pomodoro timer app?

"A stopwatch function, sometimes I don't have a specific duration I want to focus for, and I'm really just trying to keep track of how much time I spend focusing on certain hobbies."

"I can see how much time I spend on different subjects.. Sometimes I end up only studying subjects I like so being able to keep track really helps me not do that"

"Tracking my stats" x2

"Blocking apps probably"

Are there any features you wish were in your app?

"Forest doesn't let me pause when I'm in a focus session which is super annoying because things can happen or come up while I'm revising, not being able to pause to accurately log my times is frustrating."

"Wish I could keep track of tasks"

On a scale of 1-10, how important is the visual design/interface of a Pomodoro timer app to you?

1	2	3	4	5	6	7	8	9	10
0	0	1	2	0	4	0	8	3	0

The top row is a rating of 1-10, and the bottom is how many people rated that number.

How important are community features, such as leaderboards or a social aspect?

"Not at all, that makes it too cluttered. I want something distraction free."

"Yes I'm competitive. I wanna be able to compete with friends so we can motivate each other, I think Duolingo's also pretty good at that."

"No I will probably get caught up in that"

Lastly, would you be interested in a Pomodoro timer app that incorporates gamification elements, such as a pet system, rewards and challenges?

Yes: 79% No: 21%

1.5.3 User survey analysis

A summary of what I understood from the surveys

The survey revealed key challenges students face while studying, with the most common being distractions (43%), followed by issues with organization (32%) and motivation (24%). Many students reported struggling to concentrate due to the easy access to entertainment, being surrounded by technology. However, those who used Pomodoro timers generally found it easier to stay focused, with 30% using Forest, one of the apps I analysed. A majority expressed that timing their study sessions gave them a clear goal and improved their productivity. Preferred features included time tracking and task management. Additionally, 79% of respondents showed interest in gamified elements like a pet system, while opinions on social features such as leaderboards were mixed. Visual design was moderately important, with most users rating it between 6 and 9 out of 10.

As a result of this survey data, here are some key objectives I've considered including in my project:

- Penalization for Getting Distracted
 - The survey highlighted that distractions—particularly from mobile phones and technology—are a significant challenge for students. Many of my peers mentioned being sidetracked by apps like social media, making it

hard to stay focused. Based on this, I could potentially incorporate a feature that prevents users from switching to other apps during study sessions, similar to the Forest app. If users leave the timer app, they'll either lose progress or fail the session, creating an additional incentive to stay focused.

- ▶ Furthermore, following my original idea of there being a pet, another penalty is the fact that the happiness of the user's pet decreases if they get distracted
- Tracking of user statistics & focus times
 - ▶ Many students in the survey stated that they struggle with knowing how much time they've spent on different tasks. This is the case for me too, for example I'll spend too much time on one subject over the other. Features like tracking total study time, average session length, will allow users to better understand their productivity. Additionally, visualizing this data will help students reflect on their performance, identify patterns, and adjust their study habits.
- Timer pausing
 - ▶ Firstly, I especially want to include this one, as sometimes things can get in the way of my revision. Forest timer is actually unpausable without penalty, making for an inconvenient experience at times. Secondly, from the survey, one respondent mentioned frustration with being unable to pause the timer during a session, especially when unexpected events occur. This is an important feature to include in the app, as it would give users flexibility when life interruptions happen. A timer pause would ensure users can adjust their study time to reflect real-world changes without losing track of their progress.
- Simple, attractive UI
 - ▶ Most people expressed a preference for a clean, distraction-free interface, evident by the fact that there was an average rating of 7/10 on how important the aesthetics of an app is. This definitely supports my earlier market research where I concluded minimal and attractive UI's are a necessity. The app should have an easy to navigate UI that allows users to start a session instantly, distraction free. However, it will also include options for more advanced customization, such as theme changes and the ability to adjust timer sounds or layout preferences, giving users control without overwhelming them.
- Leaderboard
 - ▶ The survey showed mixed feelings about social features. While some students find leaderboards motivating, others prefer to avoid distractions. Therefore, the leaderboard could be an optional feature that users can

choose to engage with. This could provide a healthy, competitive environment for those who enjoy tracking their progress against others while avoiding clutter for users who prefer a distraction-free experience.

- Reward system for each session studied
 - Some people expressed interest in gamification elements, such as rewards or challenges, which tie directly into the pet system. Each session studied could reward the user with points contributing to the pet's happiness and health. This will reinforce positive behavior and motivate users to maintain their focus and keep studying.

1.6 Objectives

From my market research along with the user survey, I have compiled a list of objectives that I wish to meet with my project, as well as the reason behind certain decisions. I have separated them into 3 categories for easier understanding.

1.6.1 Timer functions

Related to features the timer will have, including the storage of user statistics.

- A timer where users can customize durations
 - Each student works effectively with different study times. Although 25-minute intervals of focus are the pomodoro standard, I would like my app to be versatile and customizable for each and every user
- Timers can be paused and resumed
 - From the survey, a student has mentioned the frustrations of not being able to pause a timer. I also agree that pausing is a necessary feature
- A break timer
 - As the survey showed that some students have trouble with getting back on track after a break, I decided an optional break timer would be a convenient feature for some.
- Timers can be skipped after a duration. The duration already focused will be logged
 - This is so that focus times are accurately logged
- When the session ends, users will be sent a push notification
- Each study session is logged, along with the duration of the session
 - To some, being able to monitor productivity is a crucial must. Monitoring user productivity allows them to adjust their habits as necessary. One way I may implement this is by using an SQL style database.
- Users can add labels and categorize the activities they focus on e.g studying, playing an instrument
 - Not all users will be using this app for studying purposes only, perhaps they want to track other activities or hobbies, such as playing a sport.
 - Labels nevertheless will allow students to track how much students spend on different subjects/topics.
- Streak measuring how many days the user has focused consecutively
 - Studies have proven that a login streak adds addictive quality. It is apparent that they strongly influence one's commitment to certain goals. This is demonstrated in the famous Duolingo language app, and its popularity from the streak and reward system.

- Users can change themes
- Users should be able to view the number of sessions completed today, their current and highest streak, and the number of hours focused for today
 - Many other pomodoro timers feature a reports page showing users their stats. My app should display these contents on the statistics and pet page

1.6.2 Pet Functions

How the pet system will function along with its purpose.

- Pets have a happiness meter which increases the more students focus and declines the more students miss their study goal
 - I believe this happiness meter will induce a sense of responsibility in players to take care of their pet by focusing.
- Happiness meter is related to the goal time that the user sets for themselves
 - Different people need to focus for different amounts of times, for different things. Rather than setting a default goal of hours users need to focus for to maintain their pet's happiness, I will allow users to set a custom goal. The aim of the app is not to force users to study through a stressfully strict regime, but to encourage them to better themselves at their own pace.
- Pets will output randomly selected motivational comments while a user is focusing
 - A trend in responses from my survey showed that students struggle with motivation. Perhaps optimistic boosts will help them get through the session.

1.6.3 Task management

The organizational aspects of the app.

- Users can create a task
- Users can edit a task
- Users can check off a task
- Users can clear all completed tasks at the same time

1.6.4 Optional Features

Features that I don't deem to be absolutely necessary, or am not sure will be appropriate given the time and resources I have to give towards this project. Perhaps I will consider them in the future.

- External app blocking
 - Some users may not be able to help but get distracted by the plethora of apps and games on their phones. In app blocking is a great way to remind them to stay on track
- Screensaver mode
 - This will just be something that moves the timer/any other static design feature once in a while. I thought about this feature since OLED screens suffer from screen burn in when an image is static for a long amount of time.
- Users gain a number of coins proportional to the time focused for to spend in the pet shop
- Items/accessories from the pet shop can be bought and equipped by the pet

1.7 User Limitations

This app should serve as a supplement to students' studies, increasing their efficiency when working. Therefore, overcomplicated features with time consuming learning curves is what I'm looking to avoid. The design of my app must be straight forward and intuitive, so that users don't get hung up and distracted, hindering their productivity. This means I will have to design both the UI and functionality of my app in a sleek and concise way.

Furthermore, no matter what form the app takes (web app, android app, ios app), it must use minimal processing power, as students may be running it on phones or weaker laptops used only for school work. My app should also take up a low amount of RAM/memory. I believe this is crucial, as when I myself work, I tend to have many browser tabs and applications due to my workflow. An example of this is when I have multiple tabs up for a textbook, mark-scheme and answer booklet. Although this isn't a problem that has been specified by other students, I personally believe it is a necessary design choice.

1.8 Potential Solutions

Solution	Pros	Cons
Web app/website, just like pomofocus • HTML/CSS/JavaScript • VSCode	• Compatibility across any device • I already have experience with HTML/CSS/Javascript	• Requires internet to operate, this can either result in the user being distracted or not being able to access the app at all
Android app • Kotlin • Android-Studio	• Offline functionality • Can be used anywhere, as it's just an app on your phone • Data can be stored locally to the device, resulting in faster access and improved performance when retrieving and managing user data • I get the opportunity to learn Kotlin and android development, an area of software development I'm highly interested in	• Will only work on android devices, limiting accessibility to iOS users • Phones are distracting to many people, so some may not want a timer in the form of an app on their form, however I can combat this with features penalizing users for using other apps on their phone • I have no experience in Kotlin, so development may be slower and more difficult
iOS app • Swift • Xcode	-Offline functionality • Just like an android app, you have the convenience of accessing the app anywhere as long as you have your phone • Data can also be stored locally resulting in improved performance in management of user data	• Will only work on iOS devices • More difficult to debug as I don't have access to an iPhone to test my app on

1.9 Chosen Solution

In the end I chose to develop a mobile android app, due to the convenience of users not requiring internet to run it. This is an important feature to me as I personally like to switch my internet off to limit distractions while I revise. Moreover, it presents me with the opportunity to expand my horizons in the world of software development, specifically for android devices.

The reasons I stated reflect why I chose a mobile app. As for how I chose android over iOS is simply due to the fact that I have a higher personal interest in android development. Additionally, this is more convenient for me as I own an android device that I could use to debug and test the program more effectively, opposed to having to use a virtual machine to emulate an iOS device.

Choosing to work with android means I will have to learn to store data locally, a few libraries and built in utilities I have decided to look into is Rooms databases and sharedPreferences. On top of this, I will have to choose a suitable format to program the UI in, which I decided to also be XML, as Android-Studio seamlessly integrates XML into the android app development process, by showing real time changes to app pages as I alter the XML code.

2 Design

2.1 Introduction / Foundational Info

Before I started designing my app, I had a look into the most critical components offered by Android Studio that make up an android app. Note that any other components that are highly specific to a particular sub-section of my app not listed here will be listed and explained in those relevant sub-sections.

The ones listed below are components that I will keep referring to across the entirety of my app's design, so I have created a sort of glossary to be referred back to here.

UI components

- Applications
 - An application is a full screen window that's launched as the app is running. This acts as an entry point for users to interact with the app. Each activity has an xml layout that defines UI components in the activity, such as buttons and such.
- Fragments
 - Fragments are pieces of activities allowing for modularity. They can act as a single page in an app.
- XML UI files
 - As briefly mentioned in the explanation of applications, XML files are used to define UI components. For example, text boxes, images and colour attributes.
- TextView
 - Self explanatorily, is a view of text. A text box.

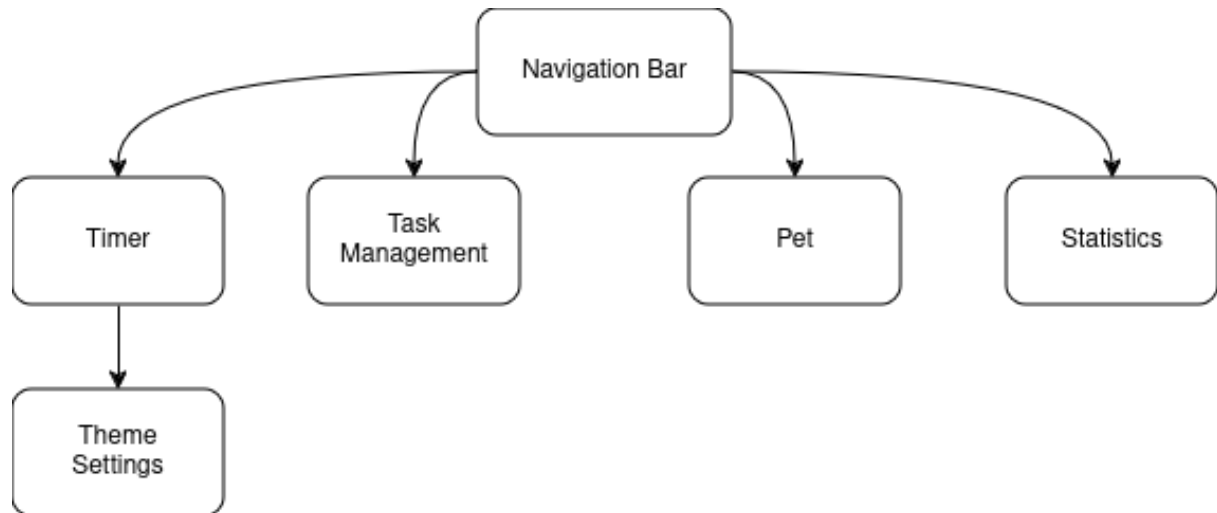
Local Storage Options

- SharedPreferences
 - This a way of storing data locally in key-value pairs.
- SQLite
 - This is a database (using typical column-row format) provided in Android used to store data in the form of a text file
 - The rooms library provides a layer of abstraction over SQLite, simplifying the implementation

In the following sections, I will expand on how I believe I should program specific parts of my app. The sections will also include why I have used specific components.

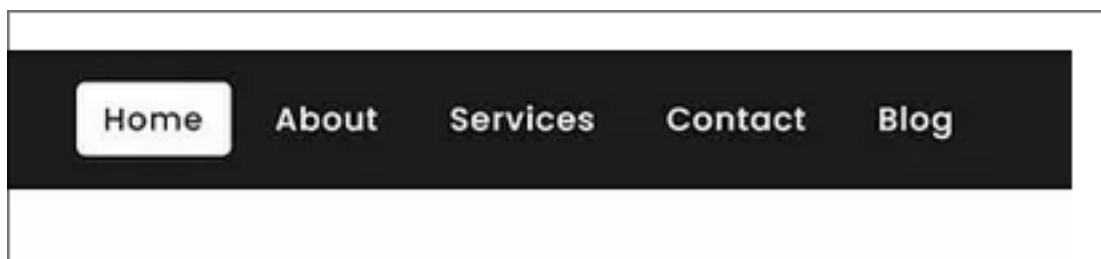
2.2 General Structure

Much like a website, an android app consists of multiple pages the user can access and interact with. This section describes the pages my app will contain, along with the relationship between each page. Below is a modular diagram depicting how different pages can be accessed.



This means, the timer page; task management page; pet page and statistics page can be accessed by a navbar/sidebar in the app. Evident by how they all stem and are accessible from the nav bar.

A navbar I'm referring to is similar to this:



(Image sourced from <https://medium.com/@sahilangrio/building-a-common-navbar-component-in-angular-a-step-by-step-guide-96833dbb2bcb>)

As for the Theme Settings, this can be accessed through a button on the timer page. This is evident in how the theme settings page stems from the time page. See the UI section under Design to view these concepts in place.

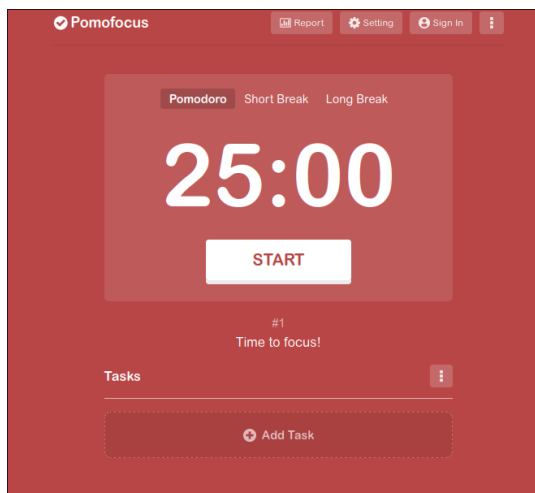
The way I want to implement the app's general structure is by having a singular activity launch upon startup, containing 4 fragments that can be switched between one another. Each fragment will contain each page as described above. The nav bar is what will allow me to cycle between pages.

Upon opening the app, the user should land on the timer page, as it will be the main focus of the app. The app opening on any other page first could potentially serve as distraction to the user. There are a few specific layouts I've considered:

1. A tab structure to switch between the main pages (Tasks, Timer, Statistical reports and Pet page). Uniquely, settings should be accessed from a separate button on the timer page.

I considered having the settings button be accessible by the navigation bar for easy access, however I would like to simplify the UI as much as possible to once again reduce any potential distraction or feeling of clutter.

2. I was thinking of making tasks quickly accessible through a smaller window/section of the timer page (inspired by pomofocus), reducing it to a minor, less customizable aspect of the app:



(unlike the Focus To-Do app which puts heavy emphasis on their task management system).

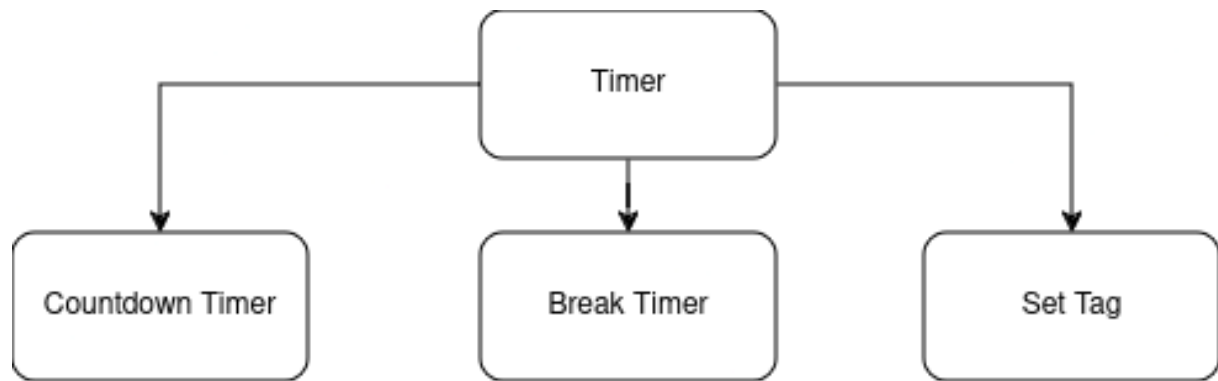
3. As the pet's happiness directly corresponds to the user's frequency of focusing and therefore their overall stats, I considered making the statistics page accessible from the pets page.

This third design idea makes the layout of the app more densely compact, simplifying the overall nav-bar design. However, this has said to be counter-intuitive by a few people I questioned. In order to not cause further confusion, I opted to give statistical reports its own shortcut.

2.3 Timer Page

2.3.1 Description

This section will describe the contents of the timer page, along with the programming logic & classes it will use & access. Here is a modular diagram showcasing functions of the Timer page

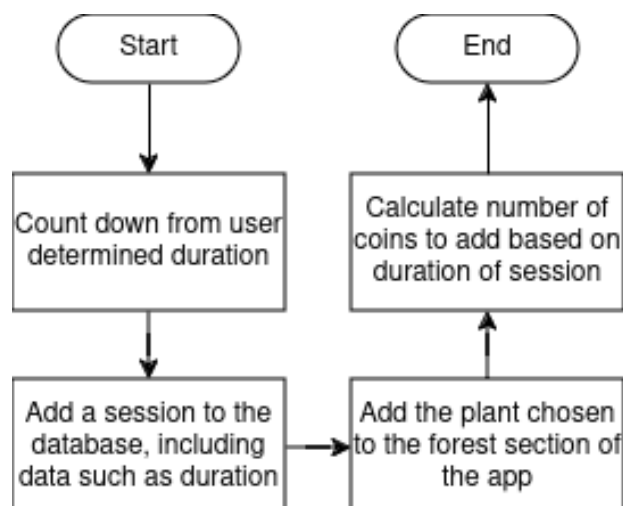


This page will effectively be a Main() function. Users will be able to:

- Start/Skip a countdown timer
- Pause timer
- Set the tag to the subject they're focusing

As data will be stored each time a session is complete, timer logic (such as start/stop timer) should call a Statistics class/object (see Statistics under Design) each time a session is to be logged. I have chosen not to implement the adding of a session within any timer function (again like start/stop timer), and only call this in the Main() class, outside the scope of those timer functions. For example:

```
fun Main():  
    startTimer();  
    addSession();
```



Above is the graph I derived while analysing the forest timer (see analysis). As you can see, the countdown function and session addition are separate processes. I will stick to this design.

2.3.2 Data Dictionary & APIs

The timer page and functions will be the manager of every other aspect of the program such as the statistics and pet classes. The following sections will explain specific functions related to the timer logic, so below is a data dictionary of relevant data, functions & APIs.

VARIABLE	TYPE	DESCRIPTION
initialTime	Long	The duration that a user wants a session to last. By default, this is 25 minutes.
timeRemaining	Long	The time remaining until a session has completed. The timeRemaining before the start of each pomodoro session is set to initialTime.
breakLength	Long	It has the same purpose as the initialTime variable, except it's for breaks specifically. Users can also customize this value. By default, it's 5 minutes.
timerRunning	Bool	Used to determine whether the timer is running or not.
isPomodoro	Bool	Used to determine whether the current countdown is for a session or for a break to ensure break times don't get logged.
timerTextView	Text View	Currently a stand in variable describing the UI component displaying the number on the timer.

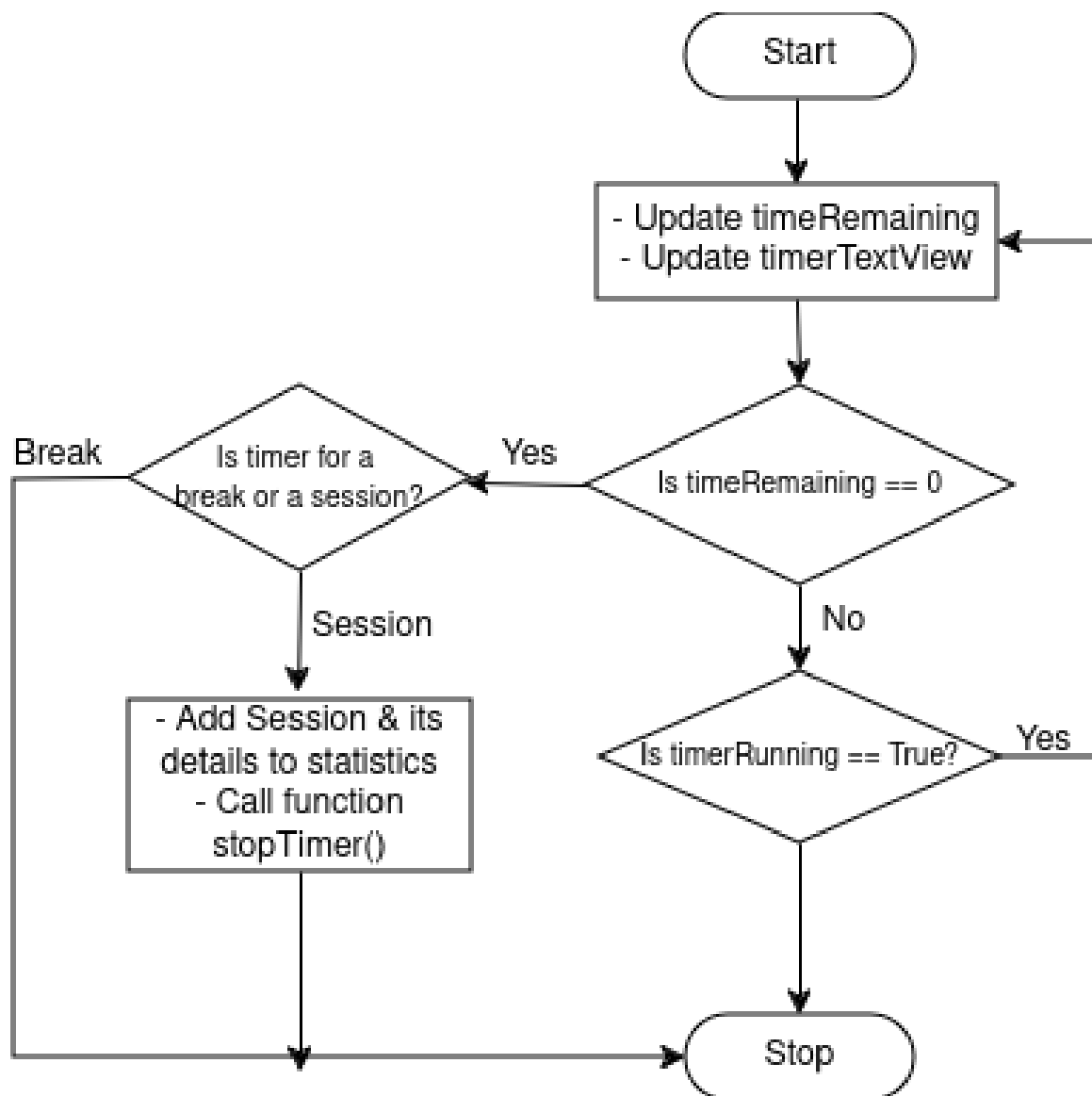
Functions	Description
startTimer()	Starts timer countdown
stopTimer()	Ends timer countdown
pauseTimer()	Pauses timer countdown
skipTimer()	Skips timer countdown

An API that I have decided to use to code this is `CountDownTimer`. Some functions it provides are:

- `onTick(millisUntilFinished)`: This function is called at regular custom set intervals. The argument is the duration it lasts for in ms.
- `onFinish()`: Here, I can define custom events that occur once the countdown finishes, such adding a session to a database
- `start()/cancel()`: Starts/cancels the timer.

2.3.3 Start Timer

Flow Diagram



Description

The flow chart represents what is executed with each tick of the timer, so it is the logic I'll put inside `onTick()` function of `CountDownTimer` API. `start()` will be used to start the timer. The variables `timeRemaining` and `timerTextView` will be updated. This can be:

- `timeRemaining` decrements by 1 second
- `timerTextView` (which will be a visible label on the app displaying the time left for the session) will be updated to show the user how much time is left.

Next, the variables `timeRemaining` and `timerRunning` is checked. If there's no time remaining, then the entire session will be logged (depending on whether the timer was set for a break or a session).

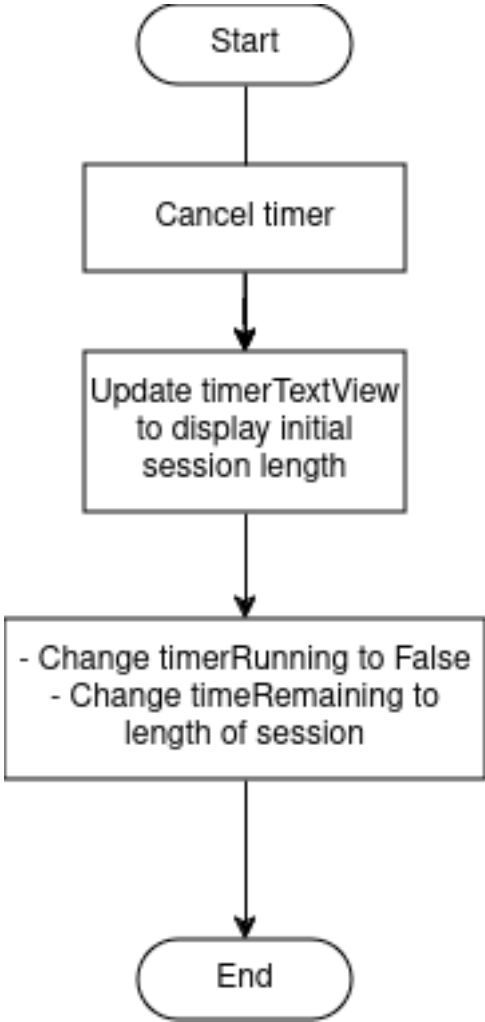
If there is time remaining, and timerRunning is true, timeRemaining and timerTextView are updated accordingly.

In the case that timeRemaining is not 0, and timerRunning == False, it indicates that there has been an interrupt such as a "pause timer" routine called.

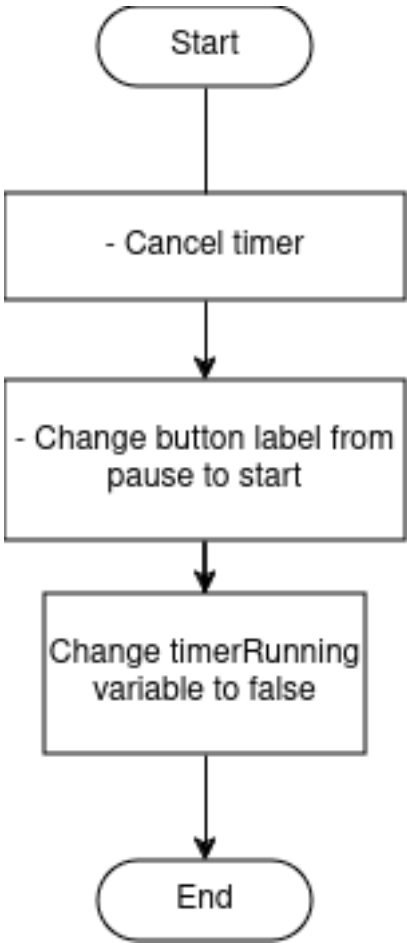
When timeRemaining is 0 and the timer mode is not on break mode (as breaks should not be logged), session details are logged with the session class and its methods, and stopTimer() is called.

This function should be called when a button labelled start is clicked on by the user.

2.3.4 Stop Timer

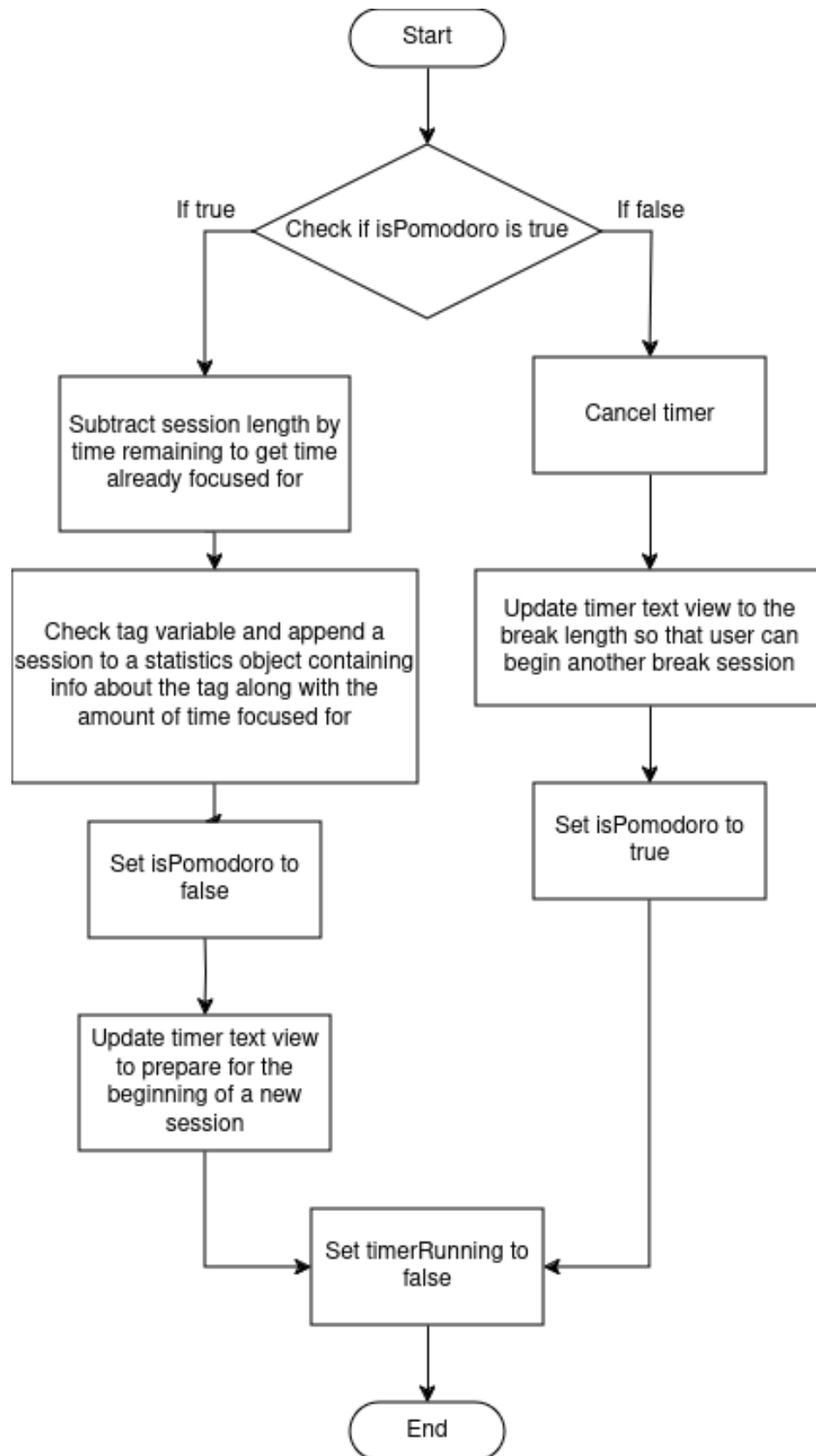
Flow Diagram	Description
 <pre>graph TD; Start([Start]) --> Cancel[Cancel timer]; Cancel --> Update[Update timerTextView to display initial session length]; Update --> Change["- Change timerRunning to False
- Change timeRemaining to length of session"]; Change --> End([End])</pre>	The stop timer function should be accessible the moment the function to start timer has been called. This button should be hidden, until the timerRunning variable is true (because timer can only be stopped once it's started running). When stop timer function is started, the timer is cancelled. Then, the UI is updated, ready for a new session to begin. This could include the timer displaying the default starting time, and the stop timer button being hidden again. In practice, there will be a variable initialTime which is by default 25 minutes but can be changed later on by the user. There will be another variable (timeRemaining) holding the value of seconds remaining of the session. When the timer ends, both the text view displaying the timer's status as well as timeRemaining variable will revert back to the initialTime value, indicating a new session which can be started whenever the user is ready again.

2.3.5 Pause Timer

Flow Diagram	Description
 <pre>graph TD; Start([Start]) --> Cancel[- Cancel timer]; Cancel --> ChangeLabel[- Change button label from pause to start]; ChangeLabel --> ChangeVar[Change timerRunning variable to false]; ChangeVar --> End([End]);</pre> The flow diagram illustrates the process of pausing a timer. It begins with a 'Start' terminal, followed by a process box labeled '- Cancel timer'. This leads to another process box labeled '- Change button label from pause to start'. The next step is a process box labeled 'Change timerRunning variable to false'. Finally, the flow ends at an 'End' terminal.	This method will cancel the timer (as thus far i cannot find a prebuilt pause function), change the button label from pause to start and change the timerRunning variable to false to reflect the timer not running. There is a possibility I may have to pass the parameter of the time remaining just before the timer is paused, and use this value as the remainingTime variable when the timer is started again so that when the start timer method is executed, it uses the time remaining before paused as the starting time that it counts down from.

2.3.6 Skip Timer

Flow Diagram



Description

Check if the timer is running: `timerRunning = True`. If running, then eligible for a skip. Also, whether the timer is in focus mode or not is determined by the `isPomodoro` variable.

If timer was in focus mode:

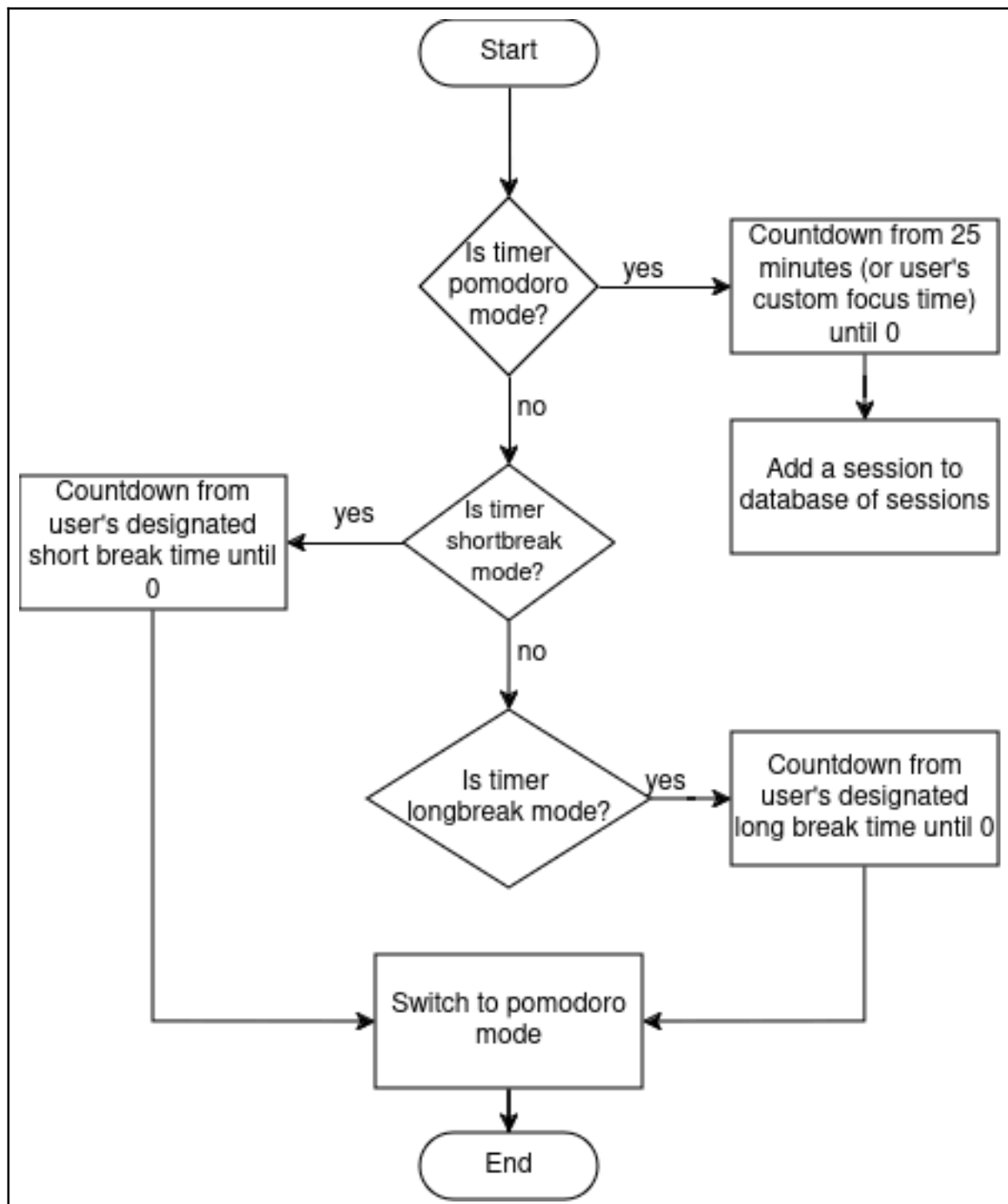
1. First the method will obtain the time that the user has focused for by subtracting the initial session length by the time remaining.
2. Next, the subject tag will be checked. These two values will be added to the session that is to be stored into the statistics object.
3. `isPomodoro` will be set to false so that after a skip of a session in focus mode, the user is automatically directed to a break session.

If timer was not in focus mode:

1. `isPomodoro` suggests the timer is in break mode, so there's no need to log a session. As the break has been skipped, it means that the user is ready to focus, so variables are updated to reflect the beginning of a focus session
2. `isPomodoro` is set to true so that the next session (presumably a focus session) will be logged

The way I used `timerRunning` variable to determine whether the session should be logged or not is influenced by my earlier breakdown of apps in the Analysis section, especially for the pomofocus timer.

As you can see in the flowchart I created before:



As you can see in the diagram, my pomofocus breakdown compares what kind of timer mode the app is on (since theres 3 modes). As my app only includes one kind of timer mode, it only needs to do 1 comparison of whether is it in focus mode or not.

2.3.7 Theme Settings

Themes will be accessed from the timer page, by clicking on an icon. This is demonstrated in the UI section.

Previously, I've experimented in setting themes for a web app using HTML, CSS and JS. I believe the logic and method behind the implementation I used can also be applied to my android app. I had a 'theme' variable stored as a string and used switch cases for my program to know what hex codes to load for the app's colourscheme depending on the name of the theme. Below is the (simplified) code I used in the app I previously worked on, with added comments and removal of code unnecessary to showcasing the logic:

```
const savedTheme = localStorage.getItem('theme');
const theme = savedTheme || 'light';

setTheme(theme);

function setTheme(theme) {

  switch (theme) {

    case 'light':
      document.documentElement.style.setProperty('--light', '#7EA4C3');
      break;

    case 'dark':
      document.documentElement.style.setProperty('--light', '#274054');
      break;

    case 'autumn':
      document.documentElement.style.setProperty('--light', '#274054');
      break;

    case 'retro':
      document.documentElement.style.setProperty('--light', '#274054');
      break;

  }
  localStorage.setItem('theme', theme);
}
```

1. When a theme is picked, it must be stored to local storage, otherwise when the user logs out or closes the tab, the theme is reset to default rather than the user's preferred theme. In this case, the default can be seen to be light mode as seen in the line: `theme = savedTheme || 'light';`. This means the theme being loaded is always the one stored locally. In the case that there is no locally stored theme, the light one is loaded.

2. Using a button component, the function `setTheme(theme)` was called, with the theme parameter being a string, such as in those cases: `'retro'`, `'light'`, `'dark'`, etc.
3. Then, the argument passed through `setTheme(theme)` would be used in a switch case statement to determine the theme chosen.
4. Finally, each time a theme was set/changed, the name of the theme would again be locally stored to ensure it would be automatically loaded up upon each startup of the app.

I believe that this is a suitable way to implement themes into my android app. Through research, I have discovered a popular means of local storage to be `sharedPreferences`. `sharedPreferences` works effectively on small amounts of data (such as user preferences), and allows users to store data persistently even after the app is closed. It is important for me to utilize local storage, as in the past I struggled with getting themes to persist across different devices logged in with the same account, due to themes not being stored correctly.

As `sharedPreferences` takes key-pair values, I plan to have the key be "Currently Picked Theme" while the value stored is the string of the theme name (e.g, light, dark etc). Then, like above a switch case will be applied to find out which colour palette should be applied.

For the theme to be applied across the entire app, every fragment/activity must be refreshed, thus I will have the theme page be launched as a new activity upon clicking the button. Then, when the user changes their theme, the reloading of the main activity containing the main 4 fragments will appear more seamless, as there is a small animation for launching/discarding leaving.

Theme activity launched -> User picks Theme ->

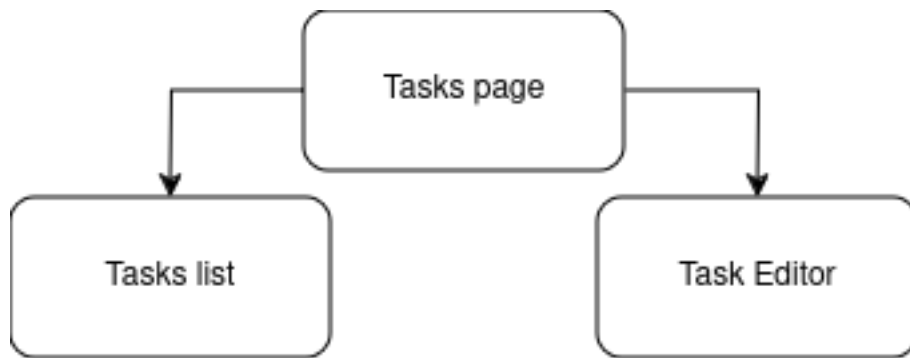
Main activity refreshed -> Theme activity closed -> User can now see main activity after theme page is closed

2.4 Task Management

2.4.1 Description

This section will describe the contents of the tasks page, along with the programming logic & classes it will use & access.

Here is a modular diagram showcasing the features of the tasks page:



On the task management page, I plan for users to:

- Create Task
- Edit Task
- Delete task
- Complete task
 - The 'completed' status will either be True or False, and toggled by checkBox()
- Clear completed task

2.4.2 Database Implementation

I will implement tasks as an SQLite database (likely using the Rooms library), storing the entities for a Task Item. Each task item is an instance of the entity, a record in a database. Below shows how I will implement the TaskItem entity & database along with some examples:

TaskItem(TaskID, Name, Tag, DueDate, Completed)

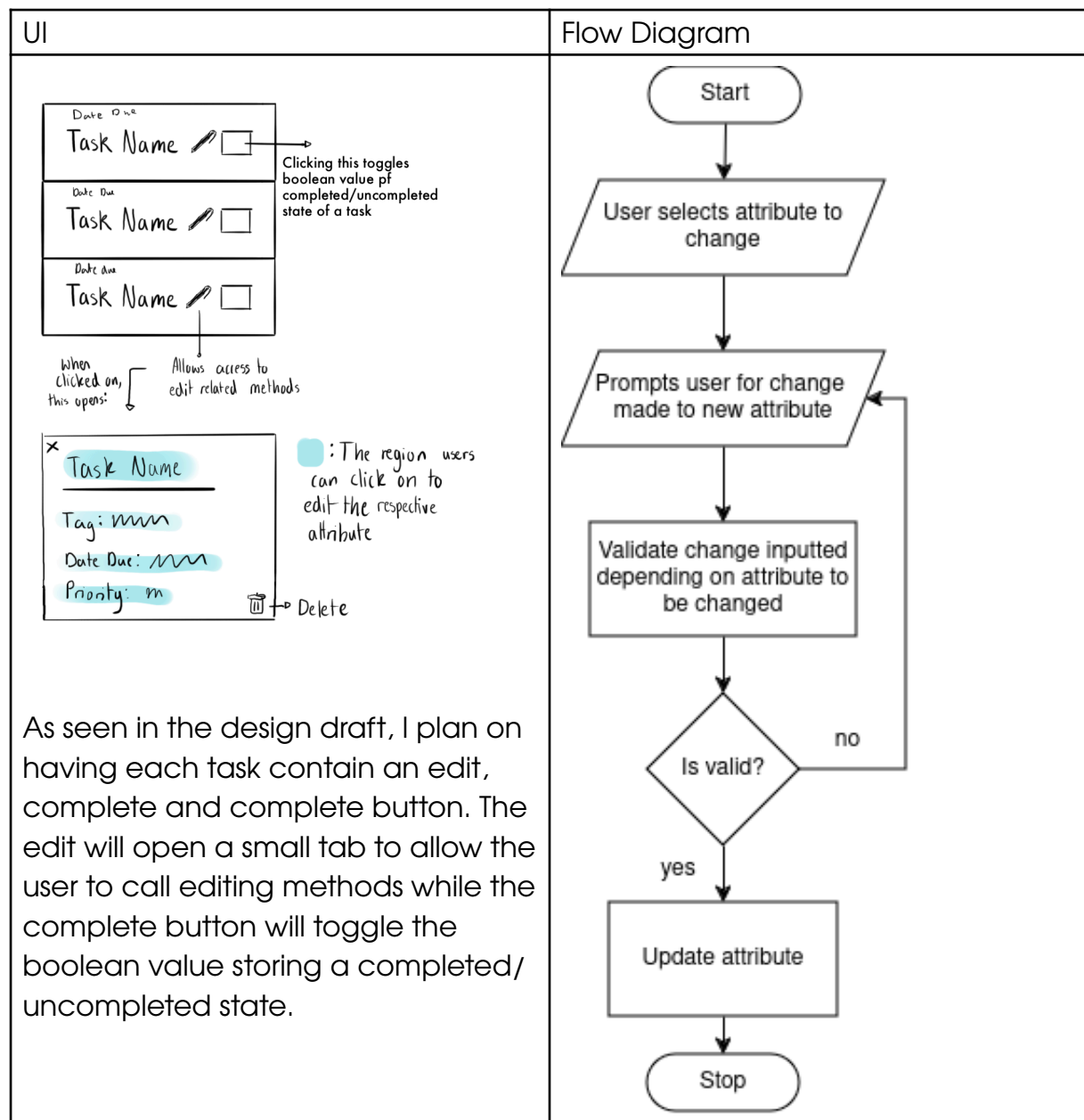
FIELD	TYPE	DESCRIPTION
TASKID	Int	Will act as the primary key, identifying each unique TaskItem
Name	Str	Name of the task
DueDate	LocalDateTime	When the task is due for completion
Tag	Str	The subject that is being focused on
Completed	Bool	True = completed and vice versa

TaskID	Name	Tag	DueDate	Completed
1	Exercise 2	Further Maths	17/07/2025	0
2	Revise Databases	CS	01/01/2024	1

2.4.3 Edit Task

For the edit function, users will input the attribute they want to use. Their input will then be validated, and the new attribute will be updated accordingly. I plan to take these inputs through buttons- each task will have an edit button,

or even be the edit button itself. Depending on the attribute clicked a suitable function will be called. For example, if the user clicks on the name attribute, something like `Edit(name, "Clean Room")` will be called.



2.4.4 Delete, Complete & Clear

1. The delete function will find the task in the taskList and remove it. Below is a python example:

```
if task1.ID in database:  
    DELETE FROM TasksDatabase WHERE TaskItem.ID == task1.ID
```

Realistically, we shouldn't have to check whether the item is in the database or not because the only way users should be able to access this command is by clicking on a button that stores that existing task. If the task didn't exist, the button wouldn't either.

2. The checkbox function will use a checkbox UI element. When clicked, the completed variable will flip. Below is a pseudocode example:

```
checkboxbutton.onclick{  
    task.complete = !task.complete  
}  
}
```

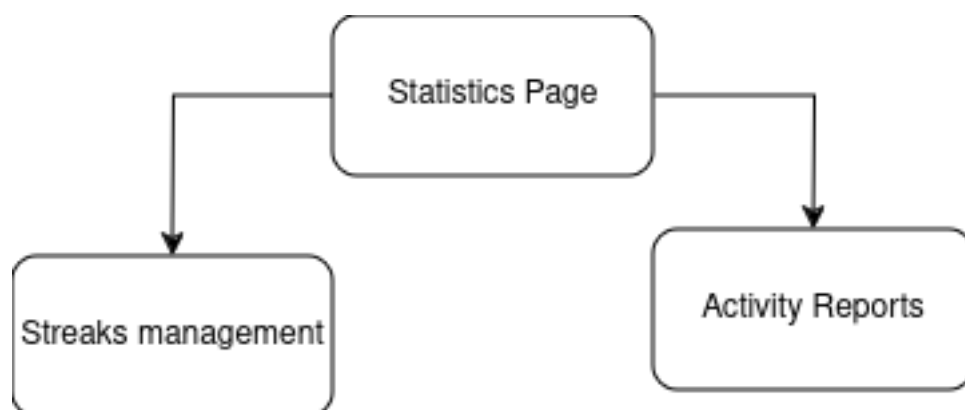
3. I will add a 'clear all completed tasks' button for the user's convenience, so that they don't have to individually delete items after checking them off. Personally, I want to see the tasks I've completed on that day, so I'd want the tasks to stay until I manually clear it. The SQL command would likely be something like:

```
DELETE FROM TaskItemsDatabase WHERE Completed == true
```

2.5 Statistics

2.5.1 Description

This section will describe mainly the contents of the statistics page, along with the programming logic and classes it will use. The modular diagram below shows the features of the statistics page.



I thought of either:

1. Implementing statistics as a singleton object containing a mutable list of the user's focus sessions. Each session is represented by:

- Length of session [Int]
- Date/time completed [Long]
- Category of session [String]

One way I can do this is by creating a user defined data type called session, and have each session type variable created be stored in the list or database called allSessions. After some research, I decided not to go with this style of implementation, as it would result in a difficult to query, complex list. I opted for a database instead, as seen below.

2. Implement statistics by creating an SQLite database whose data is handled by the class Statistics.

Below is a class diagram of Statistics:

Statistics
- currentStreak - lastSessionDate
+ addSession() + getTotalSessions() + getTotalDuration() + getAverageDuration() + deleteStats() + updateStreaks()

I will create a class for statistics despite only instantiating and using a single object of this class. I chose to do this as by encapsulating statistics-related functionality within a dedicated class, the modularity of my program is increased. As a result, I believe this will make it easier for me to maintain, understand and debug the code relating to statistics without affecting other areas of my program. Furthermore, it is to be noted that the functions within Statistics class will be called by the timer page to manipulate user data according to the timer status.

2.5.2 Data Dictionary

Below is a data dictionary followed by function descriptions (not including streaks related functions, that has its own section!):

VARIABLE	TYPE	DESCRIPTION
allSessions	List or Database	A list or database of every session variable created. Each item in the list or row is a variable that's of session type

currentStreak	Int	The current consecutive day in a row the user has logged a session for.
lastSessionDate	LocaDate	The date of the last session recorded

Function	Description
addSession()	Adds a session-type variable to the list allSessions
getTotalSessions()	Returns the number of items in allSessions list
getTotalDuration()	Returns the sum of minutes-property stored in each session in the allSessions list.
getAverageDuration()	Returns value from getTotalDuration() ÷ getTotalSessions()
deleteStats()	Deletes all users stats for when they want to completely wipe their statistics. Especially useful for debugging.

Additional info:

- All attributes will be private to ensure sessions cannot be directly altered from outside the statistics class, reduces chances of data corruption
- All the methods I have listed will be public to allow other parts of the program such as the timer functions to interact with the statistics class
- Will likely have to use Rooms library to implement SQLite

Reports of user statistics will consist of the total number of sessions, total duration focused for, and the average time per session. These stats will be calculated through the getTotalDuration() and getAverageDuration() functions. Variables and methods related to streaks are explained in *Statistics* sub-section.

2.5.3 Database implementation

An example of how i hope the database would look:

Session	Tag	Duration	DateCompleted
session1	"Piano"	120	209384342
session2	"Guitar"	60	292834482

Each row represents a session. There are 4 columns describing each session:

- Automatically generated session name (e.g session1,2,3 etc) (Likely string)
- Tag (String)
- Duration in minutes (Int)
- Date completed in milliseconds since unix epoch (Long)

Note: Unix epoch is a date (1st January 1970) that is often used as a reference to store date and time, in the format of date/time since the unix epoch. If I choose to implement an SQL style database, some commands I may use will be: let Session be the name of the database:

- SELECT * from Session [This should take every single individual session from the database]
- DELETE from Session [Deletes all sessions from the database, clearing the user's data]
- SELECT * FROM Session WHERE tag = :tag [Will select all sessions of a specific tag (such as "Guitar"). :tag will be used in kotlin as a variable containing a tag name]
- INSERT INTO Session VALUES(newSession)

2.5.4 Streaks

Another feature I'd like the statistics class to take care of is the daily streaks. Through research, I've deduced that the main ways dates in java/kotlin are stored are:

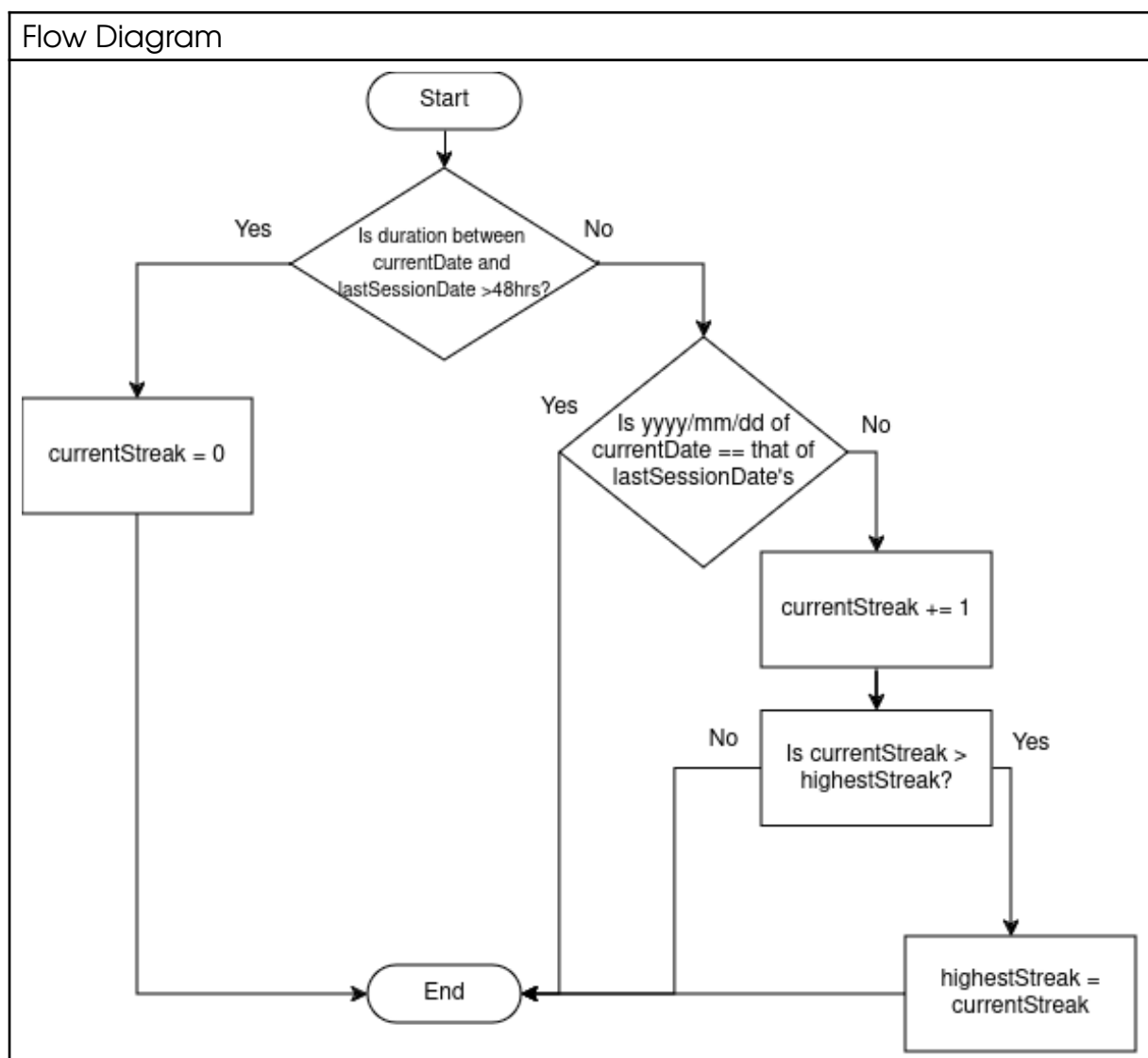
- Long numbers storing the number milliseconds that have elapsed since the Unix Epoch (00:00:00 UTC Jan 1st 1970)
- LocalDateTime (from java java.time package) which stores dates using the components Year, Month, Day, Hour, Minute, Seconds & Nanoseconds. For example:
 - LocalDateTime.of(2024, Month.JULY, 17, 16, 0, 0) represents July 17th 2024 at 4pm

Streak data will likely be stored using sharedPreferences which does not support LocalDateTime. So, it's most probable that I'll store the date as Long, and convert to LocalDateTime when required.

2.5.4.1 Data Dictionary

Variable	Description
CurrentStreak: Int	Current streak number
HighestStreak: Int	Highest streak obtained ever
lastSessionDate: Long	The last time a session was recorded, stored in epoch millis
currentDate: LocalDate	The current date
duration: Int	Duration of time between currentDate and lastSessionDate in hours

2.5.4.2 updateStreaks():



Python Example

```
def updateStreaks(self):
    if duration < (48):
        self.currentStreak = 0
    else:
        if currentYYMMDD == self.lastSessionYYMMDD:
            return 0
        else:
            currentStreak += 1
            if self.currentStreak > self.highestStreak:
                self.highestStreak = self.currentStreak
            else:
                return 0
    return 0
```

Description

===Before streaks method is called

- Each time a session is logged, lastSessionDate will be stored in localDateTime format.

===

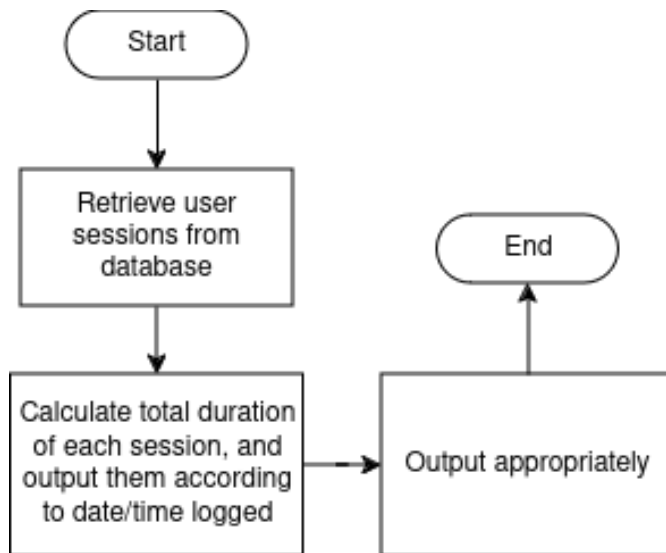
1. The duration of time between the current date and lastSession date is compared. If it's greater than 48 hours, the streak is lost. currentStreak is set to 0.
2. In the case that the duration isn't over 48 hours, we must make sure not to increment streaks value if more than 1 session has been logged in one day. To do this, I'll compare the year, month and day components only of the current date and lastSessionDate. If they're the same, then streaks are not incremented. Else, streaks is incremented by 1, and compared against highestStreak. If currentStreak is greater than highestStreak, then highestStreak is replaced.

2.5.5 Reports

All time stats:

- Total Duration is returned by finding the total amount of seconds in each session-type variable in the allSessions list/database
- Total Average duration is calculated through total duration ÷ number of items in the allSessions list

These functions I included were again heavily influenced by my prior breakdown of report/statistic pages in Analysis. This can be seen in the Focus-To-Do & Forest reports flowchart:



You can see I'm implementing reports the same way as seen in the breakdown from forest.

Weekly stats:

I want to be able to show users the statistics of their current week. In order to find the current week, I have found that TemporalAdjusters can be used to find the start and end of a week (Monday-Sunday). A useful source that has helped me understand the use of temporal adjusters (as well as other aspects of the java.time library) is:

<https://developer.android.com/reference/kotlin/java/time/temporal/TemporalAdjusters>

After determining the dates of the current week's time period, all session data can be retrieved given that they fall within this period. Then the same algorithms used in calculating total duration and total average can be used to give the user a report on their study habits this current week.

Other notes:

- I may want to add a feature that lets me view other weeks, not just current week
 - Perhaps implement a binary search to speed up searching process of the first day of the week of a specific week to find data of the following 7 days, as there is such a huge possible number of sessions

-Or, I could have arrow keys pointing to the next and previous week. When the user presses for example the back arrow (to traverse to the previous week's data) the button will call the respective function that changes the period of time for data we're looking for (in this case to 7 days prior)

EXAMPLE:

<—	Week 2	—>
Data	Data	Data

Let period be: 21st to 27th July

After clicking left arrow:

<—	Week 1	—>
Data	Data	Data

Period shifts 7 days prior to: 14th to 20th July

As you can see, the period of time being focused on is automatically binded to buttons. Each time the user exits the statistics page however I plan for the page to be automatically reset to the current week. This is just an optional side thought, though.

2.6 Pet

2.6.1 Description

This section will describe the contents of the pet page, along with the programming logic & classes required for this page.

The pet page will:

- Display an image of the pet
- Display the happiness meter at a calculated level
- Show a weekly goal and let you know if it's being met

Overall, there should be little user interaction on this page as it's mainly outputting data. Below is a pet class diagram:

Pet
- happiness - weeklyGoal - approxDailyGoal - currentFocusQuota - onTrack - studiedToday
+ updateVariables() + calculateHappiness()

The pet's happiness will be affected by the following factors:

- Whether the user is "on track" in achieving their weekly goal
- Whether the user has studied that day or not

1. weeklyGoal is divided by 7 as there are 7 days in a week. this will be stored in the 'approxDailyGoal' variable. This variable is used to determine whether the user is on track towards achieving their goal or not.
2. There is a target 'total hours that should've been focused for already' as the week progresses. After each day, the target goal increments by the value of the approxDailyGoal. So for example, given that the weekly goal is 7 hours, Wednesday's target number of hours focused should be 3 hours because $7/3$ is 1, and Wednesday is the 3rd day of the week so $1*3$ is 3. If the total hours focused this week has exceeded this target goal, then the user is said to be on track towards achieving their goal. The variable name for this value will be currentFocusQuota.
2. The happiness meter will be at maximum each day the user has:
 - Exceeded their currentFocusQuota
 - Has studied any amount that day
3. A point off the happiness meter is deducted when:
 - The user has not met their currentFocusQuota
 - Has not studied that day

Examples of point system in context:

- User has reached/exceeded their currentFocusQuota that day, but however has not studied that day. This can be achieved when the user studied extra the day before and covered all the hours necessary for today. The user then will have 2 points.
- User has not exceeded their currentFocusQuota and has studied that day. They will have 2 points.
- The user has not met their currentFocusQuota and has not studied today. They will have 1 point.

2.6.2 Data Dictionary

VARIABLE	TYPE	DESCRIPTION
happiness	Int	An integer from 1-3, 1 being lowest happiness and 3 being highest happiness.
weeklyGoal	Long	A weekly goal number of hours that the user wants to focus for. This value is stored locally and will persist until the user changes it.
approxDailyGoal	Long	$\text{weeklyGoal} \div 7$. Will be used to determine whether user is doing enough work daily to achieve their goal. Whether they're on track or not will impact the pet's happiness. This variable is abstracted from the user's

		knowledge as it'll only be used to calculate pet happiness.
currentFocusQuota	Long	approxDailyGoal x No. of days into the week. For example, if it were Wednesday that's approxDailyGoal x 3. This is a rough guide that'll be used for the app to check if the user is on track of achieving their goal. This variable is abstracted from the user's knowledge as it'll only be used to calculate pet happiness.
onTrack	Bool	This will be a boolean value. It's calculated by: totalHoursThisWeek >= currentFocusQuota. This represents whether the user has consistently studied enough this far into the week to reasonably achieve their goal.
studiedToday	Bool	If hoursFocusedToday > 0, studied today returns True.

Functions & Descriptions

Function	Description & Python example
updateVariables()	Resets happiness meter to 3 allowing happiness to be recalculated each day
resetCFQ()	Sets currentFocusQuota to 0 at the end of each week.
calculateHappiness()	Calculates the happiness of the pet given conditions of the user's app usage.

See below for logic behind how those functions will be implemented.

2.6.3 Update Variables

At midnight of each day, a function to update variables will be called to set happiness to 3 (which is the maximum happiness). It is set to 3 because for each criteria the user fails to meet, the meter will decrease by 1 when the calculateHappiness() function is called (which is straight after updating variables and various other events too). It also increments the currentFocusQuota by the value in approxDailyGoal.

```
def updateVariables():
    self.happiness = 0
    self.currentFocusQuota += self.approxDailyGoal
```

2.6.4 Resetting CFQ

This function will be called at the start of a new week. It will set `currentFocusQuota` to 0 to prepare for the next week. A reset will always be immediately followed by an `updateVariables`.

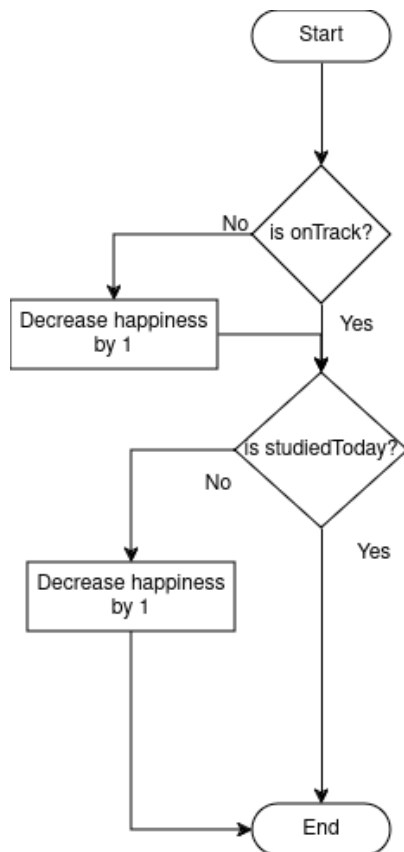
```
def resetCFQ():
    self.currentFocusQuota = 0
```

However, depending on the final product, I may decide to not use this function and instead write `self.currentFocusQuota = 0` where required. I'll mainly use this function for debugging purposes initially.

2.6.5 Calculating Happiness

If the user isn't on track, happiness decreases by 1. If user hasn't studied today, happiness decreases by 1. See the *Description* under *Pet* for the logic in more detail. Below is the logic in practice.

```
def calculateHappiness():
    if !onTrack:
        self.happiness -= 1
    if !studiedToday:
        self.happiness -= 1
```



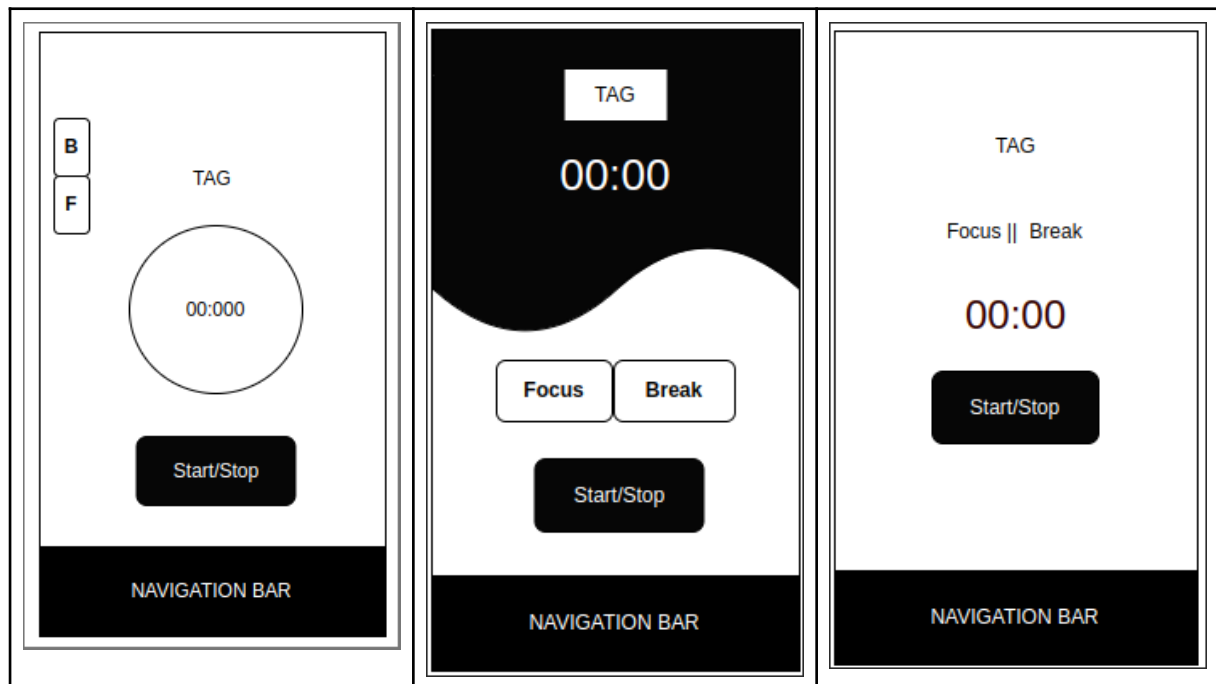
2.7 UI

Things I kept in mind for the general design is:

- An almost monochromatic design, limiting the colour palette for simplicity
 - After watching many productivity videos regarding the using of a smart device such as a phone, most people suggested applying a black and white filter to reduce the distracting nature of a phone. I took some inspiration from this to give my app a sleek design with minimal colours. Of course I may allow themes to be changed in the future, but similar to pomofocus I'll keep a limited colour palette for each theme
- The timer page must abstract any other features in the app. It will only contain what's necessary to ensure users don't feel distracted by it
- As the statistics page will contain a lot of information, it must be layed out in an intuitive easy to understand manner. This may mean lots of labels and headings, while at the same time it mustn't result in an overwhelmingly complex design that can put users off
- The pets page should be more packed with design to keep the user engaged and satisfied from studying

2.7.1 Timer Page

Below are rough drafts of the app's potential design on the timer page.



I ended up settling for the last design as it seemed to have the most effective layout to me. Personally, I find circular timers to be unattractive, but it's possible that I'll add more styles in the future. For now, I'll settle on my preferred aesthetic. The 2nd one also looks nice to me, however I think it might look a little too cluttered. On the next page, I've fleshed out that concept design a little more.

The tag will be at the top so users can easily see what they're meant to be focused on, along with the session's mode (focus/break). In the middle is the timer with a button below to control the operations of the timer (such as start and pause). Upon pausing the app, the skip button will be revealed.

- The Start button should call the `startTimer()` function
- The pause button calls the `pauseTimer()` function
- The skip button calls the `skipTimer()` function
- `stopTimer()` will be called when the timer's textView (the large timer display in the middle) reaches 0 (so, when the `timeRemaining` variable reaches 0).

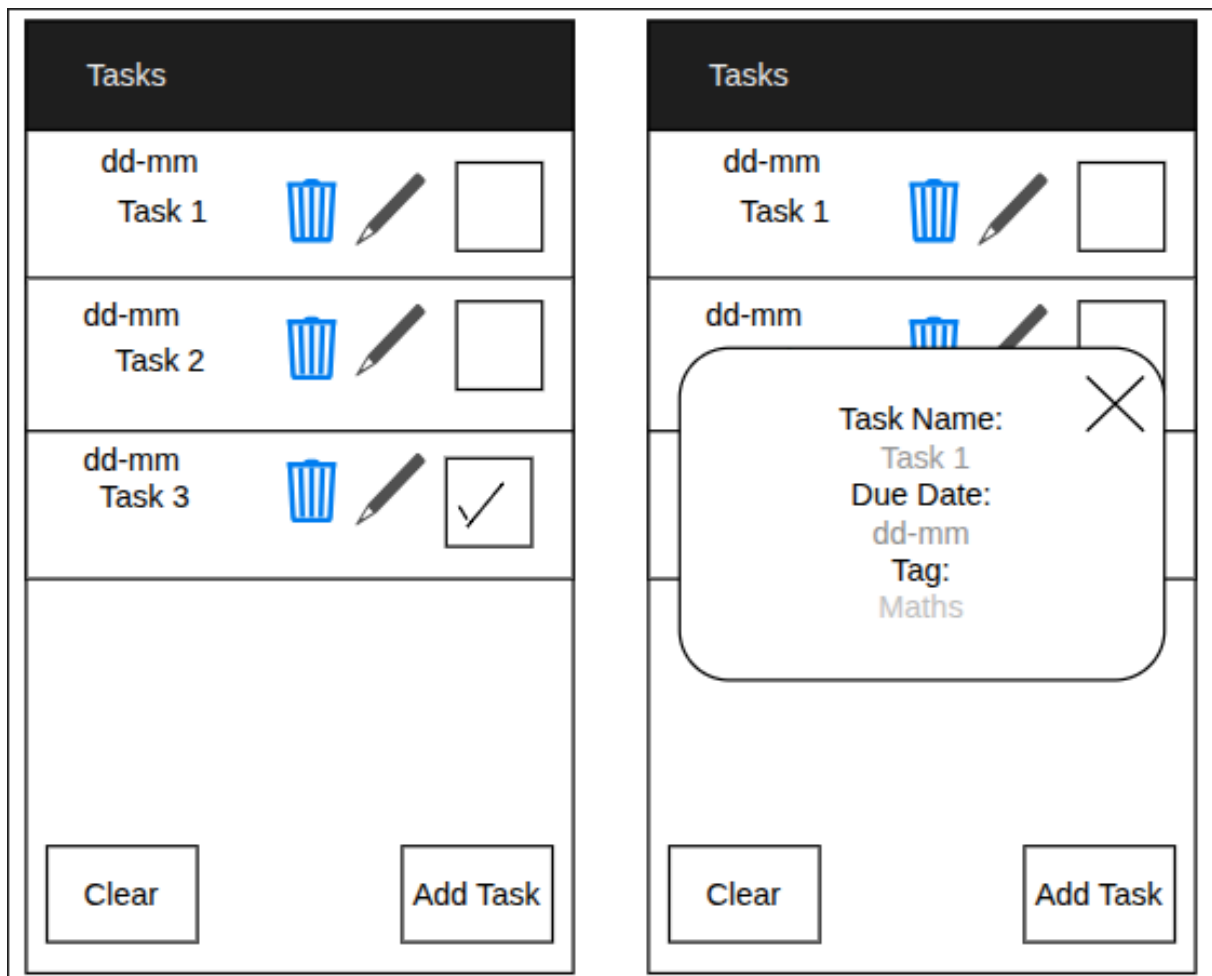
Before Session	During session	Paused Session
 • TAG Focus Break 25:00 START 	 • TAG Focus Break 24:58 PAUSE 	 • TAG Focus Break 23:47 ▶ START 

At the bottom of the page is a navigation bar that gives users access to the timer, tasks, pet and reports page. The settings page can be navigated through the button at the top.

2.7.2 Tasks Page

Below I've made 2 revisions:

1.



As seen in the 2.3 Task management section, Each task could have its respective edit and checkbox button. Completed tasks will be greyed out and have a strike-through design, a very popular style across many to-do list apps.

- Pencil Icon: clicked on to reveal a box to edit task properties
- Bin icon: Click on to delete respective tasks
- Checkboxes can be ticked or unticked to show the completion status of a task
- Clear button clears all checked tasks
- Add task adds task

The second image shows that when you click on the pencil icon to delete, a box will pop up allowing you to edit particular details.

Methods linked to the UI:

- Above each task is the due date of the task

- Clicking the checkbox calls `checkBox()` and adds a tick to the box icon
- Each priority will cause a differently coloured icon to show up
- When users click on a task's respective edit button, as seen in image 3, a window containing all the task's editable attributes will pop up allowing the users to begin altering the task

2.

Tasks	
dd-mm Task 1	☐
dd-mm Task 2	☐
dd-mm Task 3	☒

Clear

Add Task

Tasks	
dd-mm Task 1	☐
dd-mm Task 2	☐
dd-mm Task 3	☒

Task 2

Due date: dd-mm

Tag: Enter Tag

Save

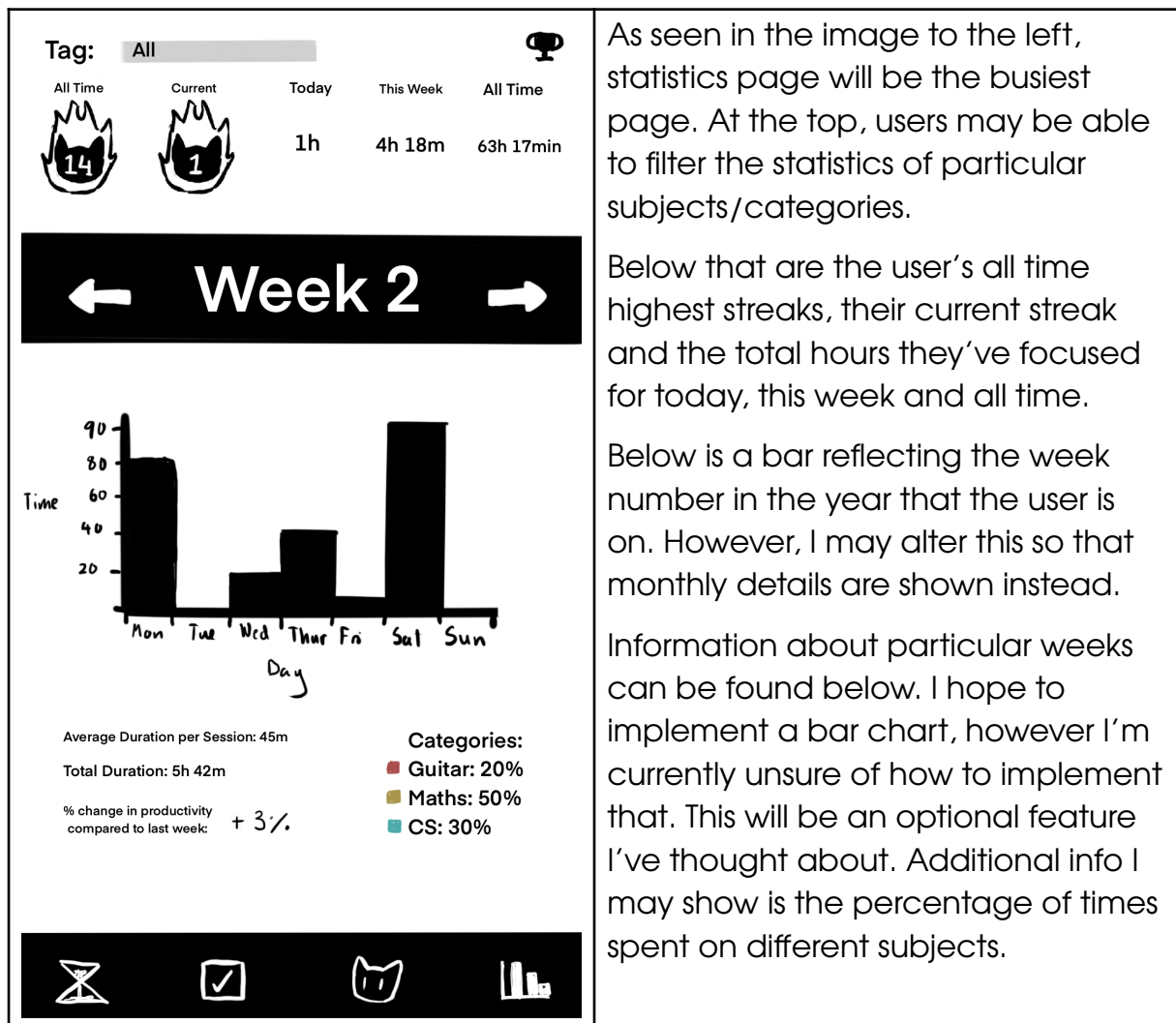
This design is more minimal, the edit button is done away with, and instead, to edit the task, users should click on the title of the respective task. I believe that this design is also intuitive.

- Click on task name to open edit mode
- Click on checkbox to check
- Clear button clears tasks
- Add button adds task

As you can see in the second image, a box will pop up upon clicking on a task.

Conclusion I have chosen revision 2 as my final goal, as it is much more minimal and uncluttered, removing many unnecessary icons such as an edit and delete button.

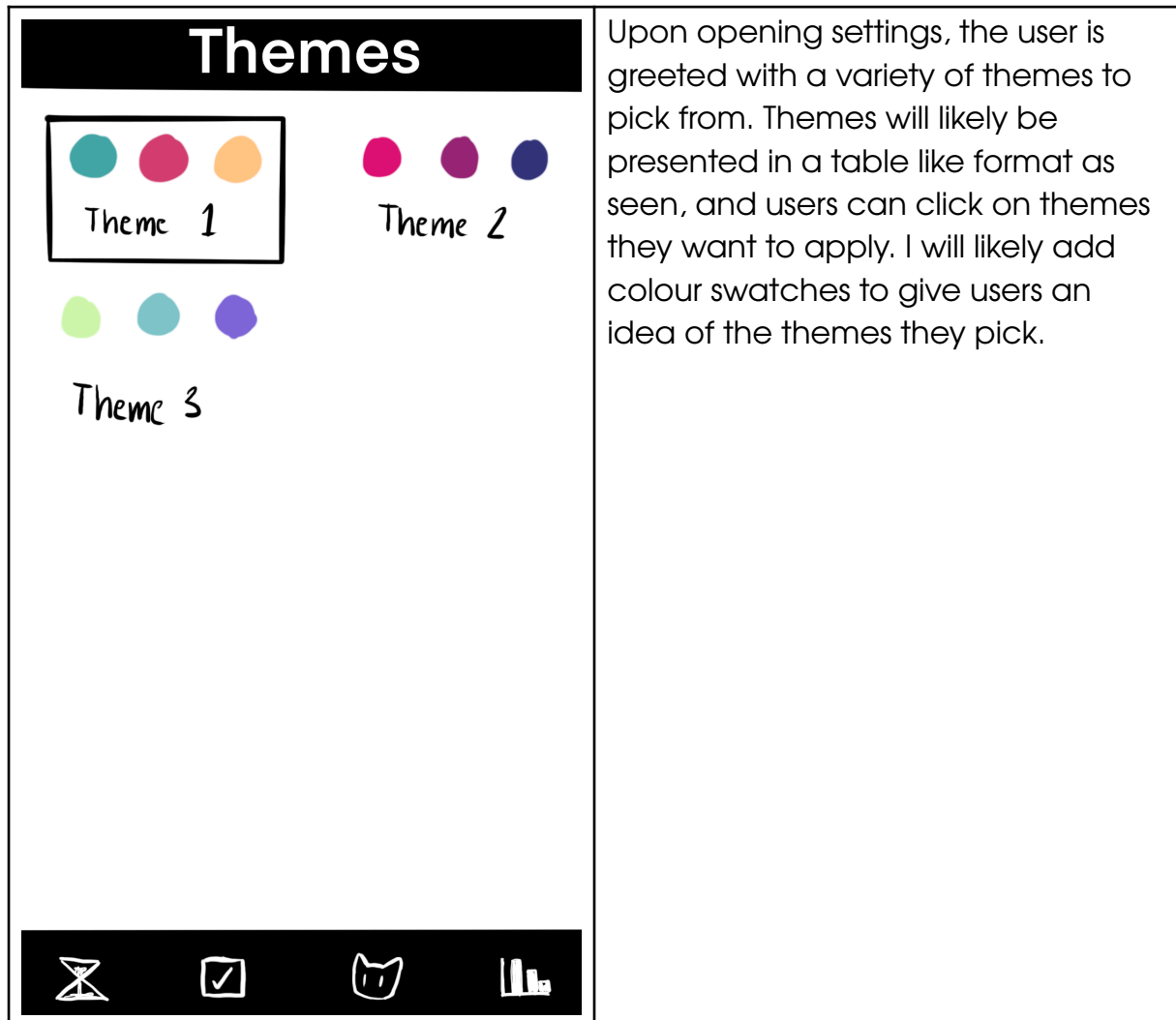
2.7.3 Statistics Page



The functions responsible for reflecting such information will be methods in the Statistics object. The values returned will need to be displayed on the respective textViews.

2.7.4 Settings/Themes Page

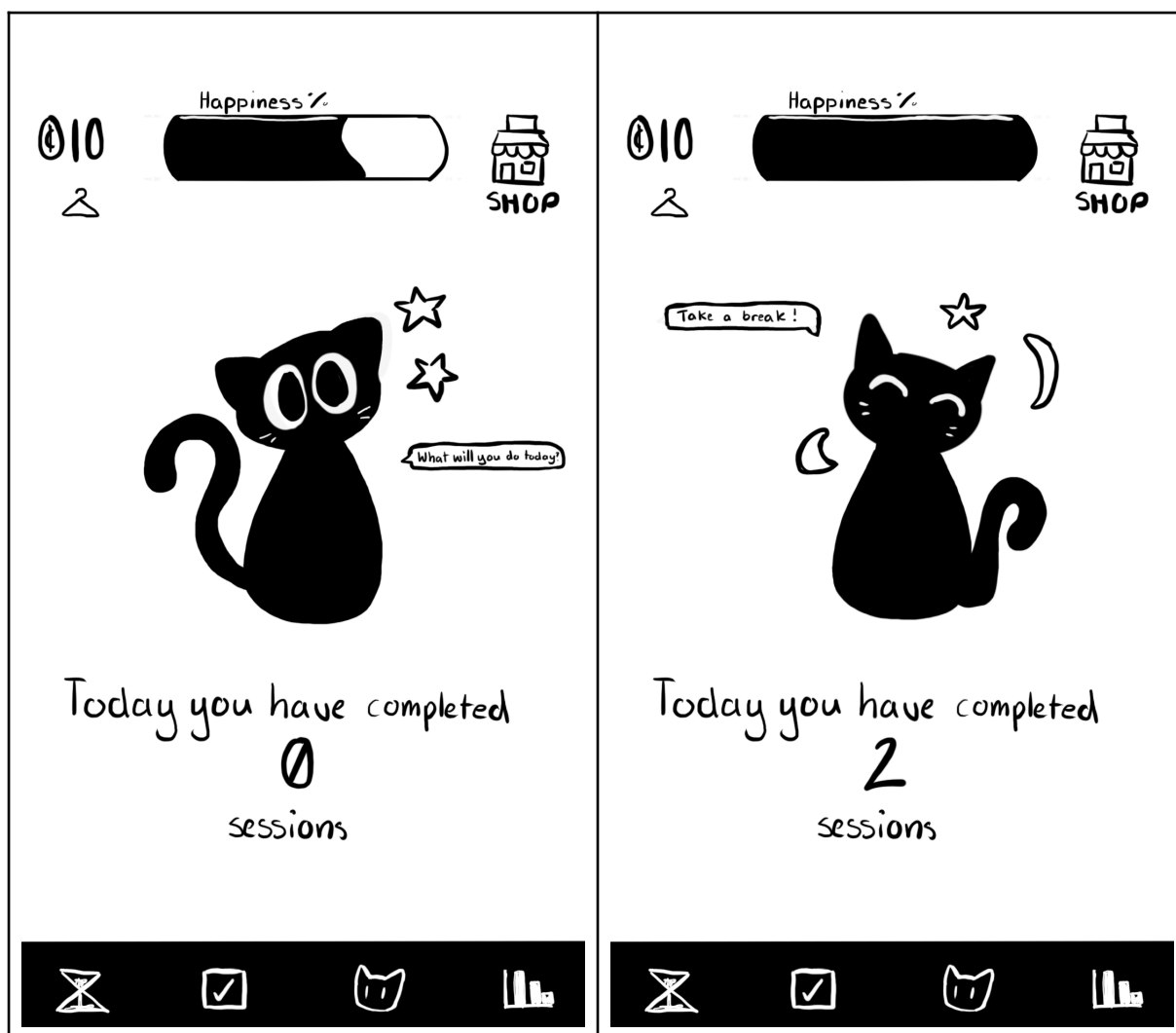
As seen in the Timer-UI section, the settings page will be accessed from an icon on the timer page. It will be split into 2 sections- the account part and then themes. This was my first and only revision, I was immediately happy with it.



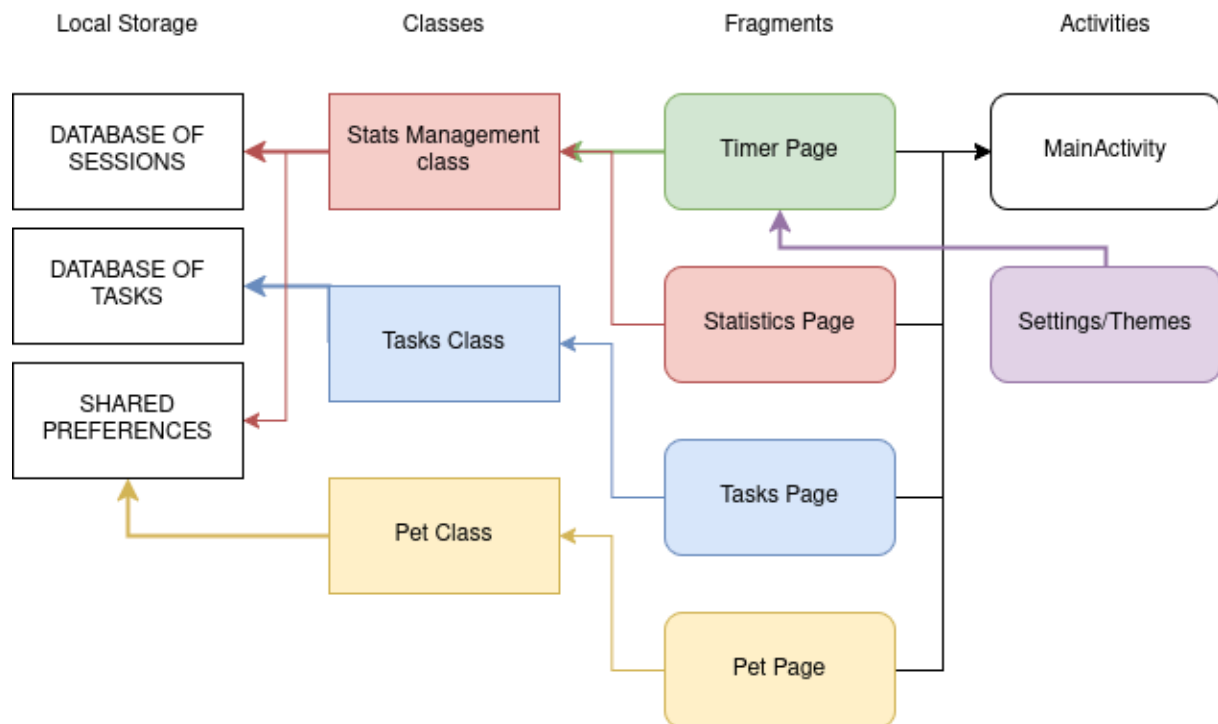
2.7.5 Pet Page

Right off the bat, the pet page will display the following:

- Happiness meter
- Amount of coins (optional)
- Number of sessions studied today (or maybe total time studied today)
- The pet along with a motivation comment and expression that should change every once in a while, also depending on the pet's happiness



2.8 Overall Relations



1. The first column of boxes represent local storage means such as the different databases. The 2nd column has each class in my program. The third represents each fragment, and the last represents each activity. Refer to the beginning of *Design* for definitions of activities, fragments, etc.
2. The arrows going from classes to local storage show which classes access which local storage locations. E.g, stats class accesses the database containing sessions, along with sharedPreferences. Again see *Design* for more definitions and explanations of local storage.
3. Arrows going from fragments to classes show which fragments utilise which classes.
4. Arrows going from fragments to MainActivity show that all 4 fragments will be contained within MainActivity. Arrow from settings/themes to the timer fragment shows that the settings/themes activity is accessed from the timer fragment.

3 Technical Solution

3.1 Completeness

Obtained straight from the objectives section in 'Analysis', I've listed the objectives I set below, along with reference points to where those objectives were completed in my code (e.g., 3.2.2.2 OnViewCreated()).

Each objective is assigned a unique code (e.g., T1, T2, etc.) to enable clear cross-referencing between the Objectives list and the Code section. For example, T1 refers to the first timer-related objective and appears in section 3.2.2.2, indicating where it is implemented. At each referenced location, the corresponding objective code is annotated alongside an explanation detailing how the code fulfills that specific objective.

Each objective is accompanied by a checkbox:

- ☒ Indicates the objective is fully completed
- ☐ indicates the objective was not fully complete

Timer Functions

- ☒ **T1** A timer where users can customize durations - 3.3.2.2 OnViewCreated()
- ☒ **T2** Timers can be paused and resumed - 3.3.2.3 startTimer & 3.3.2.5 stopPauseTimer()
- ☒ **T3** A break timer - 3.3.2.3 startTimer()
- ☒ **T4** Timers can be skipped after a duration. The duration already focused will be logged - 3.3.2.6 skipTimer()
- ☒ **T5** When the session ends, users will be sent a push notification - 3.3.2.3 startTimer() & 3.3.2.7 Notification Management
- ☒ **T6** Each study session is logged, along with the duration of the session - 3.3.2.4 logSession(), 3.3.4.4 addSession()
- ☒ **T7** Users can add labels and categorize the activities they focus on e.g studying, playing an instrument 3.3.2.2 onViewCreated()
- ☒ **T8** Streak measuring how many days the user has focused consecutively - 3.3.4.5
- ☒ **T9** Users can change themes - 3.3.1 MainActivity.kt
- ☒ **T10** Users should be able to view the number of sessions completed today, their current and highest streak, and the number of hours focused for today. - 3.3.5.2 PetFragment.kt, 3.3.4.6 - StatsFragment.kt

Pet Functions

- ✓ **P1** Pets have a happiness meter which increases the more students focus and declines the more students miss their study goal - 3.3.5.1 under happinessChecker()
- ✓ **P2** Happiness meter is related to the goal time that the user sets for themselves - 3.3.5.1 under onTrack(), happinessChecker()
- ☐ **P3** Pets will output randomly selected motivational comments while a user is focusing - Not yet met

Task Management

All the following objectives have been met under

- ✓ **M1** Users can create a task - 3.3.3.2 TaskItemDao.kt, 3.3.3.11 NewTaskSheet.kt
- ✓ **M2** Users can edit a task - 3.3.3.2 TaskItemDao.kt, 3.3.3.11 NewTaskSheet.kt
- ✓ **M3** Users can check off a task - 3.3.3.2 TaskItemDao.kt, 3.3.3.6 OnTaskCompleteListener.kt
- ✓ **M4** Users can clear all completed tasks at the same time - 3.3.3.2 TaskItemDao.kt,

NOTE

When you reach the Code section, you may see:

☐ Objective x

This indicates that part of an objective is completed here. This occurs when the completion of 1 objective is split over multiple functions, pages or classes. It does NOT mean that an objective was not successfully completed, it means that part of the code towards that objective's completion is in that respective section. I also explain which part of the objective is met and how.

3.2 Summary of Techniques Used

Below are some summaries of techniques I used in the different areas of my program.

General OOP Techniques Used

- These techniques were used over a majority of my files as Kotlin is an Object-Oriented Programming Language.

Technique	How it's used
Encapsulation	I've written private variables and functions to be encapsulated into a particular class, as there are some functions such as data manipulating ones I don't want to be accessible by just any part of the program.

Polymorphism	I have used abstract classes to keep my code modular and allow those classes to be extended easily for necessary methods for databases. Furthermore, by using an abstract class I can always switch to another type of database.
Data classes	Session is a user-defined data class that I've used to describe each session (e.g duration, task type)
Companion Object	This is an object associated with a class, in this case the taskItem class. It's used to format date strings so that I can access the dateFormatter without needing the program to instantiate a new instance of that method for each and every taskItem. It's also used in Statistics.kt to manipulate streak data. This results in memory being saved.
Asynchronous programming	This is carrying out a task in the background, NOT on the main thread, which is necessary for a stutter free and responsive UI.

Database/Local Storage Related Techniques used

- The techniques shown in the following table is implemented in the database relation sections of my code, mainly apparent in 3.3.4 Sessions & Statistics, and 3.3.3 To-Do List

Technique	How it was used
Parameterized Queries & Filtering Data	I've queried sessions from a specific period of time, using start and end dates as parameters.
Aggregate functions	I have summed minutes of session durations and created an SQL query returning sessions only of a specific type.
Lifecycle-Conscious Database Management	I've used @Volatile to follow a singleton pattern. This makes a database visible to all threads. This is important because I'm carrying out asynchronous operations, meaning if I'm reading/writing data from different parts of my program on different threads for any reason, all other threads can see this going on. This prevents outdated caching of any particular variables in my database.
Reactive programming	I've used Kotlin's Flow, which allows UIs to automatically update if there's change detected in the database, without needing a manual refresh.
Priority Queue Behaviour	Completed tasks are considered lower priority, so they are displayed at the bottom in the order that they were checked off.
API Usage	The rooms API was utilized for implementation of databases.
XML File Parsing	A form of local storage I used was shared preferences, which stores data in the form of dictionary key-pair values in an xml file.

Coding Style

Techniques	How it's used
State Management	Allows for persisting data across different lifecycles of the app, e.g even after closing the app.
Lifecycle Consciousness	I've been lifecycle conscious by using a ViewModel component. If I did not, and a reconfiguration such as screen rotation was applied to my app, data that was being processed in that moment could be lost. Furthermore, UI data may need to be unnecessarily re-fetched if there was a configuration change.
Abstraction	View models also separated UI related data management from the the UI controller. Only the UI controller, e.g my fragments are responsible for handling UI logic such as displaying certain data. Abstract Classes used are also a part of abstraction being a part of the code design.
Custom Algorithmic Logic	Used in the streak checker, an algorithm is used to decide whether to increment or reset a streak.
Conflict Handling	Conflict is automatically handled when you try insert an existing record. If a conflict of this sort occurs, I've set a variable in room to ensure the conflicting record is replaced by the newer record. Furthermore, it's impossible for erroneous data to be passed to any of my functions, as user input data will come in the form of an android component that gives you a selection of choices, only in the particular format that it should be in. This is demonstrated when I use number pickers.

3.3 Code

This section contains the code to my app. At the end of each sub-section, I have also added the relevant xml files. These files do not contain functional logic, but they describe the aesthetic layout of the app. They're the web development's CSS equivalent for android apps.

3.3.1 MainActivity.kt

```
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment
import io.github.finnimo.tamadoro.fragments.PetFragment
import io.github.finnimo.tamadoro.R
import io.github.finnimo.tamadoro.fragments.StatsFragment
import io.github.finnimo.tamadoro.fragments.TasksFragment
import io.github.finnimo.tamadoro.databinding.ActivityMainBinding
import io.github.finnimo.tamadoro.fragments.TimerFragment
import io.github.finnimo.tamadoro.sessiondatabase.SessionsDatabase

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {

        //HANDLING THEME BEFORE EVERYTHING IS RENDERED
        val sharedPrefs = getSharedPreferences("TAMADORO_THEME",
        MODE_PRIVATE)
        val selectedTheme = sharedPrefs.getString("TAMADORO_THEME",
        "light") ?: "light"

        when (selectedTheme) {
            "dark" -> setTheme(R.style.darkTheme)

            else -> setTheme(R.style.lightTheme) // Default to light
        }
    }
}
```

As seen, on the creation of MainActivity, SharedPreferences (a storage means for android) is accessed to retrieve the theme that was last picked. If there was no theme picked i.e the program has been launched for the first time, the theme will default to light.

Using the kotlin equivalent of a switch case statement, we set the theme to the one stored in the selectedTheme variable.

This checks off:

- ☑ **T9** Users can change themes
 - As this function switches between themes

```

super.onCreate(savedInstanceState)
//setContentView(R.layout.activity_main)

binding = ActivityMainBinding.inflate(layoutInflater)
setContentView(binding.root)
loadFragment(TimerFragment())

binding.bottomNavigationView.setOnItemSelectedListener {item ->
    when (item.itemId) {
        R.id.timerNav -> {
            loadFragment(TimerFragment())
            true
        }
        R.id.tasksNav -> {
            loadFragment(TasksFragment())
            true
        }
        R.id.petNav -> {
            loadFragment(PetFragment())
            true
        }
        R.id.statsNav -> {
            loadFragment(StatsFragment())
            true
        }
        else -> false
    }
}

private fun loadFragment(fragment: Fragment){
    val transaction = supportFragmentManager.beginTransaction()
    transaction.replace(R.id.frameLayout, fragment) // Replace with
your fragment container's ID
    transaction.commit()
}
}

```

While the activity is being created, each fragment will be loaded ready for use. Each fragment can be accessed via a nav bar.

3.3.1.1 activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="?attr/colorPrimary"
    android:clickable="true"
    android:focusable="true"
    android:focusableInTouchMode="true"
    tools:context="io.github.finnimo.tamadoro.activities.MainActivity">

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottomNavigationView"
        android:layout_width="409dp"
        android:layout_height="75dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:menu="@menu/bottom_navbar"
        android:background="?attr/colorOnPrimary"
    />

    <FrameLayout
        android:id="@+id/frameLayout"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintBottom_toTopOf="@+id/bottomNavigationView"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="1.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.0">

    </FrameLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

3.3.1.2 bottom_navbar.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    >

    <item
        android:icon="@drawable/ic_timer"
        android:id="@+id/timerNav"
        />
    <item
        android:icon="@drawable/ic_tasks"
        android:id="@+id/tasksNav" />
    <item
        android:icon="@drawable/ic_pet"
        android:id="@+id/petNav" />
    <item
        android:icon="@drawable/ic_stats"
        android:id="@+id/statsNav" />
</menu>
```

3.3.2 Timer - TimerFragment.kt

Unlike the other sections where multiple files are required for their respective function (e.g stats, etc), all of the following code in this section is timer related and resides inside 1 file/class - TimerFragment.kt.

3.3.2.1 Variable declarations & imports

```
//Imports for notifications
import android.Manifest
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.Context
import android.content.Context.NOTIFICATION_SERVICE
import android.content.Intent
//import for ui
import android.content.res.ColorStateList
import android.media.RingtoneManager
import android.net.Uri
import android.os.Build
import android.os.Bundle
import android.text.Editable
import android.text.TextWatcher
import android.util.Log
import android.util.TypedValue
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.EditText
import android.widget.ImageButton
import android.widget.LinearLayout
import android.widget.NumberPicker
import android.widget.TextView
import androidx.core.app.ActivityCompat
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat
import androidx.core.content.ContextCompat
import com.google.android.material.bottomnavigation.BottomNavigationView
import io.github.finnimo.tamadoro.activities.MainActivity
import io.github.finnimo.tamadoro.fragments.pethandling.Pet
import io.github.finnimo.tamadoro.R
import io.github.finnimo.tamadoro.sessiondatabase.Statistics
import io.github.finnimo.tamadoro.sessiondatabase.SessionManager
import io.github.finnimo.tamadoro.activities.SettingsActivity
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.launch
//import for countdownimer api
import android.os.CountDownTimer
```

```

class TimerFragment : Fragment() {
// VARIABLES TO ACCESS ROOMS/DATABASE & COINS CLASS
    private lateinit var manager: SessionManager
    private var timer: CountDownTimer? = null

    // BUTTONS

    private lateinit var startBtn: Button
    private lateinit var skipButton: Button
    private lateinit var breakModeBtn: Button
    private lateinit var focusModeBtn: Button
    private lateinit var newTag: EditText
    private lateinit var deleteStats: Button
    private lateinit var settingsBtn: Button

    private lateinit var pickerContainer: LinearLayout
    private lateinit var hoursPicker: NumberPicker
    private lateinit var minutesPicker: NumberPicker
    private lateinit var durationSave: Button

    // TEXTVIEW DECLARATIONS

    private lateinit var timerTextView: TextView

    //TAG FOR SESSION TYPE

    private var tag: String = "Tag"

    // OTHER VARIABLES FOR TIMER FUNCTIONS

    private var timerRunning = false
    private var initialTime: Long = 0
    //private var initialTime = 2 * 1000L for test purposes as stated above
    private var timeRemaining: Long = 0
    private var isPomodoro: Boolean = true //boolean value to indicate
    whether a timer is in break or pomodoro mode
    private var breakLength = 5 * 1000L

    //NOTIFICATIONS
    private val channelId = "ChannelID"
    private val channelName = "NotificationChannel"
    private val notifID = 1
    private val notifPermissionRequestCode = 100

```

These are the private variables I have declared for the TimerFragment. The following code blocks will explain and show how they're utilized.

3.3.2.2 onViewCreated()

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)

    //Initializing database and grabbing sessions first so that I don't
    accidentally access database before it's initialized, avoiding this crash

    manager = SessionManager(requireContext())

    initialTime =
requireContext().getSharedPreferences("TAMADORO_FOCUSDURATION",
Context.MODE_PRIVATE)
        .getLong("TAMADORO_FOCUSDURATION", 25 * 60 * 1000L)
    breakLength =
requireContext().getSharedPreferences("TAMADORO_BREAKDURATION",
Context.MODE_PRIVATE)
        .getLong("TAMADORO_BREAKDURATION", 5 * 60 * 1000L)

    GlobalScope.launch(Dispatchers.Main) {

        manager.getAllSessions()
    }

    //For notifications

    createNotifChannel()
    checkNotificationPermission()
}
```

When the view is being created, all the private lateinit variables I declared are assigned values. Private lateinit variables are variables that are currently unassigned, but will definitely be defined by t

```
// TEXT VIEWS + EDIT TEXT

timerTextView = view.findViewById(R.id.timerTextView)
newTag = view.findViewById(R.id.newTag)

// BUTTONS

startBtn = view.findViewById(R.id.startButton)
skipButton = view.findViewById(R.id.skipButton)
settingsBtn = view.findViewById(R.id.settingsBtn)

breakModeBtn = view.findViewById(R.id.breakTimer)
focusModeBtn = view.findViewById(R.id.focusTimer)
deleteStats = view.findViewById(R.id.deleteStats)

hoursPicker = view.findViewById(R.id.hoursPicker)
minutesPicker = view.findViewById(R.id.minutesPicker)
pickerContainer = view.findViewById(R.id.pickerContainer)
durationSave = view.findViewById(R.id.durationSaverBtn)

button = view.findViewById(R.id.button)

//TAG

tag = newTag.text.toString()

//Formats the timer to display the right count down starting number
timerTextView.text = formatTime(initialTime)
```

This section of TimerFragment is binding UI components to variables so that i can access and manipulate those components in code. As seen in the line `timerTextView.text = formatTime(initialTime)`, the Text displayed on my timer textView component will contain a string of characters returned by my `formatTime()` function which is show below.

```
private fun formatTime(millis: Long): String {
    val minutes = millis / 1000 / 60
    val seconds = (millis / 1000) % 60
    return "%02d:%02d".format(minutes, seconds)
}
```

This function formats duration from milliseconds to minutes and seconds, to be displayed onto the timerTextView.

```
// The following code actively listens for change in editText component
containing the session tag, whenever it changes, that's the tag that'll be
appended to the session on timer finish

newTag.addTextChangedListener(object : TextWatcher {
    override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int,
after: Int) {
        // Optional: Do something before text changes
    }

    override fun onTextChanged(s: CharSequence?, start: Int, before: Int,
count: Int) {
        // This is called as the text is changing
        tag = (s.toString())
    }

    override fun afterTextChanged(s: Editable?) {
        // Optional: Do something after the text has changed
    }
})
```

I have an editable text component. This part of the code listens for any change in the component (likely from the user typing something into it) and updates the tag variable to the change that's been made.

This part meets the objective:

☑ **T7** Users can add labels and categorize the activities they focus on

```
// UI COMPONENTS CLICKED ACTIONS

timerTextView.setOnClickListener {
    if (!timerRunning) {

        pickerContainer.visibility = LinearLayout.VISIBLE

        hoursPicker.minValue = 0
        hoursPicker.maxValue = 12
        minutesPicker.minValue = 0
        minutesPicker.maxValue = 59

        val currentDuration: Long
        val durationType: String

        if (isPomodoro) {
            currentDuration = initialTime
            durationType = "FOCUSDURATION"
        } else {
            currentDuration = breakLength
            durationType = "BREAKDURATION"
        }
    }
}
```

As seen above, when the user clicks on the timerTextView component, a number picker is displayed. The number picker is used for setting a custom timer duration. You can also see me declaring variables so that they can later be assigned depending on whether the user is editing the focus duration or the break duration.

```
        val initHours: Int = (currentDuration / 3600000).toInt()
        val initMins: Int = (((currentDuration % 3600000) /
60000)).toInt()

        hoursPicker.value = initHours
        minutesPicker.value = initMins

        durationSave.setOnClickListener {

            val hours = hoursPicker.value
            val mins = minutesPicker.value

            val durationInMillis: Long = ((hours * 3600) + (mins * 60)) *
1000L

            if (isPomodoro) {
                initialTime = durationInMillis
            } else {
                breakLength = durationInMillis
            }
            timeRemaining = durationInMillis
            Log.d("initial time now", initialTime.toString())

            requireContext().getSharedPreferences("TAMADORO_$durationType",
Context.MODE_PRIVATE).edit().
                putLong("TAMADORO_$durationType", durationInMillis).apply()

            timerTextView.text = formatTime(timeRemaining)
            pickerContainer.visibility = LinearLayout.INVISIBLE

        }

    }

}
```

There are 2 storage locations I'm accessing in Shared Preferences, the focus timer duration and the break timer duration. After the custom duration is picked and stored in shared preferences, the number picker leaves the screen. **This block meets the following objective:**

- ☑ **T1** A timer where users can customize durations
 - Because a number picker is introduced to pick a custom duration.

```

deleteStats.setOnClickListener {
    manager.deleteAllSessions()
}

focusModeBtn.setOnClickListener {
    isPomodoro = true
    timerTextView.text = formatTime(initialTime)
    timeRemaining = initialTime
    updateButtonAppearance()
}

breakModeBtn.setOnClickListener {
    isPomodoro = false
    timerTextView.text = formatTime(breakLength)
    timeRemaining = breakLength
    updateButtonAppearance()
}

startBtn.setOnClickListener { //when start button clicked
    if (timerRunning) {
        pauseTimer()
    } else {
        startTimer(timeRemaining)
    }
    Log.d("start button pressed", "PRESSED ")
}

skipButton.setOnClickListener {
    skipTimer()
    timeRemaining = if (isPomodoro){
        initialTime
    } else{
        breakLength
    }
}

settingsBtn.setOnClickListener {
    val openSettingsActivity = Intent(requireActivity(),
SettingsActivity::class.java)
    startActivity(openSettingsActivity)
}
}

button.setOnClickListener {
    logSession(20)
}

timeRemaining = initialTime
timerTextView.text = formatTime(initialTime) }

```

3.3.2.3 startTimer()

```
private fun startTimer(time: Long) {
    timer = object : CountDownTimer(time, 1000) {
        override fun onTick(millisUntilFinished: Long) {
            timeRemaining = millisUntilFinished
            timerTextView.text = formatTime(millisUntilFinished)
        }

        override fun onFinish() { //WHEN TIMER ENDS
            //val minutes = (initialTime / 1000).toInt()
            //above is for debug
            val minutes = (initialTime / 60000)

            if (isPomodoro) {
                logSession(minutes)
            } else {
                stopTimer(breakLength)
            }
            showNotification()
        }
    }.start()

    //When timer starts, the start button will say pause

    startBtn.text = "Pause"
    skipButton.visibility = Button.INVISIBLE
    timerRunning = true
}
```

time is the duration in milliseconds that the timer starts to count down from. The timer variable is an object that contains a timer, from the CountDownTimer Library. On each tick (1000ms countdown from time), The timerTextView (the UI component displaying the timer) will update to reflect the remaining time.

When the timer finishes, a session is logged along with the number of minutes that has passed (only when the timer is in focus mode as shown through the isPomodoro variable). At the end, a notification is shown.

This function is also called when the timer has been paused. The duration it counts down from is the duration it last reached. See stopPauseTimer() in 3.2.2.5 stopPauseTimer()

If the isPomodoro variable is set to false, it means the timer is not in focus mode and therefore break mode. The session is then treated as a break and is not logged.

This section aids in meeting part of the objective:

- **T2** Timers can be paused and resumed
 - Because it sets up the start button to display “pause” while the timer runs. This contributes to intuitiveness of the UI.
- **T3** A break timer
 - Because when the timer finishes, if it’s in break mode, no data is logged.
- **T5** When the session ends, users will be sent a push notification.
 - The function to show a notification is called on the timer’s finish, meaning a notification is sent when a session ends.

3.3.2.4 logSession()

```
private fun logSession(minutes: Int) {  
    manager.addSession(minutes, tag)  
    Pet.addCoins(requireContext(), minutes)  
    stopTimer(initialTime)  
    timerTextView.text = "Finished!"  
    Statistics.updateStreaks(requireContext())  
    Statistics.updateLastSessionDate(requireContext())  
  
    //DEBUG STUFF:  
    val x = Pet.getTotalCoins(requireContext())  
    Log.d("coin amount", x.toString())  
}
```

manager was previously declared as an object of the SessionManager class. This class is showcased in StatsFragment. It handles all the data manipulation of sessions. When logSession() is called, the player is rewarded a number of coins based on the duration of the session. The same goes for the Pet class; it calls the addCoins function so that users get coins with each session they log. Although this isn’t an official objective, I added it anyway as I believe coin-collecting can serve as a motivator to study more, users can even compare coin amounts between one another.

UI components are updated to reflect the timer has finished. The log.d line just allows me to see if the correct amount of coins has been added in logcat (a debugging tool) each time logSession is called.

3.3.2.5 stopPauseTimer()

1. Initially, I had 2 separate functions for stop and pause timer:

```
private fun pauseTimer() {  
    timer?.cancel()  
    startBtn.text = "Resume"  
    skipButton.visibility = Button.VISIBLE  
    timerRunning = false  
}
```

In pauseTimer(), the timer is cancelled and the pause button now displays resume. timerRunning is set to false.

```
private fun stopTimer(time: Long) {  
    timer?.cancel()  
    timerTextView.text = formatTime(time)  
    startBtn.text = "Start"  
    timerRunning = false  
    timeRemaining = time  
}
```

As for stopTimer(), the same thing occurs. It's used for when the timer reaches a finish rather than getting paused.

2. In the end, I believed it would be more efficient to merge them into single function due to the repeating lines:

```
private fun stopPauseTimer(pause: Boolean, time: Long) {  
    timer?.cancel()  
    timerRunning = false  
    if (pause) {  
        startBtn.text = "Resume"  
        skipButton.visibility = Button.VISIBLE  
    } else {  
        timerTextView.text = formatTime(time)  
        startBtn.text = "Start"  
        timeRemaining = time  
    }  
}
```

The boolean pause variable indicates whether we want to pause or stop the timer. The time value is required in order for the timeRemaining to be reset upon the timer stopping. If the timer is to be paused, the start button that starts the timer displays "Resume"

This section meets the rest of the objective:

■ **T2** Timers can be paused and resumed

- Because this function cancels the timer and stores the duration that's remaining to allow the timer to be paused (and resumed from duration

left) and stopped. See 3.3.2.3 to see how the first part of the objective was met.

3.3.2.6 skipTimer()

```
private fun skipTimer() {
    //if its in focus mode then the skip will skip to break mode, vice versa
    if (isPomodoro) {
        val timeFocusedAlready = ((initialTime - timeRemaining + 1000) /
1000).toInt()
        manager.addSession(timeFocusedAlready, tag)
        timerTextView.text = formatTime(breakLength) //resets text view
        isPomodoro = false
    } else {
        timerTextView.text = formatTime(initialTime)
        isPomodoro = true
    }
    timer?.cancel()
    updateButtonAppearance()
    startBtn.text = "Start"
    skipButton.visibility = Button.INVISIBLE

    //resetting timer
    timerRunning = false
}
```

This section clears the following objective:

- ✓ **T4** Timers can be skipped after a duration. The duration already focused will be logged
 - Time focused already is calculated and then logged into a new session (if the timer is in focus mode). Then the timer is stopped, creating a skipping function.
 - The UI is updated to restore the timer to its full duration, and then the skip button is made invisible, as you can't skip timers when they're not running any longer.

3.3.2.7 Notification Management

In order to implement post notifications, I referenced this guide: <https://developer.android.com/develop/ui/views/notifications/build-notification> Which utilizes the NotificationManagerCompat library.

```
private fun showNotification() {  
    val notificationManager = NotificationManagerCompat.from(requireContext())  
    if (notificationManager.areNotificationsEnabled()) {  
        val intent = Intent(requireContext(), MainActivity::class.java).apply {  
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or  
            Intent.FLAG_ACTIVITY_CLEAR_TASK  
        }  
    }
```

Firstly, I initialized a NotificationManagerCompat object, which is responsible for managing notifications. It checks whether notifications are enabled for my app on this device or not.

The intent creates an intent specifying what happens when the notification is clicked on. As seen, I made it so that MainActivity is launched on click. Researched from stack overflow, the flags are present to ensure that any previous instances of MainActivity is cleared before launching this new activity. This ensures not collisions or crashes occur from multiple instances of MainActivity being launched.

```
val pendingIntent: PendingIntent = PendingIntent.getActivity(  
    requireContext(),  
    0,  
    intent,  
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE  
)
```

Pending intent allows for intent to be executed when required.

```
val notificationSound: Uri =  
    RingtoneManager.getDefaultUri(RingtoneManager.TYPE_NOTIFICATION)
```

This just gets the default notification sound from the user.

```

        val builder = NotificationCompat.Builder(requireContext(), channelId)
            .setContentTitle("Tamadoro")
            .setContentText("The timer is up!")
            .setSmallIcon(R.drawable.notification_temp)
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setCategory(NotificationCompat.CATEGORY_MESSAGE)
            .setAutoCancel(true)
            .setFullScreenIntent(pendingIntent, true)

            .setContentIntent(pendingIntent)
            .setSound(notificationSound)
            .setVibrate(longArrayOf(1000))

        with(NotificationManagerCompat.from(requireContext())) {
            if (ActivityCompat.checkSelfPermission(
                requireContext(),
                Manifest.permission.POST_NOTIFICATIONS
            ) != PackageManager.PERMISSION_GRANTED
            ) {
                return
            }
            notify(notifID, builder.build())
        }
    }
}

```

The builder is what's used to create the notification and specify some of its features. For example, I have named the Title Tamadoro (The name of my app) and the description that the user will see is "Timer is up!". I've also given it a custom icon.

The last part checks if the app has permission to send post notifications (this is necessary for all android 13+ devices). If we have permission, the notification is built/posted as seen in the last line.

```

private fun checkNotificationPermission() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (ContextCompat.checkSelfPermission(requireContext(),
            android.Manifest.permission.POST_NOTIFICATIONS) !=
            PackageManager.PERMISSION_GRANTED) {
            // Permission is not granted, request it

            requestPermissions(arrayOf(android.Manifest.permission.POST_NOTIFICATIONS),
                notifPermissionRequestCode)
        }
    }
}

```

This function requests the user permission to send notifications.

```
private fun createNotifChannel(){
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O){
        //This part ensures this only needs to run on android 8 or newer as
        notification channels are not supported on older android versions
        val channel = NotificationChannel(
            channelId,
            channelName,
            NotificationManager.IMPORTANCE_HIGH)
        val notifmanager =
requireContext().getSystemService(NOTIFICATION_SERVICE) as
NotificationManager
        notifmanager.createNotificationChannel(channel)
    }
}
```

This creates a notification channel so that users can customize notification settings such as sound and vibration. In the code block showing the private variables declared in this class, you can see the channel ID and name I assigned:

```
private val channelId = "ChannelID"
private val channelName = "NotificationChannel"
private val notifID = 1
private val notifPermissionRequestCode = 100
```

Channel name is just a user friendly way to describe/identify the channel.

This section meets the objective:

- ✓ **T5** When the session ends, users will be sent a push notification
 - This section is entirely how notifications are implemented. Permissions are requested, and a notification is built and sent when the function is called.

3.3.2.8 updateButtonAppearance

```
private fun updateButtonAppearance() {

    val primary =
getColour(androidx.constraintlayout.widget.R.attr.colorPrimary)
    val secondary =
getColour(com.google.android.material.R.attr.colorSecondary)

    if (isPomodoro) {
        focusModeBtn.backgroundTintList = ColorStateList.valueOf(secondary)
        breakModeBtn.backgroundTintList = ColorStateList.valueOf(primary)
    } else {
        breakModeBtn.backgroundTintList = ColorStateList.valueOf(secondary)
        focusModeBtn.backgroundTintList = ColorStateList.valueOf(primary)
    }
}
```

This function changes the colour of the focus mode and break mode button depending on which mode the user selects. The selected mode should have a darker background. The program decides whether we're on focus mode or not by the isPomodoro variable.

3.3.2.9 fragment_timer.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="?attr/colorPrimary"
    android:clickable="true"
    android:focusable="true"
    android:focusableInTouchMode="true"
    tools:context="io.github.finnimo.tamodoro.activities.MainActivity">

    <TextView
        android:id="@+id/timerTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="25:00"
        android:textColor="?attr/colorOnPrimary"
        android:textSize="80sp"
        android:textStyle="bold"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/timerModeSeparator"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="||"
        android:textColor="?attr/colorOnPrimary"
        android:textSize="36sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toStartOf="@+id/guideline6"
        app:layout_constraintHorizontal_bias="0.309"
        app:layout_constraintStart_toEndOf="@+id/focusTimer"
        app:layout_constraintStart_toStartOf="@+id/guideline5"
        app:layout_constraintTop_toTopOf="@+id/guideline2" />
```

```
<Button
    android:id="@+id/startButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/timerTextView"
    android:layout_marginTop="8dp"
    android:backgroundTint="?attr/colorOnPrimary"
    android:text="Start"
    android:textColor="?attr/colorPrimary"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.51"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/timerTextView" />
```

```
<ImageButton
    android:id="@+id/skipButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="20dp"
    android:visibility="invisible"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toEndOf="@+id/timerTextView"
    app:layout_constraintTop_toTopOf="parent"
    android:src="@drawable/ic_skip"

    android:layout_gravity="center"
    android:backgroundTint="@android:color/transparent"
    android:scaleX="1.5"
    android:scaleY="1.5"
/>
```

```
<Button
    android:id="@+id/deleteStats"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="DeleteStats"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/startButton"
    tools:layout_editor_absoluteX="144dp" />
```

```
<Button
    android:id="@+id/focusTimer"
    android:layout_width="100dp"
    android:layout_height="wrap_content"
    android:backgroundTint="?attr/colorSecondary"
    android:text="Focus"
    android:textColor="?attr/colorOnPrimary"
    android:layout_marginRight="20dp"
    app:layout_constraintEnd_toStartOf="@+id/guideline5"
    app:layout_constraintStart_toStartOf="@+id/guideline8"
    app:layout_constraintTop_toTopOf="@+id/guideline2" />
```

```

<Button
    android:id="@+id/breakTimer"
    android:layout_width="100dp"
    android:layout_height="wrap_content"
    android:backgroundTint="?attr/colorPrimary"
    android:text="Break"
    android:textColor="?attr/colorOnPrimary"
    android:layout_marginLeft="20dp"
    app:layout_constraintEnd_toStartOf="@+id/guideline9"
    app:layout_constraintStart_toStartOf="@+id/guideline6"
    app:layout_constraintTop_toTopOf="@+id/guideline2" />

<EditText
    android:id="@+id/newTag"
    android:layout_width="100dp"
    android:layout_height="55dp"
    android:inputType="text"
    android:gravity="center_horizontal"
    android:hint="Tag"
    app:layout_constraintBottom_toTopOf="@+id/timerTextView"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

<ImageButton
    android:id="@+id/settingsBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:backgroundTint="@android:color/transparent"
    android:src="@drawable/ic_settings" />

<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button"
    tools:layout_editor_absoluteX="20dp"
    tools:layout_editor_absoluteY="6dp" />

```

```

<LinearLayout
    android:layout_margin="7dp"
    android:id="@+id/pickerContainer"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center"
    android:visibility="invisible"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:textColor="?attr/colorOnPrimary"
    android:background="?attr/colorPrimary">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Select Duration"
        android:textColor="?attr/colorOnPrimary">
    </TextView>

    <LinearLayout

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <NumberPicker
            android:id="@+id/hoursPicker"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textColor="?attr/colorOnPrimary"/>

        <NumberPicker
            android:id="@+id/minutesPicker"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textColor="#FF0000"/>

    </LinearLayout>

    <Button
        android:id="@+id/durationSaverBtn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Save">
    </Button>

</LinearLayout>
</androidx.constraintlayout.widget.ConstraintLayout>

```

3.3.2.10 settings_main.xml

This is the xml file decorating the page that themes can be switches between on.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/darkTheme"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="dark mode"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <Button
        android:id="@+id/lightTheme"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="light mode"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

3.3.3 To-Do List

The To-Do list is made up of many files that support UI responsiveness. I referred to many sources in order to create this part of my application:

- [<https://youtu.be/RzjCMa4GBD4?si=wmzFvWtdRUGxhDL9>]
- [<https://youtu.be/RfIR4oaSVfQ?si=ffkJx4Zv-Thp7fK4>]
- [<https://youtu.be/bOd3wO0uFr8?si=x50sZtzEcTZWhztD>]
- [https://youtu.be/sWOmIDvz_3U?si=1TAEmUtH0LBmiOaf]
- [<https://developer.android.com/training/data-storage/room>]

Something that often came up in my research to finding the best way to code this was the MVVM - model-view-viewmodel - architecture. It's a popular way to structure code in android apps to make it maintainable. This style of architecture has allowed my app to be more modular, meaning I can easily isolate specific functions and test them without worrying about how other parts of my app are affected.

The model part manages data by for example querying my database. Whatever it does, i aim for it to be independent and unaware of the UI aspects of the app.

The view part is related to what we view. So, the UI components such as fragments and activities. It does things like observe changes in the view model and update the UI to reflect so.

Finally, the ViewModel connects the model & view. It basically formats data from the model in a way that the UI can use.

3.3.3.1 TaskItem.kt

```
import androidx.room.Entity
import androidx.room.PrimaryKey
import java.time.LocalDate
import java.time.format.DateTimeFormatter
@Entity
class TaskItem(
    var name: String,
    var tag: String,
    var dueDateString: String?,
    var completed: Boolean = false,
    @PrimaryKey(autoGenerate = true) var taskID: Int = 0
) {

    fun dueDate(): LocalDate? = if (dueDateString == null) null
    else LocalDate.parse(dueDateString, dateFormatter)

    companion object {
        val dateFormatter: DateTimeFormatter = DateTimeFormatter.ISO_DATE
    }
}
```

My TaskItem class is an entity representing each task the user needs to do. This file simply defines the entity.

The function dueDate() converts dueDates from localDate format to a String, as the rooms database library I'm using doesn't support the LocalDate data type.

3.3.3.2 TaskItemDao.kt

```
import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow

@Dao
interface TaskItemDao {

    @Query("SELECT * FROM TaskItem")
    fun getAllTaskItems(): Flow<List<TaskItem>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertTaskItem(taskItem: TaskItem): Unit

    @Update
    suspend fun updateTaskItem(taskItem: TaskItem)

    @Delete
    suspend fun deleteTaskItem(taskItem: TaskItem)

    @Query("DELETE FROM TaskItem WHERE completed = 1")
    suspend fun deleteCompletedTasks()

}
```

DAOs (data access objects) allow me to perform CRUD operations on my database with SQL commands.

getAllTaskItems(): Returns every TaskItem in the database as a list

insertTaskItem(taskItem: TaskItem): Inserts a TaskItem, replacing any existing TaskItems if there are any conflicts

updateTaskItem(taskItem: TaskItem): Updates a TaskItem (in the case of an edit)

deleteCompletedTasks(): Deletes TaskItems in the database where completed = true

This ticks off part of:

- ☐ **M1** Users can create a task
- ☐ **M2** Users can edit a task
- ☐ **M4** Users can clear all completed tasks at the same time

3.3.3 TaskItemsDatabase.kt

```
import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
@Database(entities = [TaskItem::class], version = 1, exportSchema = false)
abstract class TaskItemsDataBase: RoomDatabase() {
    abstract fun taskItemDao(): TaskItemDao

    companion object {
        @Volatile
        private var INSTANCE: TaskItemsDataBase? = null

        fun getDatabase(context: Context): TaskItemsDataBase {
            return INSTANCE ?: synchronized(this){
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    TaskItemsDataBase::class.java,
                    "TASKITEMS_DB"

                ).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

This is a RoomDatabase() type class. The abstract function taskItemDao() gives the database access to the DAO. Only one instance of this class is created & used through the app's lifecycle. This is apparent in the way I used a companion object. It means that only a single instance of the database exists, belonging to any instance of the database object.

3.3.3.4 TaskItemRepository

```
import androidx.annotation.WorkerThread
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
class TaskItemRepository(private val taskItemDao: TaskItemDao) {

    val allTaskItems: Flow<List<TaskItem>> =
taskItemDao.getAllTaskItems().map { tasks ->
        tasks.sortedBy { it.completed }
    }

    @WorkerThread
    suspend fun insertTaskItem(taskItem: TaskItem){
        taskItemDao.insertTaskItem(taskItem)
    }

    @WorkerThread
    suspend fun updateTaskItem(taskItem: TaskItem) {
        taskItemDao.updateTaskItem(taskItem)
    }

    @WorkerThread
    suspend fun deleteTaskItem(taskItem: TaskItem) {
        taskItemDao.deleteTaskItem(taskItem)
    }

    @WorkerThread
    suspend fun deleteCompletedTasks() {
        taskItemDao.deleteCompletedTasks()
    }
}
```

The repository class was created to abstract the logic behind data handling from other parts of my app, for example the TaskViewModel. It ensures that those other parts don't directly interact with the data. It's a kind of procedural abstraction. I've also used worker threads to execute these functions as to not block the main thread. If the main thread were blocked, UI rendering may also be delayed or blocked, resulting in an unsatisfying and unresponsive experience. This is a reason that database queries should never be run on main threads.

3.3.3.5 TaskItemViewHolder.kt

```
import android.graphics.Paint
import android.view.View
import android.widget.ImageButton
import android.widget.TextView
import androidx.cardview.widget.CardView
import androidx.fragment.app.FragmentActivity
import androidx.recyclerview.widget.RecyclerView
import io.github.finnimo.tamadoro.R
import io.github.finnimo.tamadoro.fragments.NewTaskSheet
import java.time.format.DateTimeFormatter

class TaskItemViewHolder(
    private val view: View,
    private val listener: onTaskCompletedListener
): RecyclerView.ViewHolder(view) {

    private val taskName: TextView = view.findViewById(R.id.taskName)
    private val taskDueDate: TextView = view.findViewById(R.id.taskDueDate)
    private val checkBoxBtn: ImageButton =
view.findViewById(R.id.checkBoxBtn)
    private val cardView: CardView =
view.findViewById(R.id.taskItemContainer)

    fun bindTaskItem(taskItem: TaskItem) {
        taskName.text = taskItem.name

        if (taskItem.dueDate() != null) {
            taskDueDate.text =
taskItem.dueDate()?.format(DateTimeFormatter.ofPattern("dd/MM"))
        } else {
            taskDueDate.text = ""
        }

        if (taskItem.completed) {
            taskName.paintFlags = Paint.STRIKE_THRU_TEXT_FLAG
            checkBoxBtn.setImageResource(R.drawable.ic_checkbox_checked)
        } else {
            taskName.setPaintFlags(0)
            checkBoxBtn.setImageResource(R.drawable.ic_checkbox_unchecked)
        }

        checkBoxBtn.setOnClickListener {

            taskItem.completed = !taskItem.completed
            listener.onTaskCompletedChanged(taskItem)
        }
    }
}
```

```

        cardView.setOnClickListener {

            NewTaskSheet(taskItem).show((itemView.context as
            FragmentActivity).supportFragmentManager, "newTask")

        }

    }

}

```

The TaskItemViewHolder is what defines the UI of each card view of each unique task. The listener I pass through the class listens for whether there is a change in the completed attribute of that respective task. If there is, the UI is updated and the task name is striked through. ~~Like this~~. The private vals are simply linking UI components I've defined in an xml file. For example, a TextView component to hold the task's name.

The function bindTaskItem() just sets UI components to display the respective tasks' details such as name and due date.

The checkbox listener ensures that when the checkbox is clicked on, the respective task's completed attribute is toggled to be the opposite of what it was initially.

At the bottom, you can see that if you click on the card view, a BottomSheetDialogue that I've coded called NewTaskSheet is launched/shown. Basically, if I click on a task item, a box will show up at the bottom of the screen allowing users to edit details of the app.

3.3.3.6 OnTaskCompletedListener.kt

```

interface onTaskCompletedListener {
    fun onTaskCompletedChanged(taskItem: TaskItem)
}

```

This is just an interface, so no code is specified for the function. I defined the function in TasksFragment.kt:

```

override fun onTaskCompletedChanged(task: TaskItem) {
    taskViewModel.updateTaskItem(task)
}

```

It also passes itself (so the onTaskCompletedChanged function too) onto the adapter in this line:

```

adapter = TaskItemAdapter(this, emptyList())

```

Finally in the TaskItemViewHolder which listener is passed to as seen in lines:

```
class TaskItemViewHolder(  
    private val view: View,  
    private val listener: onTaskCompletedListener  
) : RecyclerView.ViewHolder(view) {
```

When the checkbox is clicked, `taskItem.completed` is changed to reflect the correct state, and the interface is called. When the interface is called, the `TasksFragment` does what was specified; update the current task. The reason why this interface was used is because I could not access the database through the `TaskItemViewHolder` class without once again redefining all the necessary components such as the repository and `TaskViewModel` & etc. So instead, I created an interface that spanned from `TasksFragment` which already had access to the database.

All in all, this allowed me to update the task's completed attribute by clicking on the checkbox on the Card View.

This completes part of the task:

- **M3** Users can check off a task
 - As this section enables checkboxes to be ticked off, UI-wise

3.3.3.7 TaskViewModel.kt

```
import androidx.lifecycle.LiveData
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.asLiveData
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.launch
import java.lang.IllegalArgumentException

class TaskViewModel(private val repository: TaskItemRepository): ViewModel()
{

    var taskItems: LiveData<List<TaskItem>> =
repository.allTaskItems.asLiveData()

    fun addTaskItem(newTask: TaskItem) = viewModelScope.launch {
        repository.insertTaskItem(newTask)
    }

    fun updateTaskItem(taskItem: TaskItem) = viewModelScope.launch {
        repository.updateTaskItem(taskItem)
    }

    fun deleteTaskItem(taskItem: TaskItem) = viewModelScope.launch {
        repository.deleteTaskItem(taskItem)
    }

    fun deleteCompletedtasks() = viewModelScope.launch {
        repository.deleteCompletedTasks()
    }

}

class TaskItemModelFactory(private val repository: TaskItemRepository):
ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(TaskViewModel::class.java)) {
            @Suppress("UNCHECKED_CAST")
            return TaskViewModel(repository) as T
        }
        throw IllegalArgumentException("Unknown class for View Model")
    }
}
```

As seen here, TaskViewModel simply just includes routines from the repository, allowing them to simply be called without any worry or relation to the actual coding logic. This is necessary, as for example:

```
suspend fun insertTaskItem(taskItem: TaskItem){
    taskItemDao.insertTaskItem(taskItem)
}
```

this is a function in the TaskItemRepository. If I were to stop using the Rooms library and scrap the entire need for a DAO, I would only need to change the logic in the Repository (in this case it's accessing the DAO and its functions).

At first I questioned whether this is very efficient, since the functions that need to be called were very simple. However:

- If I were to switch methods of data handling (so, not use rooms library anymore), at this current stage I can't possibly tell whether the code for handling data (in that future way) is complex or not. It would be safer to include a TaskViewModel and follow MVVM structure just in case.
- Also, I believed it would be good practice to follow this MVVM structure

I implemented this layer of abstraction in case I add more complex functions in the future. I believed it was good practice to incorporate this.

The TaskItemViewModelFactory needed to be implemented as I wanted to pass the repository to the TaskViewModel. It's to do with how the creation of View Models use a no-argument constructor.

3.3.3.8 TodoApplication.kt

```
import android.app.Application
class TodoApplication: Application() {
    private val database by lazy { TaskItemsDataBase.getDatabase(this) }
    val repository by lazy { TaskItemRepository(database.taskItemDao()) }
}
```

Since multiple separate parts of my app (with different lifecycles) require the database and repository, I created a TodoApplication class to make a singleton object; there is only one instance of the repo and database across the lifespan of my app. The two most critical and separate parts of my app that accesses these resources are:

- The actual TasksFragment (e.g when I create a task I will need to add one into the database)
- Each individual task item's card view
 - Basically, the card view is it's own separate object from the fragment, and thus has its own lifecycle and such. It still needs to access the database for when I click on a checkbox on a cardview though, so that's another crucial reason as to why I have the TodoApplication

I have used lazy, meaning the database is initialized and a repo is created only when they're accessed for the first time upon startup. This just saves resources and optimizes the flow of the app upon launch, however it isn't functionally necessary. This class is instantiated before any other classes.

3.3.3.9 TasksFragment.kt

Finally, this is the class that displays the UI components of the todo list, and links everything together.

```
import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import androidx.fragment.app.viewModels
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import io.github.finnimo.tamadoro.R
import io.github.finnimo.tamadoro.taskitemdatabase.TaskItemModelFactory
import io.github.finnimo.tamadoro.taskitemdatabase.TaskViewModel
import io.github.finnimo.tamadoro.taskitemdatabase.TODOApplication
import io.github.finnimo.tamadoro.taskitemdatabase.TaskItem
import io.github.finnimo.tamadoro.taskitemdatabase.TaskItemAdapter
import io.github.finnimo.tamadoro.taskitemdatabase.onTaskCompleteListener

class TasksFragment : Fragment(), onTaskCompleteListener {

    private val taskViewModel: TaskViewModel by viewModels {
        TaskItemModelFactory((requireActivity().application as
        TODOApplication).repository)
    }

    private lateinit var adapter: TaskItemAdapter
    private lateinit var newTaskBtn: Button
    private lateinit var clearBtn: Button
    private lateinit var recyclerView: RecyclerView
```

Here, the TaskViewModel is initialized, ready to allow access of data between UI and functionality logic. The adapter (see the next section) and some UI components are declared early, to be defined later. (RecyclerView is a view that can be scrolled through, like a list).

```
override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?,
    savedInstanceState: Bundle?
): View? {
    return inflater.inflate(R.layout.fragment_tasks, container, false)
}
```

This is just binding the fragment to an xml file I created with UI components for the todo list page.

```

override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)

    setRecyclerView()
    recyclerView = view.findViewById(R.id.todoListRecyclerView)
    newTaskBtn = view.findViewById(R.id.newTaskBtn)
    clearBtn = view.findViewById(R.id.clearCompletedBtn)

    adapter = TaskItemAdapter(this, emptyList())
    recyclerView.adapter = adapter

    taskViewModel.taskItems.observe(viewLifecycleOwner) { tasks ->
        adapter.setTasks(tasks)
    }
}

```

taskItems is a list of task items in the TaskViewModel class. There is an observer for change in this list. With each change, the adapter is updated with a new list of tasks. The new list/change in list will be used to update the UI. For example, it will remove the card view of a deleted task.

```

newTaskBtn.setOnClickListener {
    NewTaskSheet(null).show(childFragmentManager, "newTask")
    setRecyclerView()
}

clearBtn.setOnClickListener {
    taskViewModel.deleteCompletedtasks()
}
}

```

When newTaskBtn button is clicked, a new task sheet is shown, allowing users to create a new task. The clear button will delete all completed tasks.

```

override fun onTaskCompletedChanged(task: TaskItem) {
    taskViewModel.updateTaskItem(task)
}

private fun setRecyclerView() {
    taskViewModel.taskItems.observe(viewLifecycleOwner) { taskItems ->
        recyclerView.apply {
            layoutManager = LinearLayoutManager(requireContext())
            adapter = TaskItemAdapter(this@TasksFragment, taskItems)
        }
    }
}
}

```

Here, the interface for `onTaskCompletedChanged()` is defined. `setRecyclerView()` updates the UI to reflect the list of tasks correctly.

3.3.3.10 TaskItemAdapter

```
import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import io.github.finnimo.tamadoro.R

class TaskItemAdapter(

    private val listener: onTaskCompletedListener,
    private var taskItems: List<TaskItem>,

    ): RecyclerView.Adapter<TaskItemViewHolder>() {

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
TaskItemViewHolder {
        val itemView = LayoutInflater.from(parent.context)
            .inflate(R.layout.singular_task_item,parent,false)
        return TaskItemViewHolder(itemView, listener)
    }

    override fun onBindViewHolder(holder: TaskItemViewHolder, position: Int)
{
        holder.bindTaskItem(taskItems[position])
    }

    fun setTasks(newTasks: List<TaskItem>) {
        taskItems = newTasks
        notifyDataSetChanged()
    }

}
```

For each item in the `taskItems` list, the adapter will create a card view for the respective task. It also binds the unique attribute of each task to the card view, displaying attributes like name and due date. See `TaskItemViewHolder.kt`.

3.3.3.11 NewTaskSheet.kt

```
import android.content.DialogInterface
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.EditText
import androidx.lifecycle.ViewModelProvider
import com.google.android.material.bottomsheet.BottomSheetDialogFragment
import com.google.android.material.datepicker.MaterialDatePicker
import io.github.finnimo.tamadoro.R
import io.github.finnimo.tamadoro.taskitemdatabase.TaskItem
import io.github.finnimo.tamadoro.taskitemdatabase.TaskItemModelFactory
import io.github.finnimo.tamadoro.taskitemdatabase.TaskViewModel
import io.github.finnimo.tamadoro.taskitemdatabase.TODOApplication
import java.time.Instant
import java.time.LocalDate
import java.time.ZoneId

class NewTaskSheet(var taskItem: TaskItem?): BottomSheetDialogFragment() {

    private lateinit var taskViewModel: TaskViewModel
    private lateinit var taskNameInput: EditText
    private lateinit var tagInput: EditText
    private lateinit var datePickerBtn: Button
    private lateinit var deleteBtn: Button
    private lateinit var saveBtn: Button
    private var dueDate: LocalDate? = null

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        super.onCreateView(view, savedInstanceState)

        taskViewModel = ViewModelProvider(
            requireActivity(),
            TaskItemModelFactory((requireActivity().application as
TODOApplication).repository)
        ).get(TaskViewModel::class.java)

        taskNameInput = view.findViewById(R.id.name)
        tagInput = view.findViewById(R.id.tagEditText)
        datePickerBtn = view.findViewById(R.id.datePickerBtn)
        deleteBtn = view.findViewById(R.id.deleteBtn)
        saveBtn = view.findViewById(R.id.saveBtn)
```

A TaskViewModel object is initialized here in order to allow NewTaskSheet to manipulate task item data. Whether it be creating a new task, or editing an existing one. Variables are defined here too.

```

val datePicker: MaterialDatePicker<Long>

    if (taskItem != null) {
        taskNameInput.setText(taskItem!!.name)
        tagInput.setText(taskItem!!.tag)

        deleteBtn.visibility = Button.VISIBLE
        deleteBtn.setOnClickListener {
            taskViewModel.deleteTaskItem(taskItem!!)
            dismiss()
        }
    }

    if ((taskItem != null) and (taskItem?.dueDate() != null)) {

        datePickerBtn.text = taskItem!!.dueDateString
        val zoneID: ZoneId = ZoneId.systemDefault()
        val prevDueDate =
taskItem!!.dueDate()!!.atStartOfDay(zoneID).toInstant().toEpochMilli()
        datePicker =
            MaterialDatePicker.Builder.datePicker()
                .setTitleText("Select date")
                .setSelection(MaterialDatePicker.todayInUtcMilliseconds())
                .setSelection(prevDueDate)
                .build()

        //next 3 lines are so that if someone made no changes to dueDate,
it would be what the prev already set dueDate was.
        dueDate = Instant.ofEpochMilli(prevDueDate)
            .atZone(ZoneId.systemDefault())
            .toLocalDate()

    } else {
        datePicker =
            MaterialDatePicker.Builder.datePicker()
                .setTitleText("Select date")
                .setSelection(MaterialDatePicker.todayInUtcMilliseconds())
                .build()
    }
}

```

If `taskItem != null`, it signifies that we are editing an existing task. In this case, we set the name and tag as what it already is in the database. Also, there will be a delete button visible. If the task item exists and the due date isn't empty, the date picker will reflect the date that was previously chosen for this task. If there is not a pre-existing date, then the date picker will display "select date".

```

datePickerBtn.setOnClickListener {
    datePicker.show(childFragmentManager, "datePicker")
}

datePicker.addOnPositiveButtonClickListener { selectedDate ->

    dueDate = Instant.ofEpochMilli(selectedDate)
        .atZone(ZoneId.systemDefault())
        .toLocalDate()

    datePickerBtn.text = dueDate.toString()

}

```

When the date picker button is clicked on, a date picker will show up. The due date variable is set to whatever the user picks.

```

saveBtn.setOnClickListener {
    saveAction()
}

override fun onDismiss(dialog: DialogInterface) {
    super.onDismiss(dialog)

    if (taskItem != null) {
        saveAction()
    }
}

private fun saveAction() {
    val tag = tagInput.text.toString()
    var name = taskNameInput.text.toString()

    if (taskNameInput.text.toString() == "") {
        name = "New Task"
    }

    val dueDateString = if (dueDate == null) null
    else TaskItem.dateFormatter.format(dueDate)

    if (taskItem == null) {
        val newTask = TaskItem(name, tag, dueDateString)
        taskViewModel.addTaskItem(newTask)
    }
}

```

```
else {  
    taskItem!!.name = name  
    taskItem!!.dueDateString = dueDateString  
    taskViewModel.updateTaskItem(taskItem!!)  
}  
  
dismiss()  
}
```

The save button will call `saveAction()`. When the sheet is dismissed, the data is saved if the task item is pre-existing.

`saveAction()` converts the editable text to `String` so it can be stored in the database. If task name is empty, then the name is automatically saved as "New Task". If `taskItem` isn't pre-existing, a new `taskItem` is added. If it is pre-existing, the attributes of the existing tasks are altered. The `taskViewModel` is updated to reflect the change in this existing task item.

This meets the objective:

■ **M1** Users can create a task

- A task sheet popping up to enter the details of a new task aids in creating a task

■ **M2** Users can edit a task

- The task sheets allow users to edit attributes of existing tasks

3.3.3.12 fragment_tasks.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="?attr/colorPrimary"
    android:clickable="true"
    android:focusable="true"
    android:focusableInTouchMode="true"
    tools:context="io.github.finnimo.tamadoro.activities.MainActivity">

    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <LinearLayout
            android:id="@+id/linearLayout2"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:orientation="vertical"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent">
```

```
<TextView
    style="@style/TextAppearance.AppCompat.Title"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="20dp"
    android:layout_marginLeft="20dp"
    android:text="To-Do"
    android:textSize="30dp"
    android:textColor="?attr/colorOnPrimary">
```

```
</TextView>
```

```
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/todoListRecyclerView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
```

```
</LinearLayout>
```

```
<Button
    android:id="@+id/newTaskBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_margin="16dp"
    android:textColor="?attr/colorOnPrimary"
    android:layout_gravity="bottom|right"
    android:text="new task" />
```

```
<Button
    android:id="@+id/clearCompletedBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_margin="16dp"
    android:textColor="?attr/colorOnPrimary"
    android:layout_gravity="bottom|left"
    android:text="Clear" />
```

```
</FrameLayout>
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

3.3.3.13 singular_task_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/taskItemContainer"
    android:layout_width="match_parent"
    android:layout_height="90dp"
    app:cardCornerRadius="5dp"
    android:layout_marginBottom="10dp"
    android:layout_marginHorizontal="7dp"
    app:cardBackgroundColor="?attr/colorPrimary">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <ImageButton
            android:id="@+id/checkboxBtn"
            android:layout_width="100dp"
            android:layout_height="44dp"
            android:layout_gravity="center"
            android:backgroundTint="@android:color/transparent"
            android:scaleX="1.5"
            android:scaleY="1.5"
            android:src="@drawable/ic_checkbox_unchecked"/>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:orientation="vertical">

            <TextView
                android:id="@+id/taskDueDate"
                style="@style/TextAppearance.AppCompat.Title"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_gravity="left"
                android:textSize="15dp"
                android:layout_marginTop="8dp"
                android:text="Due date"
                android:textColor="?attr/colorOnPrimary"/>
        </LinearLayout>
    </LinearLayout>
</CardView>
```

```

        <TextView
            android:id="@+id/taskName"
            style="@style/TextAppearance.AppCompat.Title"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="left"
            android:textSize="27dp"
            android:text="task name"
            android:textColor="?attr/colorOnPrimary"/>

    </LinearLayout>

</LinearLayout>

</androidx.cardview.widget.CardView>

```

3.3.3.14 fragment_new_task_sheet.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".fragments.NewTaskSheet">

    <LinearLayout
        android:id="@+id/linearLayout2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginVertical="15dp"
        android:orientation="horizontal"
        android:paddingHorizontal="30dp"
        android:paddingBottom="30dp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/linearLayout">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_vertical"
            android:text="Due Date: "
            android:textSize="25dp" />
    
```

```
<Button
    android:id="@+id/datePickerBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="None" />
```

```
</LinearLayout>
```

```
<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/name"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center"
    android:hint="New Task"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:textSize="30dp"
    android:padding="30dp"
    android:layout_margin="10dp"
/>
```

```
<LinearLayout
    android:id="@+id/linearLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    app:layout_constraintTop_toBottomOf="@+id/name"
    android:paddingHorizontal="30dp"
    android:layout_marginTop="30dp"
>
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_vertical"
    android:text="Tag: "
    android:textSize="25dp"
/>
```

```
<EditText
    android:id="@+id/tagEditText"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:gravity="left"
    android:hint="Enter Tag"
    android:background="@android:color/transparent"
    android:textSize="25dp"
/>
```

```
</LinearLayout>
```

```
<Button
    android:id="@+id/saveBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:paddingLeft="30dp"
    android:paddingBottom="24dp"
    android:text="Save"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/linearLayout2" />
```

```
<Button
    android:id="@+id/deleteBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:paddingRight="15dp"
    android:paddingBottom="24dp"
    android:text="Delete"
    android:visibility="invisible"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/linearLayout2" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

3.3.4 Sessions & Statistics

3.3.4.1 Session.kt

This is the user defined data class I made:

```
@Entity
data class Session (
    @PrimaryKey(autoGenerate = true) val id: Int = 0, //used to identify each
    row
    var minutes: Int,
    var tag: String = "None",
    var dateTime: Long = System.currentTimeMillis(),
)
```

Refer to 2.4.2 for the design.

3.3.4.2 SessionDao.kt

```
@Dao
public interface SessionDao {
    //add 1 session
    @Insert
    fun addSession(sessions : Session)//parameter is of Session data type
    //delete all sessions
    @Query("DELETE FROM Session")
    fun deleteAllSessions()
    //get all sessions
    @Query("SELECT * FROM Session")
    fun getAllSessions() : List<Session>
    //get all sessions of a specific tag
    @Query("SELECT * FROM Session WHERE tag = :tag")
    fun getSessionByTag(tag : String): List<Session>
    //get total duration
    @Query("SELECT SUM(minutes) FROM Session")
    fun getTotalDuration(): Int
    //get duration from today
    @Query("SELECT SUM(minutes) FROM Session WHERE ((dateTime >= :startDate)
and (dateTime <= :endDate))")
    fun getTotalDurationInPeriod(startDate: Long, endDate: Long): Int
    //currently working in seconds, make sure it works in minutes
    @Query("SELECT * FROM Session WHERE dateTime >= :startDate")
    fun getSessionInPeriod(startDate: Long): List<Session>
}
```

As seen above, the SessionDao is an interface allowing me to perform SQL instructions on my database. I've assigned SQL instructions to kotlin functions.

3.3.4.3 SessionsDatabase.kt

```
import androidx.room.Database
import androidx.room.RoomDatabase
@Database(entities = [Session::class], version = 1)

//a layer of abstraction simplifying database management. it allows me to use
//my entities and Daos
//essentially handles boilerplate code that comes with using SQLite directly
abstract class SessionsDatabase : RoomDatabase() {
    abstract fun getSessionDao() : SessionDao

    companion object {
        const val NAME = "Sessions_DB"
    }
}
```

See the comments above.

3.3.4.4 SessionManager.kt

```
import android.content.Context
import android.util.Log
import androidx.room.Room
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.util.Calendar
class SessionManager(context: Context) {

    private var database = Room.databaseBuilder(
        context.applicationContext,
        SessionsDatabase::class.java,
        SessionsDatabase.NAME
    ).build()

    private val sessionDao = database.getSessionDao()

    private fun timestamp(): String {
        return System.currentTimeMillis().toString()
    }

    suspend fun getAllSessions(): List<Session> {
        return withContext(Dispatchers.IO) {
            sessionDao.getAllSessions()
        }
    }
}
```

This is the first half of this class's file. Here, I have built the database that's to be interacted with, and created a sessionDao object in order to carry out the SQL instructions seen in the section before.

Below, the addSession function takes the parameters of duration in minutes of a session along with tag name. A new session is created and inserted into the database via the sessionDao's methods. dateTime is automatically taken from the user's phone's date/time settings. The Log.e session allows me to clearly see if a session has been logged or not, in console, without checking the database.

```
fun addSession(minutes: Int, tag: String) { //change this to minutes when
necessary
    GlobalScope.launch(Dispatchers.IO) {
        val newSession = Session(
            minutes = minutes,
            dateTime = System.currentTimeMillis(),
            tag = tag
        )
        sessionDao.addSession(newSession)
        Log.e("SessionManager.addSession", "Session added on
${timestamp()}")
    }
}
```

The functions below are just further procedural abstractions from the sessionDao. It's so that I can simply use SessionManager to manage sessions, rather than initializing a Dao straight into the fragments file as the code would be too complex to maintain. This created modularity. If I wanted to initialize a sessionDao and other tool to manipulate data, I would have to add many more parameters into my PetFragment file.

```
fun deleteAllSessions() {
    GlobalScope.launch(Dispatchers.IO) {
        sessionDao.deleteAllSessions()
        Log.d("SessionManager.deleteAllSessions()", "All sessions deleted on
${timestamp()} ")
    }
}

fun getTotalDuration(): Int {
    return sessionDao.getTotalDuration()
}

fun getTotalDurationThisWeek(start: Long): Int {
    //val currentTime = System.currentTimeMillis()
    return
    sessionDao.getTotalDurationInPeriod(start, System.currentTimeMillis())
}
```

The last 2 functions in this file make use of the calendar library to find the start of the day or week to find the duration that has been focused for since then.

```
fun getTotalDurationThisWeek(): Int {
    val calendar = Calendar.getInstance()
    calendar.set(Calendar.HOUR_OF_DAY, 0)
    calendar.set(Calendar.MINUTE, 0)
    calendar.set(Calendar.SECOND, 0)
    calendar.set(Calendar.MILLISECOND, 0)
    val startOfDay = calendar.timeInMillis
    val durationThisWeek: Int =
        sessionDao.getTotalDurationInPeriod(startOfDay, System.currentTimeMillis())
    return durationThisWeek
}

fun getTotalSessionToday(): Int {
    val calendar = Calendar.getInstance()
    calendar.set(Calendar.HOUR_OF_DAY, 0)
    calendar.set(Calendar.MINUTE, 0)
    calendar.set(Calendar.SECOND, 0)
    calendar.set(Calendar.MILLISECOND, 0)
    val startOfDay = calendar.timeInMillis
    val sessionsToday = sessionDao.getSessionInPeriod(startOfDay)
    Log.d("no. of sessions", sessionsToday.size.toString())
    return sessionsToday.size
}
```

3.3.4.5 Statistics.kt

This class mainly handles streaks.

Below are functions to retrieve SharedPreferences (local storage locations) of currentStreak and HighestStreak, along with a function that returns the currentStreak and highestStreak itself.

```
import android.content.Context
import android.content.SharedPreferences
import android.util.Log
import java.time.Instant
import java.time.LocalDateTime
import java.time.ZoneId
import java.time.temporal.ChronoUnit

class Statistics {
    private fun getCurrentStreakSP(context: Context): SharedPreferences {
        return context.getSharedPreferences("TAMADORO_CURRENTSTREAK",
Context.MODE_PRIVATE)
    }

    fun getCurrentStreak(context: Context): Int {
        val sharedPref = getCurrentStreakSP(context)
        return sharedPref.getInt("TAMADORO_CURRENTSTREAK", 0)
    }

    fun getHighestStreak(context: Context): Int {
        val sharedPref = getHighestStreakSP(context)
        return sharedPref.getInt("TAMADORO_HIGHESTSTREAK", 0)
    }

    private fun getHighestStreakSP(context: Context): SharedPreferences {
        return context.getSharedPreferences("TAMADORO_HIGHESTSTREAK",
Context.MODE_PRIVATE)
    }
}
```

editCurrentStreak is used to either increment a streak value or reset it to 0.

```
private fun editCurrentStreak(context: Context, type: Boolean) {
    val currentStreakSP = getCurrentStreakSP(context)
    var currentStreak = getCurrentStreak(context)
    if (type) {
        currentStreak++
        currentStreakSP.edit().putInt("TAMADORO_CURRENTSTREAK",
currentStreak).apply()
    }
    else {
        currentStreakSP.edit().putInt("TAMADORO_CURRENTSTREAK", 0).apply()
    }
}
```

updateLastSessionDate is called each time a session is logged, so we can see what day the user last studied.

```
fun updateLastSessionDate(context: Context){
    val sharedPref =
context.getSharedPreferences("TAMADORO_LASTDATE", Context.MODE_PRIVATE)
    val newDate = System.currentTimeMillis()

    sharedPref.edit().putLong("TAMADORO_LASTDATE", newDate).apply()
    Log.d("Last Session Date recorded:", "new last date:" +
sharedPref.getLong("TAMADORO_LASTDATE", 0).toString())
}
```

updateStreaks checks the date difference between the last time a session was logged and the current date. (The code is on the next page)

```
fun updateStreaks(context: Context) {
    val lastSessionSP =
context.getSharedPreferences("TAMADORO_LASTDATE", Context.MODE_PRIVATE)
    val currentStreak = getCurrentStreak(context)

    val lastSessionLong =
lastSessionSP.getLong("TAMADORO_LASTDATE", 0)

    val lastSessionLocalDate =
LocalDateTime.ofInstant(Instant.ofEpochMilli(lastSessionLong),
ZoneId.systemDefault()).toLocalDate()

    val currentDate = LocalDateTime.now().toLocalDate()
    val daysApart = ChronoUnit.DAYS.between(lastSessionLocalDate,
currentDate).toInt()

    Log.d("Days Apart last session date & current date",
daysApart.toString())

    if (lastSessionLong == 0.toLong()) {

        editCurrentStreak(context, true)

    } else {
        if ((daysApart == 1) || (currentStreak == 0)) {
            //if its been 1 day since last session:
            editCurrentStreak(context, true)
        } else if (daysApart > 1) {
            // Reset if its been more than 1 day
            editCurrentStreak(context, false)
        }
    }
    updateHighestStreak(context)
}
```

```

    Log.d("Last session logged at:
",lastSessionSP.getLong("TAMADORO_LASTDATE", 0).toString())
    Log.d("Current time:", System.currentTimeMillis().toString())
    Log.d("current streak:", getCurrentStreak(context).toString())
}

```

The functions below ensure that if the current streak is updated and it's higher than the highest streak logged, then the highest streak is updated to reflect the new highest streak. The last function ensures that the streaks displayed upon app opening is correct. If it's been days since the user last focused, their streak will be reset to 0.

```

        private fun updateHighestStreak(context: Context) {
            val highestStreak = getHighestStreak(context)
            val currentStreak = getCurrentStreak(context)
            if (currentStreak >= highestStreak) {
                context.getSharedPreferences("TAMADORO_HIGHESTSTREAK",
Context.MODE_PRIVATE).edit().putInt("TAMADORO_HIGHESTSTREAK",currentStreak).apply()
            }
        }

        fun streakCheckOnViewCreated(context: Context): Int{
            val lastSessionSP =
context.getSharedPreferences("TAMADORO_LASTDATE", Context.MODE_PRIVATE)
            val lastSessionLong = lastSessionSP.getLong("TAMADORO_LASTDATE",0)
            val lastSessionLocalDate =
LocalDateTime.ofInstant(Instant.ofEpochMilli(lastSessionLong),
ZoneId.systemDefault())

            val daysApart = ChronoUnit.DAYS.between(lastSessionLocalDate,
LocalDateTime.now()).toInt()
            if (daysApart > 1) {
                editCurrentStreak(context, false)
            }
            return getCurrentStreak(context)
        }
    }
}

```

The above section covers this objective:

- ☑ **T8** Streak measuring how many days the user has focused consecutively
 - As this class handles incrementation of streaks each day, and checking to see if it has been broken each day

3.3.4.6 StatsFragment.kt

The fragment below is just assigning values to UI components to reflect the user's stats, using the functions in the Statistics.kt file.

```
import android.os.Bundle
import android.util.Log
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.lifecycle.LifecycleScope
import io.github.finnimo.tamadoro.R
import io.github.finnimo.tamadoro.sessiondatabase.SessionManager
import io.github.finnimo.tamadoro.sessiondatabase.Statistics
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.math.RoundingMode
import java.time.DayOfWeek
import java.time.LocalDate
import java.time.ZoneId

class StatsFragment : Fragment() {

    private lateinit var manager: SessionManager
    private lateinit var highestStreakTV: TextView
    private lateinit var currentStreakTV: TextView
    private lateinit var totalDurationTV: TextView
    private lateinit var totalDurationTodayTV: TextView
    private lateinit var totalDurationThisWeekTV: TextView
    private lateinit var averageSessionDurationTV: TextView

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_stats, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        manager = SessionManager(requireContext())
        GlobalScope.launch(Dispatchers.Main) {
            val sessions = manager.getAllSessions()
            //val sessionDuration = manager.getTotalDuration()
            // averageSessionDurationTV.text = (sessionDuration/
            size).toString()
        }
    }
}
```

manager.getAllSessions() is called to ensure the database is initialized before attempts to access it are made.

For the next section of code, no logic is coded, it is mainly retrieving data with functions from the Statistics files and outputting them in text views.

```
//initializing text views

        highestStreakTV = view.findViewById(R.id.highestStreakTextView)
        currentStreakTV = view.findViewById(R.id.currentStreakTextView)
        totalDurationTV = view.findViewById(R.id.totalDurationTextView)
        totalDurationTodayTV =
view.findViewById(R.id.totalDurationTodayTextView)
        totalDurationThisWeekTV =
view.findViewById(R.id.totalDurationThisWeekTextView)
        averageSessionDurationTV =
view.findViewById(R.id.averageSessionDuration)

        viewLifecycleOwner.lifecycleScope.launch {
            val totalDuration = withContext(Dispatchers.IO) {
                manager.getTotalDuration()
            }

            val totalDurationToday = withContext(Dispatchers.IO) {
                val startOfDay =
LocalDate.now().atStartOfDay(ZoneId.systemDefault()).toInstant().toEpochMilli()
                manager.getTotalDurationThisWeek(startOfDay)
            }

            val totalDurationThisWeek = withContext(Dispatchers.IO) {
                val startOfWeek =
(LocalDate.now().with(DayOfWeek.MONDAY)).atStartOfDay(ZoneId.systemDefault())
                .toInstant()
                .toEpochMilli()
                manager.getTotalDurationThisWeek(startOfWeek)
            }

            totalDurationTV.text =
"$${((totalDuration.toDouble()/3600).toBigDecimal().setScale(2,
RoundingMode.UP).toDouble())} Hrs"
            totalDurationTodayTV.text =
"$${((totalDurationToday.toDouble()/3600).toBigDecimal().setScale(2,
RoundingMode.UP).toDouble())} Hrs"
            totalDurationThisWeekTV.text =
"$${(totalDurationThisWeek.toDouble()/3600).toBigDecimal().setScale(2,
RoundingMode.UP).toDouble())} Hrs"
        }
```

```
val highestStreak = Statistics.getHighestStreak(requireContext())
highestStreakTV.text = highestStreak.toString()

Statistics.streakCheckOnViewCreated(requireContext())
val currentStreak =
Statistics.getCurrentStreak(requireContext()).toString()
Log.d("current Streak", currentStreak)
currentStreakTV.text = currentStreak

}
```

Throughout this entire file, the following objectives are completed:

- **T10** Users should be able to view the number of sessions completed today, their current and highest streak, and the number of hours focused for today.
 - In this file, relevant values are retrieved using the SessionManager class, to be displayed onto UI components

3.3.4.7 fragment_stats.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".fragments.StatsFragment">

    <!-- TODO: Update blank fragment layout -->

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">

        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="vertical">
            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Highest Streak">
            </TextView>

            <TextView
                android:id="@+id/highestStreakTextView"
                android:layout_width="76dp"
                android:layout_height="104dp"
                android:text="highest streak"
                android:textSize="30dp"/>

        </LinearLayout>
```

```

<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Current Streak">
    </TextView>
    <TextView
        android:id="@+id/currentStreakTextView"
        android:layout_width="85dp"
        android:layout_height="83dp"
        android:text="current streak"
        android:textSize="30dp" />
</LinearLayout>

<LinearLayout
    android:id="@+id/allTimeContainer"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="All Time">

    </TextView>

    <TextView
        android:id="@+id/totalDurationTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="all time"
        android:textSize="30dp"/>
</LinearLayout>

<LinearLayout
    android:id="@+id/thisWeekContainer"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This Week">
    </TextView>

```

```

        <TextView
            android:id="@+id/totalDurationThisWeekTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="this week" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        app:layout_constraintTop_toTopOf="parent">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Today">

    </TextView>

    <TextView
        android:id="@+id/totalDurationTodayTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="TextView" />
    </LinearLayout>

</LinearLayout>
<TextView
    android:id="@+id/averageSessionDuration"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Average"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

3.3.5 Pet Functions

3.3.5.1 Pet.kt

This class handles all pet related data flow.

```
import android.content.Context
import android.content.SharedPreferences
import java.util.Calendar
class Pet(val context: Context) {

    private val totalCoins: Int =
getSharedPrefs(context, "TAMADORO_COINS").getInt("TAMADORO_COINS", 0)
    private var weeklyGoal: Int =
getSharedPrefs(context, "TAMADORO_MINS_GOAL").getInt("TAMADORO_MINS_GOAL", 0)
```

Total coins and weekly goal are both values that are retrieved from local storage. If they don't already exist, the value returned is 0.

```
fun getSharedPrefs(context: Context, name: String): SharedPreferences{
    return context.getSharedPreferences(name, Context.MODE_PRIVATE)
}
```

The above function returns access to sharedPreferences, so that the number of times I repeatedly type this line out is reduced.

Below, the feature of coins is purely experimental.

```
fun getTotalCoins(): Int {
    return totalCoins
}

fun getWeeklyGoal(): Int {
    return weeklyGoal
}
```

The two functions above return the values of the private variables totalCoins and weeklyGoal. This design choice prevents totalCoins and weeklyGoal from being inappropriately accessed and altered.

```
fun changeWeeklyGoal(mins: Int) {

getSharedPrefs(context, "TAMADORO_MINS_GOAL").edit().putInt("TAMADORO_MINS_GOAL", mins).commit()
    weeklyGoal =
getSharedPrefs(context, "TAMADORO_MINS_GOAL").getInt("TAMADORO_MINS_GOAL", 0)
}
```

Above, the contents of the local storage location where the weekly goal is stored is altered to the user's liking.

```

fun addCoins(durationFocused: Int) {
    var coins = totalCoins
    val noOfCoins = (0.1 * durationFocused).toInt()
    coins += noOfCoins

    getSharedPreferences(context, "TAMADORO_COINS").edit().putInt("TAMADORO_COINS", coins).apply()
}

```

In `addCoins`, a multiplier is applied to the duration of the study session, returning the number of coins that the user gained from it. Then, this value is added to the original value. `addCoins()` is just an experimental feature for now.

```

fun getCurrentFocusQuota(): Int {
    val calendar = Calendar.getInstance()
    var dayOfWeekInt = calendar.get(Calendar.DAY_OF_WEEK)

    dayOfWeekInt = if (dayOfWeekInt == 1) {
        7
    } else {
        dayOfWeekInt - 1
    }
    // above is so that monday is always day 1
    // this return the amount of hours that shouldve been studied already
    this week
    return dayOfWeekInt * weeklyGoal / 7
}

```

The current focus quota is calculated. The focus quota is the number of hours the users should have studied up until the current day, in order to meet their weekly goal.

```

fun onTrack(duration: Int): Boolean {
    val currentQuota = getCurrentFocusQuota()
    return duration >= currentQuota
}

```

On track checks if the user has met their focus quota.

```

fun happinessChecker(onTrack: Boolean, focusedToday: Boolean): Int {
    var tempHappiness = 3
    if (!focusedToday) {
        tempHappiness -= 1
    }
    if (!onTrack) {
        tempHappiness -= 1
    }

    getSharedPreferences(context, "TAMADORO_HAPPINESS").edit().putInt("TAMADORO_HAPPINESS", tempHappiness)
    return
    getSharedPreferences(context, "TAMADORO_HAPPINESS").getInt("TAMADORO_HAPPINESS", 0)
}

```

Happiness checker returns a happiness value of 1, 2 or 3, 3 being the highest happiness, based on criteria described in Design.

The following objective is completed here:

- ✓ **P1** Pets have a happiness meter which increases the more students focus and declines the more students miss their study goal
 - If the student has focused 0 hours today and aren't meeting their dailyFocusQuota, the happiness is directly affected
- ✓ **P2** Happiness meter is related to the goal time that the user sets for themselves
 - The value of the variable 'onTrack' is based on the weekly goal hours the user has set. This variable is used to calculate happiness, thus meeting the objective of the focus goal being related to happiness.

3.3.5.2 PetFragment.kt

The pet fragment is the screen on which the pet, happiness meter and all functions surrounding the pet will run on.

In the code below, private variables containing UI components of the pet page are declared lateinit.

```
import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.ImageView
import android.widget.LinearLayout
import android.widget.NumberPicker
import android.widget.TextView
import androidx.lifecycle.LifecycleScope
import io.github.finnimo.tamadoro.fragments.pethandling.Pet
import io.github.finnimo.tamadoro.R
import io.github.finnimo.tamadoro.sessiondatabase.SessionManager
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.math.RoundingMode

class PetFragment : Fragment() {

    private lateinit var coinsTV: TextView
    private lateinit var meter: ImageView
    private lateinit var weeklyGoalTV: TextView
    private lateinit var sessionsTodayTV: TextView
    private lateinit var pet: Pet
    private lateinit var manager: SessionManager
```

A pet and session managing object is created, and the manager fetches the session first thing, to prevent later access attempts to an unopened database.

```
override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)

    pet = Pet(requireContext())
    manager = SessionManager(requireContext())
    GlobalScope.launch(Dispatchers.Main) {
        manager.getAllSessions()
    }
}
```

```

viewLifecycleOwner.lifecycleScope.launch {
    val sessionsToday = withContext(Dispatchers.IO) {
        manager.getTotalSessionToday()
    }
    val currentWeekDuration = withContext(Dispatchers.IO) {
        manager.getTotalDurationThisWeek()
    }

    val focusedToday = (sessionsToday > 0)
    sessionsTodayTV.text = sessionsToday.toString()

    val happiness =
pet.happinessChecker(pet.onTrack(currentWeekDuration), focusedToday)
    if (happiness == 1) {
        meter.setImageResource(R.drawable.happiness_1)
    } else if (happiness == 2) {
        meter.setImageResource(R.drawable.happiness_2)
    } else {
        meter.setImageResource(R.drawable.happiness_3)
    }

}

```

That section of the code is used to calculate the happiness of the pet and then display the corresponding UI component. It will crash if not coded using a lifecycle scope. This is to prevent crashes from trying to update UI components that no longer exist when a fragment's view has been destroyed. (E.g the user leaves the pet fragment to enter the timer fragment.)

Here, the following objective is met when sessionsTodayTV is assigned a value:

- **T10** Users should be able to view the number of sessions completed today, their current and highest streak, and the number of hours focused for today.
 - Because sessionsTodayTV is a UI component containing sessions focused for today that the user can see

Below, some UI components are given values to show the user.

```
coinsTV = view.findViewById(R.id.coinsTV)
meter = view.findViewById(R.id.happinessMeter)
weeklyGoalTV = view.findViewById(R.id.weeklyGoalTV)
sessionsTodayTV = view.findViewById(R.id.sessionsTodayTV)

coinsTV.text = pet.getTotalCoins().toString()
var goalInHrs =
    (pet.getWeeklyGoal().toDouble()/60).toBigDecimal().setScale(2,
    RoundingMode.UP).toDouble()

weeklyGoalTV.text = goalInHrs.toString()

val pickerContainer = view.findViewById<LinearLayout>(R.id.pickerContainer)
    val hoursPicker = view.findViewById<NumberPicker>(R.id.hoursPicker)
    val minutesPicker =
view.findViewById<NumberPicker>(R.id.minutesPicker)
    val saveDurationBtn =
view.findViewById<Button>(R.id.durationSaverBtn)

weeklyGoalTV.setOnClickListener {
    pickerContainer.visibility = LinearLayout.VISIBLE

    hoursPicker.minValue = 0
    hoursPicker.maxValue = 120
    minutesPicker.minValue = 0
    minutesPicker.maxValue = 59

    val initHours: Int = (pet.getWeeklyGoal() / 60)
    val initMins: Int = pet.getWeeklyGoal() % 60

    hoursPicker.value = initHours
    minutesPicker.value = initMins
```

```

saveDurationBtn.setOnClickListener {

    val hours = hoursPicker.value
    val mins = minutesPicker.value

    var durationInMins: Int = (hours * 60) + mins

    pet.changeWeeklyGoal(durationInMins)
    goalInHrs =
    (pet.getWeeklyGoal().toDouble()/60).toBigDecimal().setScale(2,
    RoundingMode.UP).toDouble()

    weeklyGoalTV.text = goalInHrs.toString()
    pickerContainer.visibility = LinearLayout.INVISIBLE

}

}

```

The code above allows for users to enter a weekly goal in minutes to focus for, using a number picker. The code just ensures that the number picker opens up and shows values that were previously used. This section was purely for a smoother UI.

```

override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View? {

    val view = inflater.inflate(R.layout.fragment_pet, container, false)

    return inflater.inflate(R.layout.fragment_pet, container, false)
}

```

The code above is a function that comes automatically with every fragment and ensures that the fragment is tied to an XML file that contains and described UI components. For example, a white button that opens up a number picker.

3.3.5.3 fragment_pet.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".fragments.PetFragment">

    <TextView

        android:id="@+id/textView"
        android:layout_width="match_parent"

        android:layout_height="wrap_content"
        android:layout_marginLeft="100dp"
        android:layout_marginTop="20dp"
        android:text="Happiness %"
        android:textColor="?attr/colorOnPrimary"

        app:layout_constraintTop_toTopOf="parent">

    </TextView>
```

```

<ImageView
    android:id="@+id/happinessMeter"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:adjustViewBounds="true"

    android:padding="0dp"
    android:src="@drawable/happiness_1"
    app:layout_constraintEnd_toStartOf="@+id/empty_space"
    app:layout_constraintHorizontal_bias="1.0"
    app:layout_constraintStart_toEndOf="@+id/coinsTV"
    app:layout_constraintTop_toBottomOf="@+id/textView"
    app:layout_constraintWidth_percent="0.6" />

<View
    android:id="@+id/empty_space"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintWidth_default="percent"
    app:layout_constraintWidth_percent="0.2">
</View>

<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    app:layout_constraintEnd_toStartOf="@+id/happinessMeter"
    app:layout_constraintTop_toBottomOf="@+id/textView"
    app:layout_constraintWidth_percent="0.2">

    <ImageView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:adjustViewBounds="true"
        android:src="@drawable/ic_coin">
    </ImageView>

    <TextView
        android:id="@+id/coinsTV"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="000"
        android:textColor="?attr/colorOnPrimary"
        android:textSize="30dp">
    </TextView>

</LinearLayout>

```

```

<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.5"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/happinessMeter">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Goal Hrs:"
        android:textColor="?attr/colorOnPrimary"
        android:textSize="20dp"></TextView>

    <TextView
        android:id="@+id/weeklyGoalTV"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:text="Weekly Goal"
        android:textColor="?attr/colorOnPrimary"
        android:textSize="30dp"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/happinessMeter" />
</LinearLayout>

<androidx.constraintlayout.widget.Guideline
    android:id="@+id/guideline4"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    app:layout_constraintGuide_percent="0.28170595" />

<ImageView
    android:id="@+id/imageView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:adjustViewBounds="true"
    android:src="@drawable/cat_1"
    app:layout_constraintTop_toTopOf="@+id/guideline4"
    tools:layout_editor_absoluteX="0dp" />

```

```

<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/imageView">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Today you have completed"
        android:textColor="?attr/colorOnPrimary"
        android:textSize="27dp">

    </TextView>

    <TextView
        android:id="@+id/sessionsTodayTV"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Sessions Today"
        android:textColor="?attr/colorOnPrimary"
        android:textSize="40dp"
        app:layout_constraintTop_toTopOf="@+id/guideline3"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="sessions"
        android:textColor="?attr/colorOnPrimary"
        android:textSize="28dp">

    </TextView>

</LinearLayout>

<LinearLayout
    android:id="@+id/pickerContainer"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:background="?attr/colorPrimary"
    android:gravity="center"
    android:orientation="vertical"
    android:textColor="?attr/colorOnPrimary"
    android:visibility="invisible"
    app:layout_constraintEnd_toEndOf="parent"
    tools:layout_editor_absoluteY="147dp">

```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Select Goal"
    android:textColor="?attr/colorOnPrimary">

</TextView>

<LinearLayout

    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <NumberPicker
        android:id="@+id/hoursPicker"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="?attr/colorOnPrimary" />

    <NumberPicker
        android:id="@+id/minutesPicker"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="#FF0000" />

</LinearLayout>

<Button
    android:id="@+id/durationSaverBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Save">

</Button>

</LinearLayout>

<androidx.constraintlayout.widget.Barrier
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:barrierDirection="top" />

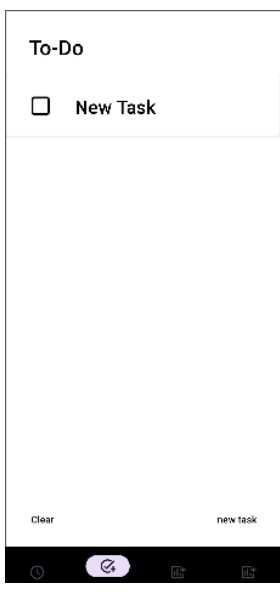
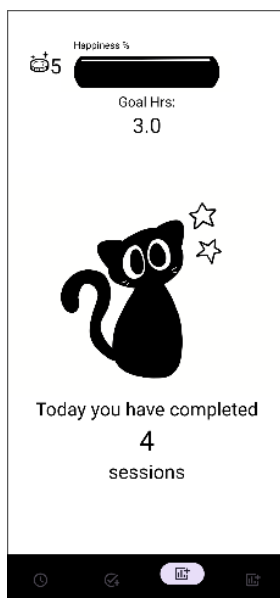
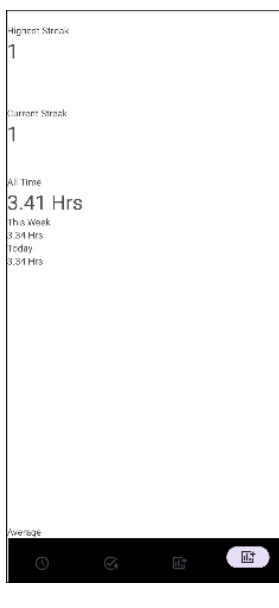
</androidx.constraintlayout.widget.ConstraintLayout>
```

4 Testing

4.1 MainActivity

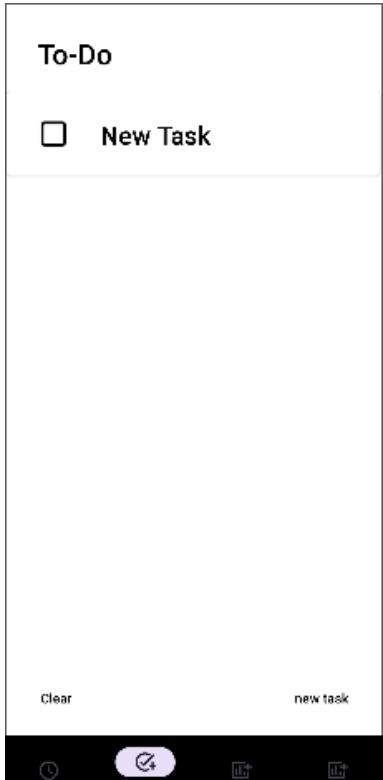
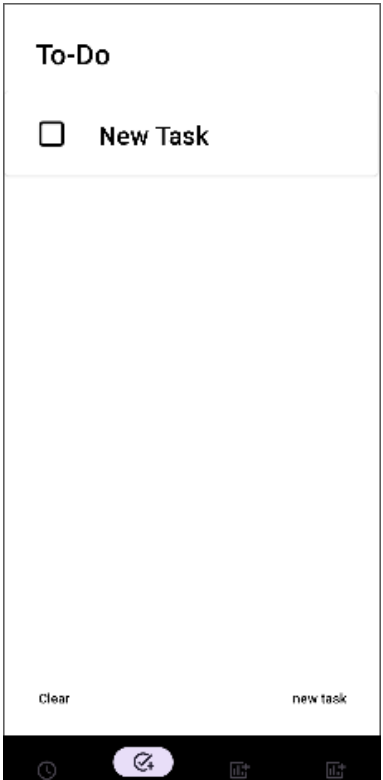
The main activity can display one of 4 fragments when you click on the respective buttons of the nav bar.

Here is what happens when you click on each button once:

Timer	To-Do	Pet	Stats
			
TEST PASSED	TEST PASSED	TEST PASSED	TEST PASSED

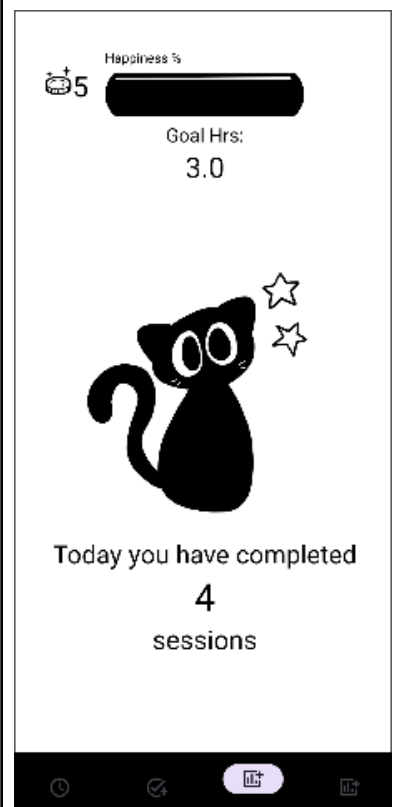
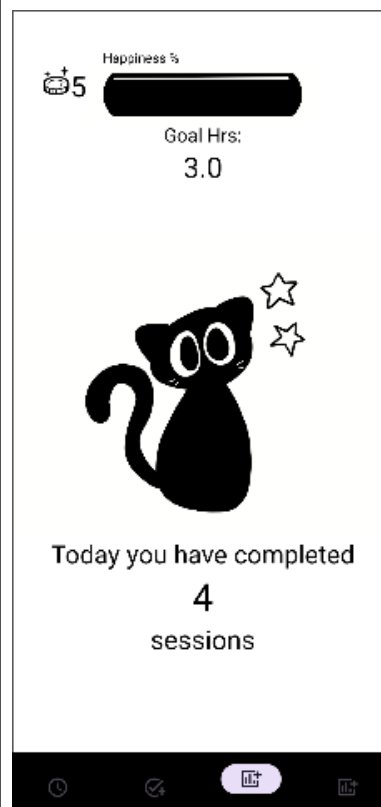
As you can see on the bottom, the purple highlights which page you're on.

Below are tests for what happens when a nav-button is clicked on twice for each page.

Page test	Before	After
Timer PASSED Notes: Seamless refresh		
To-Do PASSED Notes: Seamless Refresh		

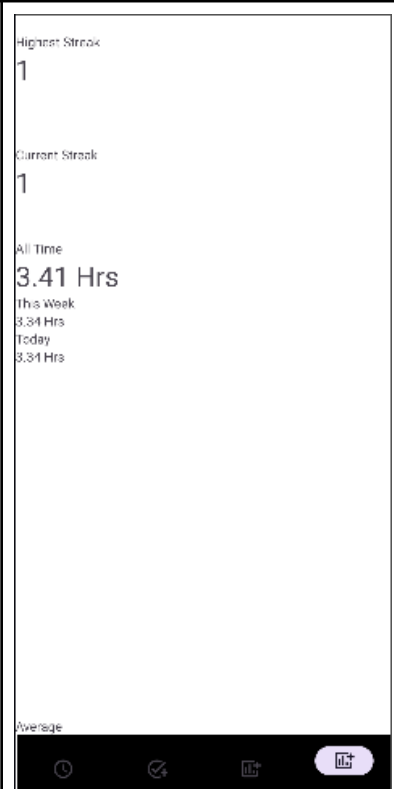
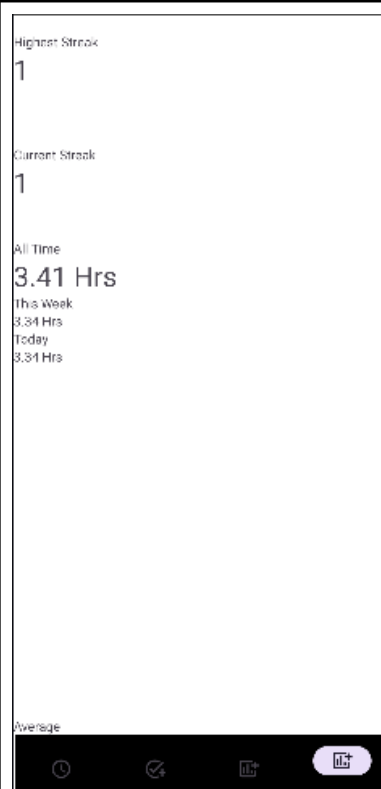
Pet:

Notes: Slight flickering
when loading PNG



Stats PASSED

Notes: Slight flickering
when retrieving all time
value, every other
component seamlessly
refreshes though.



4.2 Timer

4.2.1 Formatting Time

Here I'm testing the ability of the program to take in inputs in milliseconds, and display to the user the time remaining in hours and minutes. I've isolated `formatTime(millis)` and ran tests. Input is time in milliseconds, the output should be in hours and minutes. Y/N signifies whether the test return the expected output.

Input	Output	Y/N
0	00:00	Y
1000	00:01	Y
60000	01:00	Y
61000	01:01	Y
3599000	59:59	Y
3600000	60:00	Y
123456	02:03	Y
59999	00:59	Y
120000	02:00	Y
30000	00:30	Y
90000	01:30	Y
180000	03:00	Y
123999	02:03	Y
86400000	1440:00	Y
123	00:00	Y
599	00:00	Y
1001	00:01	Y
5999	00:05	Y
9999	00:09	Y

All the tests were successful in outputting the correct duration and rounding milliseconds to the nearest second.

4.2.2 Starting Timer

This is a test to start the timer countdown. Once I click on the button, the timer should start to decrement.

Running	Paused
 The 'Running' interface shows a large digital timer at 74:52. Above the timer are three elements: a grey rounded rectangle labeled 'Focus', a pause icon (two vertical bars), and the word 'Break'. Below the timer is a black rounded rectangle labeled 'Pause'.	 The 'Paused' interface shows a large digital timer at 74:23. Above the timer are three elements: a grey rounded rectangle labeled 'Focus', a pause icon (two vertical bars), and the word 'Break'. Below the timer is a black rounded rectangle labeled 'Resume'.

Criteria of Success:

- ✓ Start button becomes pause button
- ✓ Timer text view shows decrease by the second
- ✓ Executes `logSession(minutes)` (indicated by a print statement) onFinish if `isPomodoro`
- ✓ Notification shown
- ✓ Executes `stopPauseTimer()` when `isPomodoro` False

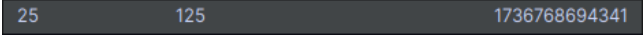
4.2.3 Adding Sessions

I created a button that immediately logs sessions, where x is the number of minutes logged:

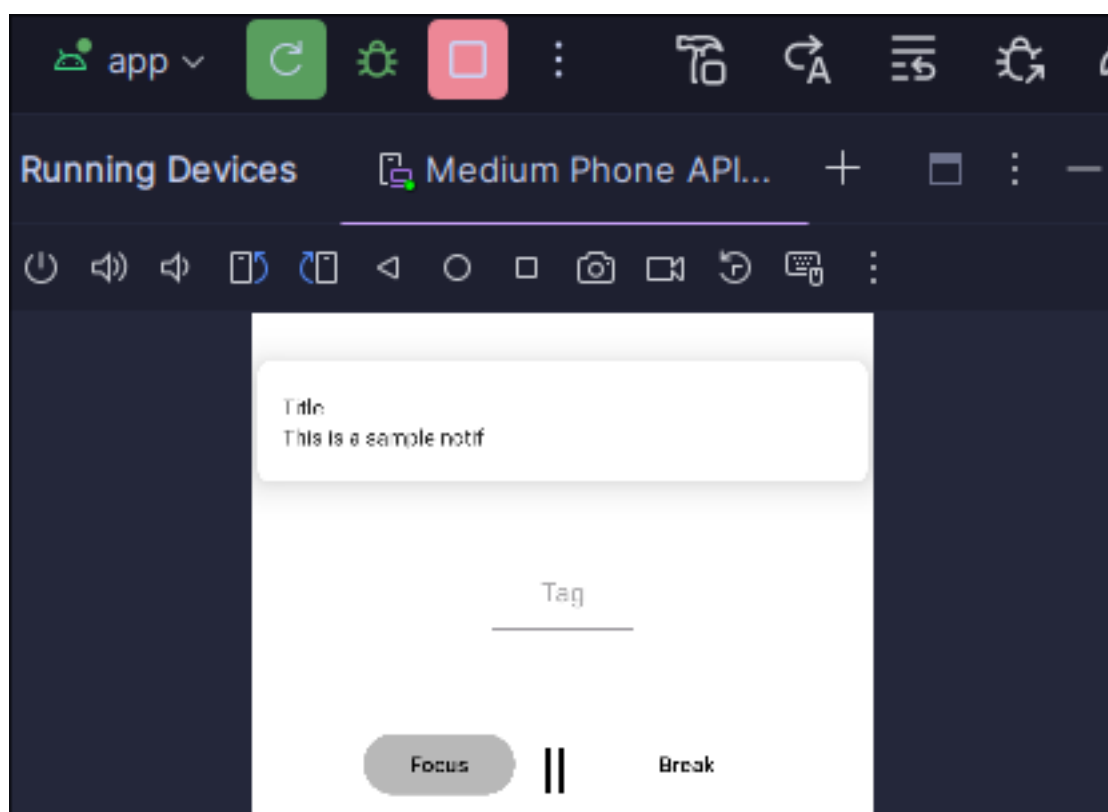
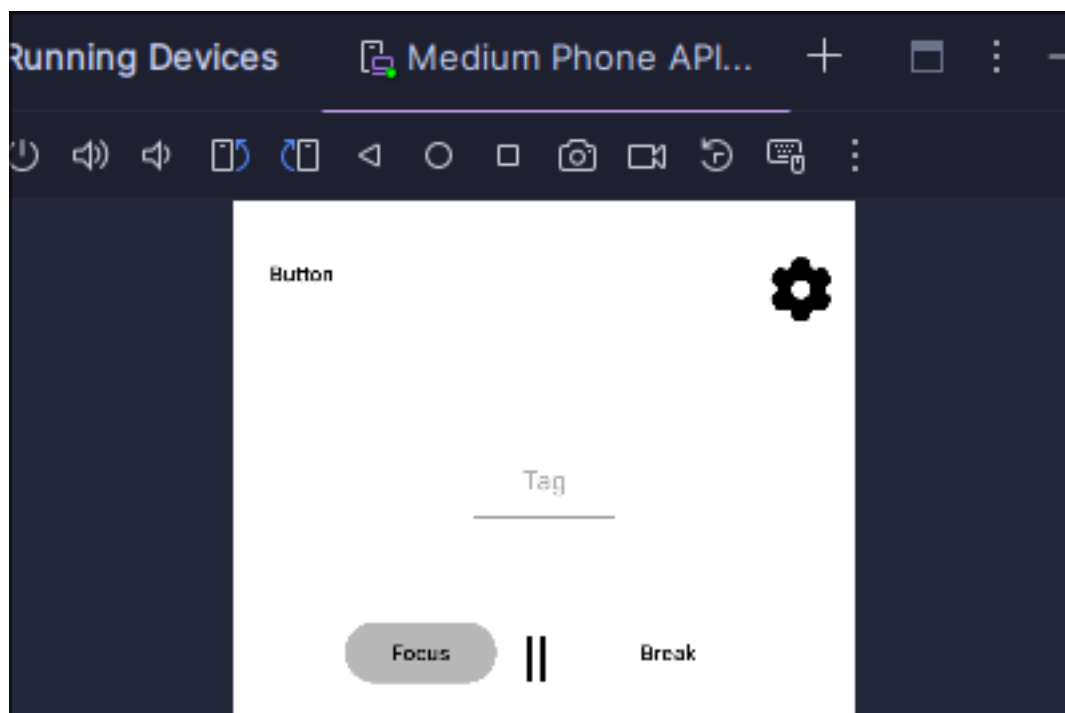
```
button.setOnClickListener {  
    logSession(x)  
}
```

As seen in my Code section, within logSession contains the function addSession. The logic of this function is contained within addSession, as logSession is just a function I created for further procedural abstraction. If the results are valid, then addSession works.

Input x is the number of minutes being logged. The dateTime logged should be the dateTime of the moment I click the button, in milliseconds. Valid is whether the output is expected or not (y/n).

x	Expected output	Screenshot	Valid
1	minutes: 1; dateTime: Mon Jan 15th; Tag: None		y
60	minutes: 60; dateTime: Mon Jan 15th; Tag: None		y
125	minutes: 125; dateTime: Mon 15th Jan; Tag: None		y
45	minutes: 45; dateTime: Mon 15th Jan; Tag: None		y
1500	minutes: 1500; dateTime: Mon 15th Jan; Tag: None		y
3000	minutes: 3000; dateTime: Thursday 6th Feb; Tag: Piano		y
3000	minutes: 3000; dateTime: Thursday 6th Feb; Tag: Guitar		y

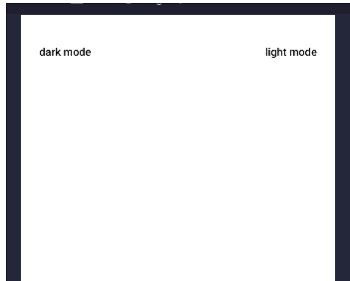
4.2.4 Notifications



When I click on the button that calls `logSession()`, the notification successful comes through. The same outcome occurred 5 times in my 5 test.

4.2.5 Settings

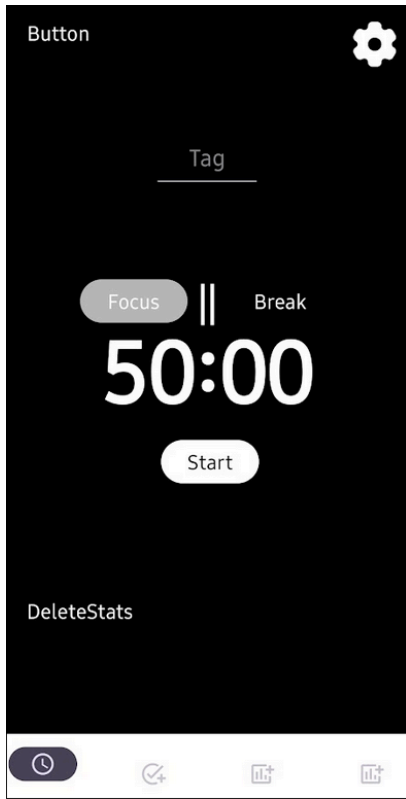
This tests whether the settings activity gets launched or not when I click on the settings button.

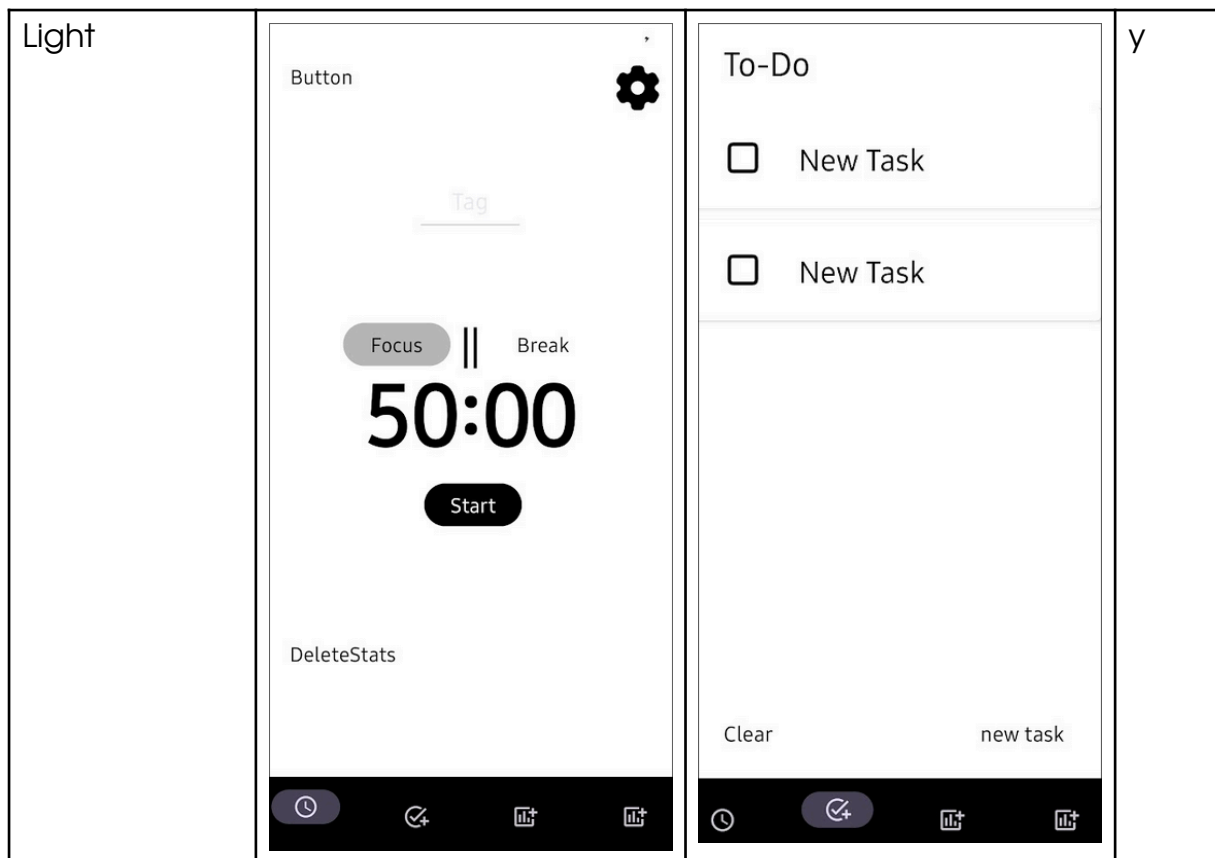
Test	Result	Screenshot	PASS
Click on settings button	Opened up settings activity		Y

In settings, there are 2 buttons to toggle themes. See Themes in 4.2.6.

4.2.6 Themes

This tests the output of “dark” or “light” button being pressed in the themes activity.

Button clicked	Screenshot1	Screenshot2	Valid
Dark			y



4.2.7 deleteStats()

Test	Before Delete	All Time after Delete	PASS
1	13.35	0	Y
2	0	0	Y
3	15.01	0	Y
4	18.35	0	Y

Notes:

- Stats successfully deleted in every trial.
- No crashes experienced when deleting stats, when there were no previously recorded sessions

4.2.8 Updating button appearance & Custom Duration

This is a test of saving a custom duration onto the timer, along with testing the UI/aesthetic functions of the timer being in break and focus mode. The input is always automatically Int, there is no need to test for the case that erroneous data is inputted, because the only way to alter the timer is to use android's "number picker" component. This is the ONLY way possible for the app to take in an input for timer start values.

Button	Screenshot	Pass?
Focus 75 mins		Y
Break 5 mins		Y
Focus 15 mins		Y

Focus 615 mins		Y
Break 23 mins		Y

4.2.9 Skip Timer

This is the test of what occurs when the skip button is clicked. When clicked, stopTimer() should be called.

FD stands for focus mode duration and BD stands for break duration Mode signifies the start mode/the mode the timer is on when skipping. Expected results should be the timer stops and the mode changes to the opposite mode.

Test	Before	After	PASS?
Mode: Focus FD: 615 mins BD: 23 mins			Y

Mode: Focus FD: 15 mins BD: 23 mins	Focus Break 14:58 ▶ Resume	Focus Break 23:00 Start	Y
Mode: Focus FD: 15 mins BD: 25 mins	Focus Break 14:58 ▶ Resume	Focus Break 25:00 Start	Y
Mode: Break FD: 25 mins BD: 15 mins	Focus Break 24:53 ▶ Resume	Focus Break 15:00 Start	Y

4.2.10 Focus/Break button

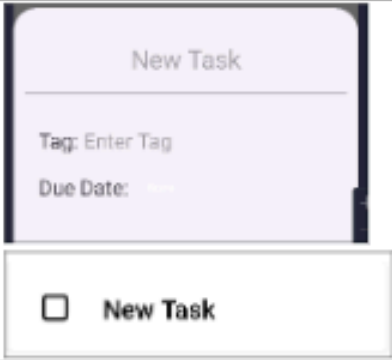
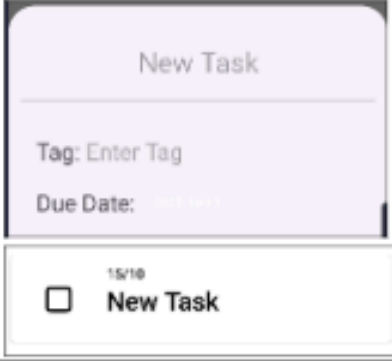
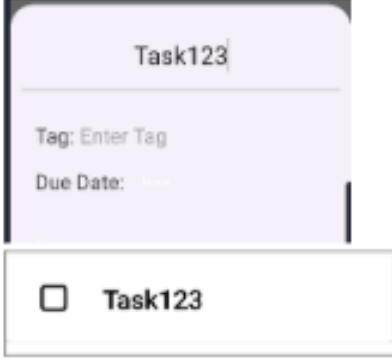
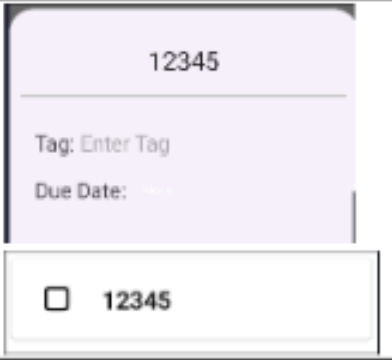
Action	Expected result	PASS
Click on break button while in focus mode	Mode changes to Break	Y
Click on Focus button while in break mode	Mode changes to Focus	Y
Click on focus button while in focus mode	Mode stays in Focus mode	Y
Click on break button while in break mode	Mode stays in break mode	Y

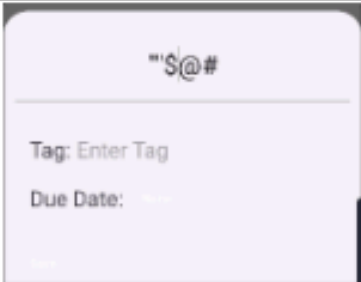
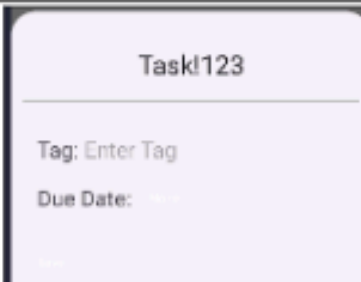
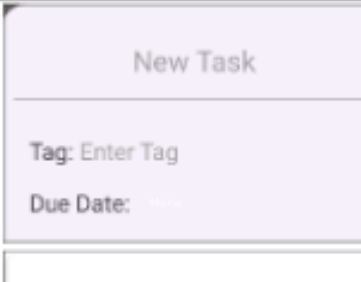
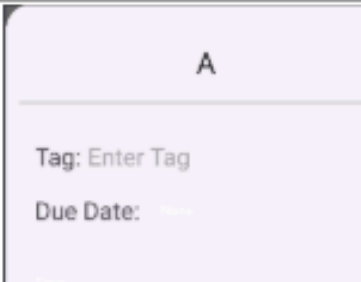
4.3 To-Do List

A majority of tests in this section will be of the taskViewModel class, since I have used that class to be the “wrapper” /procedural abstraction of every single function relating to the To-Do list. For example, in the taskItemDao, I have defined SQL queries to manipulate task properties. Rather than testing the Dao itself, I will test the view model, since the view model inherits every function from every other class I created for this part of my program. The modularity of my program using the view model was implemented for ease of testing.

4.3.1 Add Tasks

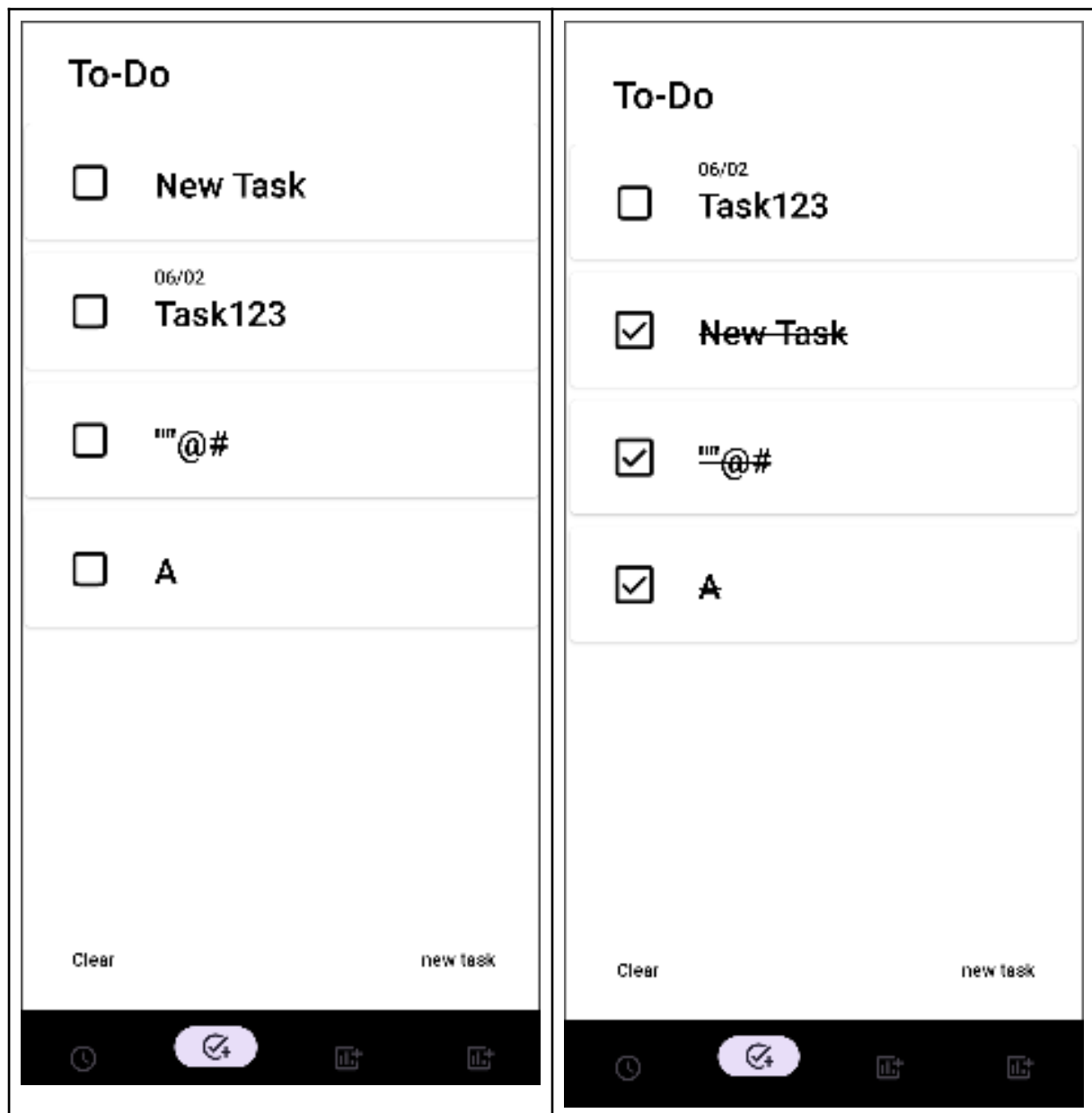
In order to add an item, `saveAction()` must be called, hence this is also a test for `saveAction()`. This test is to ensure any combination characters make a valid name, even unusual symbols.

Name Input	Expected result	Screenshot	PASS
Empty date field	Date field remains empty	 The screenshot shows a mobile app interface for adding a new task. At the top, it says 'New Task'. Below that is a text input field for the task name. Underneath the name field are two smaller fields: 'Tag: Enter Tag' and 'Due Date:'. The 'Due Date' field is currently empty. At the bottom of the screen, there is a white button with a black checkbox and the text 'New Task'.	Y
Past Date (2023-10-25)	Date displays as 15-10	 The screenshot is similar to the previous one, but the 'Due Date' field now displays '15/10'.	Y
Alphanumeric characters only (Task123)	Name displays as "Task123"	 The screenshot shows the 'New Task' form with the text 'Task123' entered in the name field. The 'Tag' and 'Due Date' fields are still empty. The bottom button now says 'Task123'.	Y
Numbers only (12345)	Name displays as "12345"	 The screenshot shows the 'New Task' form with the text '12345' entered in the name field. The 'Tag' and 'Due Date' fields are still empty. The bottom button now says '12345'.	Y

Symbols only "\$@#	Name displays as "\$@#	 Task card showing symbols only: "\$@# Tag: Enter Tag Due Date: View ☐ "\$@#	Y
Mixed characters (Task!123)	Name displays as "Task!123"	 Task card showing mixed characters: Task!123 Tag: Enter Tag Due Date: View ☐ Task!123	Y
Empty name field	Name defaults to "New Task"	 Task card showing empty name field: New Task Tag: Enter Tag Due Date: View ☐ New Task	Y
Single character name "A"	Name displays as "A"	 Task card showing single character name: A Tag: Enter Tag Due Date: View ☐ A	Y

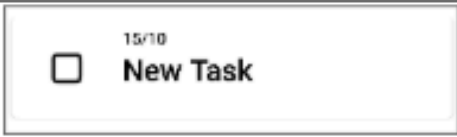
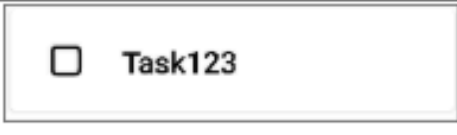
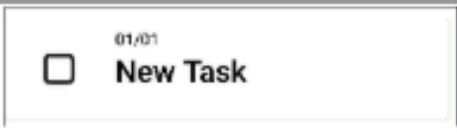
Future date (2050-12-31)	Date displays as 31-12		Y
Current date	Date displays as today's date		Y

Some screenshots of the entire To-do list:



4.3.2 Updating Tasks

I will be using the same values as in addTaskItem() when editing the task:

Name edited to	Expected result	Screenshot	PASS
Empty to Past Date (2023-10-15)	Date displays as 15-10		Y
Change to Alphanumeric characters only (Task123)	Name displays as "Task123"		Y
Change to numbers only (12345)	Name displays as "12345"		Y
Change to Symbols only <u>""\$@#</u>	Name displays as <u>""\$@#</u>		Y
Change to mixed characters (Task!123)	Name displays as "Task!123"		Y
Single character name "A"	Name displays as "A"		Y
Change from future date to past date (2000-01-01)	Date displays as 01-01		Y
Change from other date to Current date	Date displays as today's date		Y

4.3.3 Delete Tasks

Test	Expected result	Result
Delete a single task	Task is removed from the list	Y
Delete the first task in the list	First task is removed, list updates correctly	Y
Delete the last task in the list	Last task is removed, list updates correctly	Y
Delete a task from the middle of the list	Middle task is removed, list updates correctly	Y
Delete multiple tasks in sequence	Each task is removed correctly, list updates after each deletion	Y
Delete all completed tasks in the list	All completed tasks are removed	Y
Delete a task and verify data persistence	Task is removed, changes are saved correctly	Y
Delete a task with associated metadata (e.g., due date, priority)	Task with associated metadata is removed	Y
Delete a task with a long name	Task with long name is removed correctly	Y
Delete a task with an empty name	Task with empty name is removed correctly	Y

4.4 Statistics

4.4.1 Update Streak

Test	Expected Result	Pass?
Last session logged 2 days ago	Streaks reset to 0	y
Last session logged today	Streaks do not increment	n
Last session logged one week ago	Streaks reset to 0	y
Last session logged one day ago	Streaks increment	y

There was an issue in the way I implemented streaks- I did not consider that if a session was already logged today, the streak should not increment if another session is logged in the same day.

Correction:

Before	After
<pre>fun updateStreaks(context: Context) { ... } else { if ((daysApart <= 1) (currentStreak == 0)) { editCurrentStreak(context, true) } else if (daysApart > 1) { // Reset if its been more than ... } } }</pre>	<pre>fun updateStreaks(context: Context) { ... } else { if ((daysApart == 1) (currentStreak == 0)) { editCurrentStreak(context, true) } else if (daysApart > 1) { // Reset if its been more than ... } } }</pre>

Before, I set days apart ≤ 1 when it actually should have been exactly $= 1$.

4.4.2 Update Highest Streak

Test	Expected outcome	Pass?
Set current streak lower than highest streak	Highest streak should not change	y
Set current streak to $==$ highest streak	highest streak does not change	y
Set current streak to 1 higher than highest streak	Highest streak updates to this new value	y
Set current streak to 3 higher than highest streak	Highest streak should update to this value of current streak	y

4.4.3 All time Stats

Test	Expected result	Pass?
Add 1 hour to a fresh installation	1 Hour focused all time	y
Add 3 hours to 20.85 existing hours in the database	All time stats should display 23.85	y
Add 0 hours to 2 existing hours in the database	All time stats should stay at 2	y
Add 0 hours to 0 existing hours in the database	All time stats should display 0	y

4.5 Pet

4.5.1 Goal Hour picker

These tests are to check that the Goal hour correctly formats hours and minutes into hours only.

Input	Expected Result	Pass?
2 Hours 58 mins	2.97 Hours	y
2 Hours	2.0 Hours	y
0 Hours	0.0 Hours	y
115 Hours 32 mins	115.54 Hours	y
1 Hour 27 mins	1.45 Hours	y

4.5.2 Sessions Counter

These tests are to make sure the pet page counts the number of sessions logged today correctly.

Test	Expected Result	Pass?
3 Sessions added	Displays 3	y
0 Sessions added	Displays 0	y
10 Sessions added	Displays 10	y
20 Sessions added	Displays 20	y
13 Sessions added	Displays 13	y
4 Sessions added	Displays 4	y
7 Sessions added	Displays 7	y
1 Sessions added	Displays 1	y
9 Sessions added	Displays 9	y
2 Sessions added	Displays 2	y

4.5.3 Happiness Meter

This checks if the correct happiness value has been calculated.

Test	Expected Result	Pass?
• Current Focus Quota Met • Focused Today	3, full bar displayed	y
• Current Focus Quota not met • Focused Today	2, 2/3 bar displayed	y
• Current Focus Quota met • Not focused today	2, 2/3 bar displayed	y

• Current Focus Quota not met • Not focused today	1, 1/3 bar displayed	n
--	----------------------	---

Error	Fix
<pre>fun happinessChecker(onTrack: Boolean, focusedToday: Boolean): Int { var tempHappiness = 3 if (!focusedToday) { tempHappiness -+ 1 } }</pre>	<pre>fun happinessChecker(onTrack: Boolean, focusedToday: Boolean): Int { var tempHappiness = 3 if (!focusedToday) { tempHappiness -= 1 } }</pre>
In the original, I wrote tempHappiness-+1, meaning the value of happiness doesn't change even when the user hasn't focused today.	It was a typo, so I fixed it by replacing the -+ with -=.

5 Evaluation

5.1 Objectives Met

The following objectives I set previously are colour coded to show whether they've been met or not.

Colour Key:

Objective met	Objective not met
---------------	-------------------

Objective
A timer where users can customize durations
Timers can be paused and resumed -
Include a break timer
Timers can be skipped after a duration. The duration already focused will be logged.
When the session ends, users will be sent a push notification
Each study session is logged, along with the duration of the session
Users can add labels and categorize the activities they focus on e.g studying, playing an instrument
Streak measuring how many days the user has focused consecutively
Users should be able to view the number of sessions completed today, their current and highest streak, and the number of hours focused for today.
User can change app theme
Pets have a happiness meter which increases the more students focus and declines the more students miss their study goal
Happiness meter is related to the goal time that the user sets for themselves
Pets will output randomly selected motivational comments while a user is focusing
Users can create a task
Users can edit a task
Users can check off a task
Users can clear all completed tasks at the same time

Unfortunately I did not implement the pet outputting randomly selected quotes, however more on how I'd implement this along with other possible features for the future can be seen in the following sections.

As a whole, the rest of my objectives were completely and thoroughly met. The app is fully functional and useable, there isn't a single feature contributing to the functionality of the app missing.

5.2 Possible future developments

In this section I'll discuss future developments I could make onto this app, whether it's completing an unfinished objective or the possible ways I could implement potential features.

One objective I missed out was the feature of pets randomly outputting motivational quotes. In the future, I believe this can be implemented the following way:

- Set a textView component above the pet sprite
- Have a list containing various strings of motivational text
- Have a random number generator determine which quote should be randomly outputted
- Each time the pet fragment is opened up, the textView's value should be assigned one of the random quotes determined by the random number generator

You may have seen in my screenshots and technical solution a coins feature, where users gain a number of coins proportional to the time they spent on a session. This was an experimental feature I added, in case I wanted to add a shop in the future.

Below are things I have personally thought of implementing later on in the future.

- External app blocking
 - One way I could go about programming this is detecting whether the device has opened up an app, compare the app's name to a list of blacklisted app, and then obtrusively open an activity immediately, alerting the user of the fact that they're distracted
- Screensaver mode
 - Screensaver as explained in the objectives ensure UI components don't stay in one place for too long, burning in screens. This can be implemented by generating random x and y coordinates that UI components should periodically move to.
- Items/Accessories can be bought and equipped by the pet
 - I would likely implement this by having a database of items, with attributes such as unlocked/not unlocked, and equipped/unequipped. Then, based on the state of these attributed, the respective sprites should be loaded in.

5.2.1 Feedback

Not including myself, I have given the apk to my app to 5 of my peers to use. Although I wished I could have more people test, I had not come across enough people who use android and are interested in focus timers. Below, I've bulleted verbal feedback that I've gotten over the testing period:

Positive

- Aesthetic, minimal UI
- Cute Sprites
- Can easily create and clear tasks
- Glad that completed tasks aren't immediately deleted so that day's productivity can be looked back upon
- The themes match their preferred aesthetic
- Love the cute avatar

Criticisms

- Statistics page could be nicer looking, maybe use black text colour and add icons
- A guide should be included to change the goal number of hours, because I was confused on that for a bit
- A task tracker would be cool
- Probably add like a tutorial page or something
- Add more themes
- More features relating to the pet page, it's largely for aesthetics at the moment
- Tasks should have priorities

I have also conducted a formal survey too, to extract more data. See below:

How easy was it to customise timer durations? 1 = Very hard, 10 = Very easy

Rating	1	2	3	4	5	6	7	8	9	10
Freq.	0	0	0	0	0	0	0	0	0	5

What would you improve about the timer experience if anything?

1. I'd like a "Save Preset" button so I can switch between my default and long-study modes faster.
2. I didn't realize I had to click on the timer to customize it until you told me so I guess add a tutorial
3. Nothing it was super simple.
4. Not really anything.
5. Allow presets for durations cause I use it a lot in pomofocus

Rate the general timer experience out of 5 stars.

Rating	1	2	3	4	5
Freq.	0	0	0	1	4

Which theme did you prefer?

Theme	No. of people who preferred this theme
Light	4
Dark	1
Neither	0

Are there any changes you felt like you wanted to the UI?

1. Not really.
2. The dark mode is kinda ugly
3. I think more themes would be good
4. Animations would make the app more lively
5. Add more themes. Or let the user make a custom theme.

Rate the UI out of 5 stars

Rating	1	2	3	4	5
Freq.	0	0	2	1	1

Did you find the stats page useful? What would you change about it?

1. It's nice that I can see my stats this week, but also I would like to be able to view previous weeks.
2. No
3. Comparing it to the others' all time durations was honestly fun
4. I didn't like the design to be honest
5. It's ok, a bit plain though,

Rate the Stats page out of 5 stars

Rating	1	2	3	4	5
Freq.	1	3	1	0	0

Did you use the task page features?

Answer	No. of people
Yes	3
No	2

Which features did you use on the task page? Check all that apply.

Answer	No. of people checked
Due Date	3
Tagging	1
Checkbox	5
Clearing checked tasks	5

Rate the Tasks page experience out of 5 stars.

Rating	1	2	3	4	5
Freq.	0	0	0	0	5

What would you change about the task page, if anything?

1. Nothing
2. I think a task priority system would be cool
3. I'd like to reorder tasks manually—drag and drop would be great. Also, maybe optional subtasks?
4. Nothing
5. Probably make it so that tags actually show up like maybe add a box on each task item that displays the tag

How much did the pet feature help you stay motivated to study?

Answer	Number of people
Yes, a lot	2
A little	2
Not at all	0

What about the pet feature did you like, and what could make it better?

1. The sprite was really cute but you should add more. Like, for different moods
2. It was very cute but I think there should be more interactions like idk feeding
3. The page looked really nice and aesthetic, but more interactions would be nice, maybe accessories for the pet? Since there's already a coin feature you could add a shop
4. It was cute but I didn't pay too much attention to it. If you added more stuff to do with the pet that would probably have helped
5. The UI was nice but there wasn't enough stuff to do with the pet.

Was the weekly goal feature helpful in keeping you on track?

1. Somewhat. It helped see my progress, but I think adding reminders or notifications when I'm falling behind would make it even more useful.
2. I didn't really use it

3. No
4. I didn't pay TOO much attention to it
5. Yes. It motivated me because I liked seeing the happppiness meter at the max

Rate the Pet page experience out of 5 stars.

Rating	1	2	3	4	5
Freq.	0	0	4	1	0

What did you enjoy most about the app?

1. The timer was simple and worked. That's all I needed. The streaks system was neat too.
2. The pet page
3. The To-Do list was actually very useful
4. To-Do list, it was the most practical part of the app imo
5. The pet

Summary of data:

User Interface

Feedback on the UI was mixed. While some users said it didn't need changes, others found the dark mode less visually appealing or requested more customization options. Several users expressed interest in adding animations and more theme variety, including the possibility of user-created themes. Light mode was overwhelmingly preferred, with four users favoring it over dark mode. UI ratings were mostly mid-range, indicating room for aesthetic and interactive improvements.

Stats Page

The stats page was seen as one of the weaker aspects of the app. Most users found it plain, with limited value beyond seeing weekly progress. Some appreciated the ability to view current stats and compare with others but suggested a feature allowing them to see data of older weeks to make it more meaningful. Ratings for the stats page leaned toward the lower end, with most giving it 2 or 3 stars.

Task Management

The task page was very well received. All users who interacted with it used the checkbox and clear-completed features, with most also setting due dates. Though tagging was less frequently used, suggestions were made to display tags more clearly. Users also expressed interest in more advanced features like task prioritization, drag-and-drop reordering, and optional subtasks. Despite these requests, the task page received unanimous 5-star ratings.

Pet Feature

The pet feature was popular in concept, but several users felt it was underdeveloped. While two users said it helped them stay motivated, others said it was cute but not interactive enough to hold their attention. Suggestions included adding feeding interactions, more sprites to show mood changes, and even a shop system to spend in-app coins on accessories. The pet page received mostly 3-star ratings, indicating users liked the idea but wanted more functionality.

Overall Summary

It's safe to say my app was generally met with positivity, with a few requests and comments about the aesthetics of the app. My friends were able to use it to monitor their productivity throughout the weeks and adjust their habits to meet the goals set. We were even able to have a mini competition to see who could meet the goal first. It was clear that the statistics page was the least popular due to the aesthetics of the interface, so in the future, I will think about improving the design. Moreover, the pet page was undoubtedly the most popular despite feedback suggesting it lacked features. As a result, I'll think about adding more game like features. But I'll explain future possible updates more extensively below.

Some improvements I believe could implement as a result of the feedback are:

1. Increase the aesthetic appeal of my app by adding in a greater variety of themes and colour palettes. Another way of improving it is by creating more sprites for the pet. This can be different colours, types of pets or even separate sprites for different pet moods. Lastly, I should consider animations, for example a simple transition animation between each page, or a celebration animation when the timer finishes.
2. The statistics page should be able to display stats that are specific to certain task groups e.g maths, physics, etc that the user has self-inputted. A few users reported the unattractiveness of it.
3. Include a tutorial page, likely in the form of a fragment that can be re-accessed from the settings page. This tutorial page should include photos of the app along with brief descriptions of each feature on each page.
4. I will try to make the pet page a more significant aspect of my app, by including features such as accessories and food, utilizing the coins feature that already is in place as one user suggested.
5. Another thing my friend thought would be useful is including the ability to assign priorities to tasks. I could implement this by adding a new columns to

the tasks database, stating a priority that's described by integer, e.g 1 is the highest priority.

6. A great suggestion that only came up once was the idea of adding reminders to the app for when users are behind. Similarly to the well-known and successful language-learning app duolingo, which reminds users of their study streak.
7. Statistics of previous weeks should be made viewable to the user. This is definitely within my scope as of now, as I've already written a function that returns all tasks between a certain period of time. I could try implementing this in the future utilizing this function, setting the periods to particular weeks.

5.3 Final Conclusion

The development of my Pomodoro timer app featuring a virtual pet has been largely successful. The core functionality (timing study sessions, tracking productivity, and maintaining a happy pet) has been fully implemented and works as intended. Others found the app helpful in managing their study habits, and even engaging in friendly competitions to meet their goals. The pet feature, despite lacking a planned element (randomized motivational quotes), was well-received and served as a motivating factor for users.

The app's foundation is, according to the surveys and tests I've conducted, incredibly strong, and easily expandable. This is particularly due to the fact that I modularized my app effectively, splitting it into fragments and self-contained classes. Given the positive reception from testers and the clear potential for growth, this app demonstrates both present utility and vast possibilities for further development, allowing me to conclude it a successful project.